# HOUSE RUMMY

The Customisable Rummy Platform for Online Play

# Table of Contents

ANALYSIS: ..... 5
Background and Stakeholders: ..... 5
Current Situation: ..... 5
Proposed Benefits of a New System: ..... 5
Research: ..... 5
Current Systems: ..... 5
Interviews: ..... 11
Objectives: ..... 17
DESIGN: ..... 19
System Overview: ..... 19
IPSO Chart: ..... 21
Level 1 DFD Diagram: ..... 23
Data Structures: ..... 24
Database Structures: ..... 24
Runtime Structures (Backend – Python) ..... 24
OOP Design: ..... 26
Objects: ..... 26
Algorithms: ..... 30
Utilities: ..... 30
Validation Algorithms: ..... 31
Deck Algorithms: ..... 37
Wild Assignments: ..... 38
Ruleset Validation: ..... 40
Database Design: ..... 41
Human Computer Interface: ..... 42
Homepage: ..... 43
Sign-up/Login: ..... 43
Game Lobby: ..... 44
Friends List: ..... 45

GameState: 46
Final Designs: 47
Main Menu: 47
Sign-up: 47
Login: 48
Custom Rulesets: 48
Friends List: 48
Technical Solution: 49
Files & Skills used: 49
Game Logic: 49
Database Layer: 50
Frontend (Routes, sockets, and decorators): 51
Game Logic Layer: 52
Utilities: 52
Card Class: 54
Deck class: 57
Discard Pile class: 59
Hand class: 61
Meld class: 65
Player class: 70
Ruleset class: 73
Validator Class: 80
GameState Class: 86
GameService class: 96
Database Layer: 108
Hashing: 108
DBConnections Class: 109
UserRepository Class: 111
GameRepository Class: 116
MoveRepository Class: 119
LinkRepository Class: 122

DatabaseService class: ..... 127
Frontend Layer: ..... 134
Authentication Decorators: ..... 134
Game Decorators: ..... 135
Socket Decorators: ..... 135
Authentication routes: ..... 137
Game Routes: ..... 140
Game Events: ..... 141
Social Routes: ..... 144
Social Events: ..... 145
Appendix: ..... 148
Glossary: ..... 148
Bibliography: ..... 148

# ANALYSIS:

## Background and Stakeholders:

I run board game nights with a group of 6-7 people. We enjoy a large variety of board and card games. Through these sessions, we observed a weakness with the adaptations of card games, with rummy being a prime example of these issues. This is due to the large number of regional variations of the game, with only few variations adapted into a digital application. Many of these adaptations include mandatory gambling mechanics, which are unsuitable to casual and social games.

This context is to highlight the need for digital solutions that support both rule variations and socialisation.

### Current Situation:

We explored a method to play general card games through Tabletop Simulator, a general-purpose tabletop physics simulator, which allows interaction with various board game pieces, including a deck of cards. We experimented with playing rummy, however due to its open-ended design and a user interface on memorising keyboard shortcuts, it resulted in a steep learning curve.

Setting up the game requires approximately 30 minutes, a similar amount of time went into familiarising yourself with the controls. This level of initial commitment discouraged 3 players from participating.

Furthermore, due to the open-ended nature of the simulator, it does not provide automated rule enforcement. Players are responsible and resolve disputes manually. This becomes problematic quickly, especially when experimenting with different variations. Players felt a need for a “Cheat sheet” document open to reference during gameplay. This impacted the enjoyment, leading to the decision to not continue using the platform for Rummy.

### Proposed Benefits of a New System:

A dedicated customisable rummy platform would benefit users by reducing setup time, minimising rule disputes, and removing the need for external documentation. This will be achieved through rule validation, a clear and accessible user interface, and clear configuration systems for the game.

More broadly, it allows people to share a part of their culture with friends, by introducing them to the different variations of the game, supporting social interactions.

## Research:

### Current Systems:

There are several ways to play Rummy. First is using a physical set of cards. Alternatively, dedicated platforms for Rummy can be used, such as RummyCircle and Rummy Palace which allow for online play. These systems will be analysed in depth to evaluate their strengths and limitations.

## *Physical Cards:*

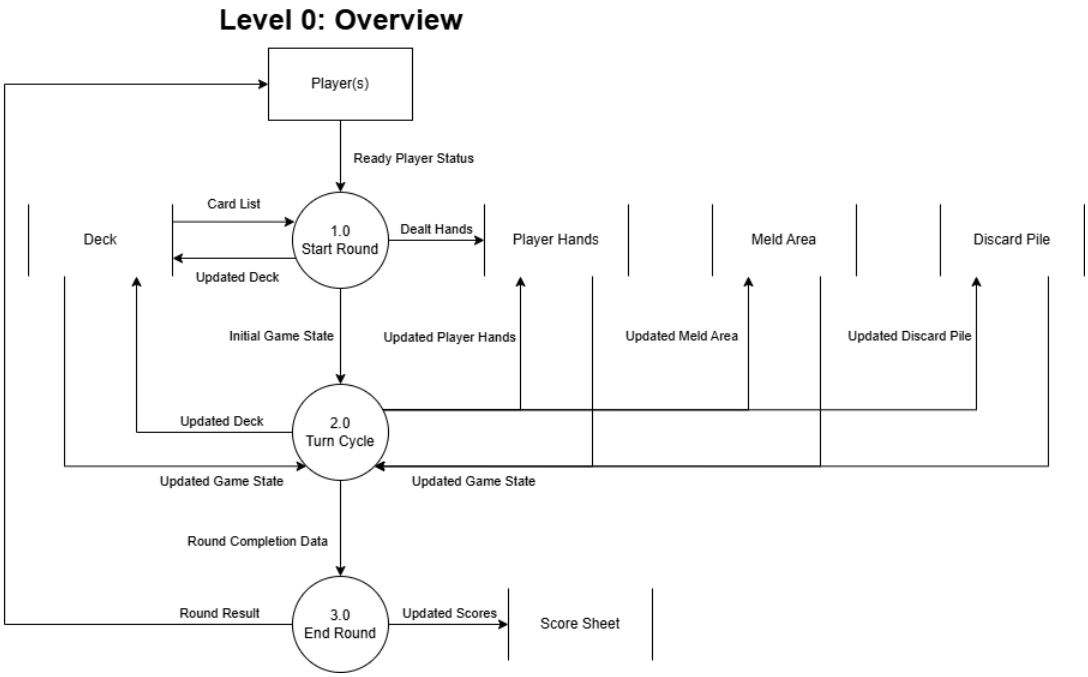
Through analysis of the traditional method of play, which remains the most common, insight can be gained on the core mechanics that must be replicated digitally, while also calling attention to the limitations that can be addressed in a digital platform. For a systematic analysis to take place, IPSO charts and Data Flow diagrams are used to analyse the processes, data flows, and other mechanisms of physical gameplay.

### IPSO Chart:

| | |
|---|---|
| Input(s) | - Deck of cards.<br>- Player actions (drawing cards, discarding cards, laying down melds.)<br>- Player decisions (what cards to discard/meld, when to Lay Down.) |
| Process(es) | - Manual shuffling<br>- Manual dealing<br>- Manually laying off melds<br>- Manually drawing card<br>- Manual rule tracking<br>- Manual calculation of scores. |
| Storage | - Physical deck of cards<br>- Players hands<br>- Laid-down melds<br>- Score sheets<br>- Players memory |
| Output(s) | - Visible board state<br>- Current player states<br>- Scores<br>- Game winner |

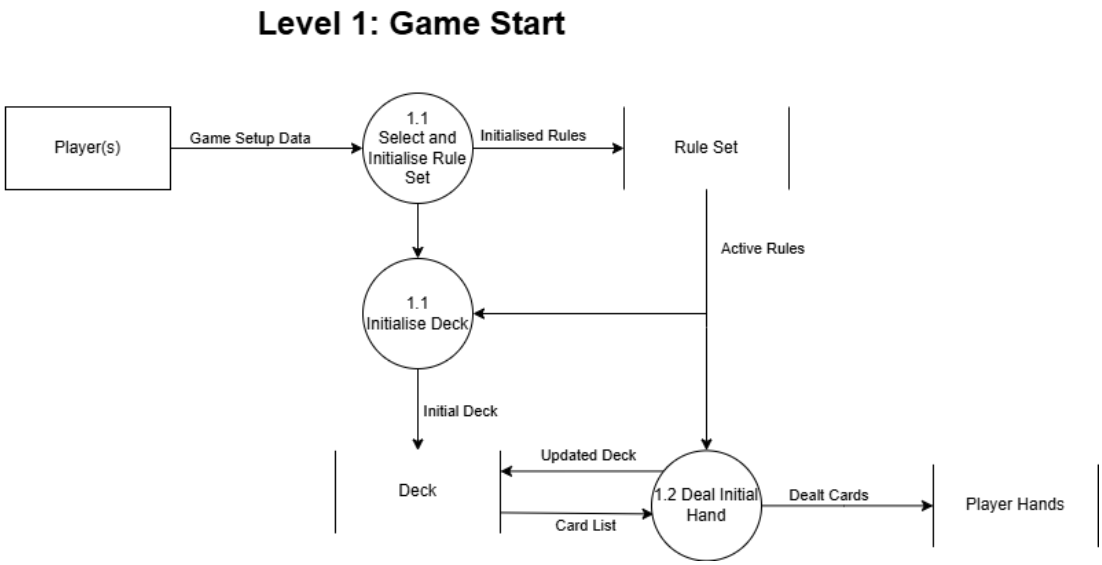
This IPSO reveals how the system relies on the physical actions of players, along with their experience to remember and correctly apply rules to the other players. This dependence on human processes reduces the reliability and consistency of the game, leading to disputes among players regarding the method and state of play, especially regarding different variations. Furthermore, the storage is distributed across multiple physical objects, along with player memory, creating multiple points of failure where data can be lost or corrupted. Manual validation of the multiple data stores, that are in multiple formats, could be a time consuming process. Due to the absence of automated rule enforcement, the players are required to agree on the rules beforehand, relying on knowledge and good faith, rather than automated validations.

## Level 0 - High level Structure:



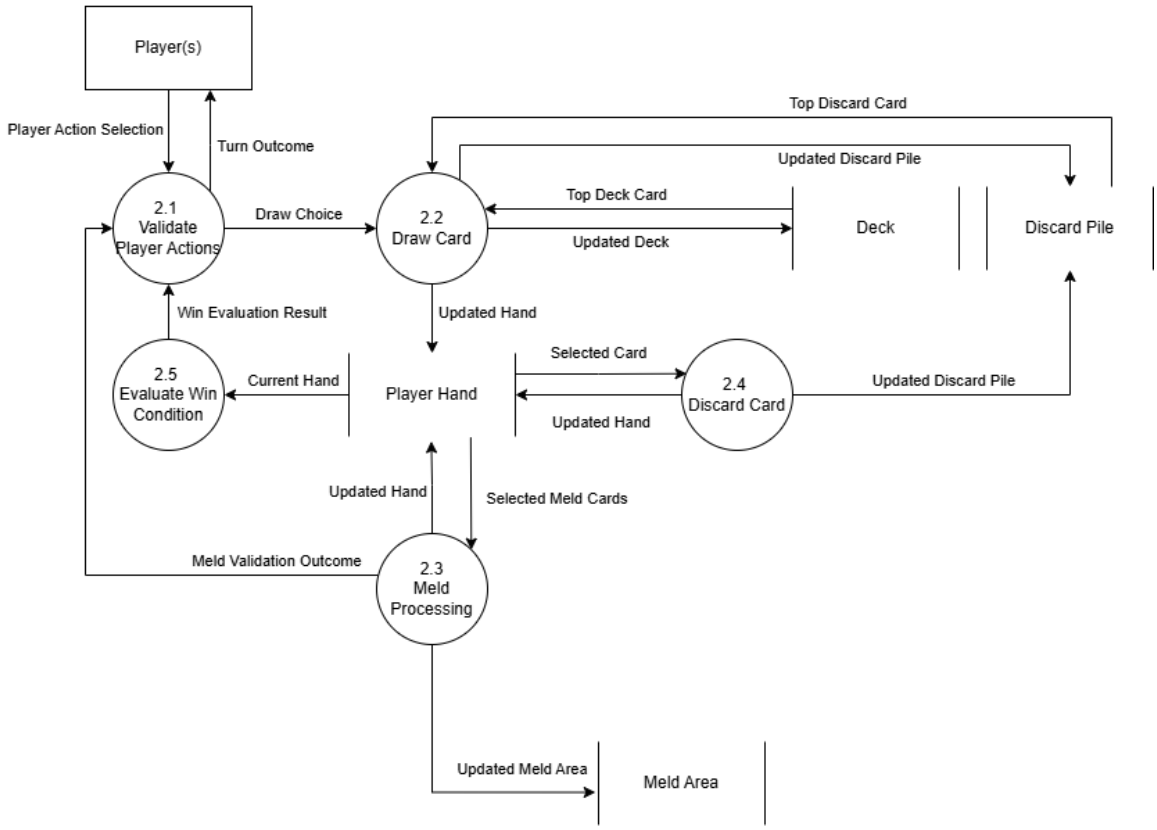

The level 0 diagram reveals the limitations that exist in the high level structure of the system, particularly the lack of a centralised validation process. As such, all data must flow between players, and they must carry out verification of the different processes done. As such, the responsibility for correct execution of rules relies on the players themselves.

## Level 1 - Process Decomposition:




In the above decomposition of 1.0, it is revealed how the rule initialisation works for this system. Players must first agree on rules, Process 1.1 (Select and Initialise Rule Set), which are stored in the players memory (Rule Set data store). This selection creates ambiguity, as there is no persistent storage of the ruleset, and can create ambiguity in the middle of games, due to players misremembering the rules agreed upon. This is a key limitation that a digital system could address by showing the active rules during gameplay.

**Level 1: Turn Cycle**




The level 1 decomposition reveals a substantial amount of hidden complexity within each process. Particularly in the 2.0 decomposition, exposes 5 subprocesses that must be validated manually each turn. Each of these validation steps is a potential point of failure, due to either bad faith actions or human error compromising the agreed upon rules.

A direct example is the Process 2.3 (Meld Processing). As melds are the most complex part of the game when verifying, as there are two different possible melds. Sets and runs have different requirements, (same rank but different suits vs same suit and ascending rank). Due to the players having the responsibility of validation, there are often moments where an error goes unchallenged. This commonly happens because players are distracted or inexperienced, and the invalid meld enters the meld area datastore permanently. In a digital system, Process 2.3 will have an algorithm to automatically check whether a meld is a valid run or set, before allowing the “Updated Meld Area” update to occur.

## Error Handling and Other System Elements:

Through the DFDs, it is shown that there are no error handling flows. As such, when mistakes occur, such as a drawing mistake or an invalid meld that isn't found, there is no algorithmic process to rebuild the game state prior to the error. Instead, players are required to reconstruct the game from memory, leading into disputes and errors.

The lack of error handling is exemplified in the absence of any ‘game history’ datastore. While the score sheet datastore tracks results, all intermediate data is lost when a new game is started. Therefore, a digital implementation could add a data store for game history/move sequence, enabling game replays and the ability to analyse games after their completion.

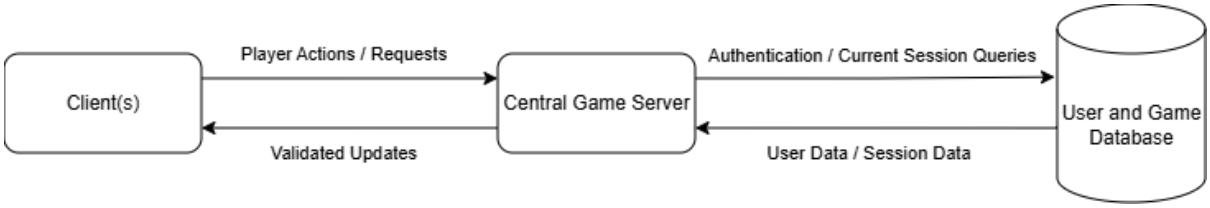
### Player Experience and Social Dynamics:

Despite the many limitations, through the dependency on manual processes, the system offers tangible interactions between players creating a strong sense of social engagement. The personable quality inherent to physical cards is a primary reason that this remains the most popular system for playing the game. However, due to the implicit nature of the rules, with players often assuming a shared understanding, a barrier is created to new players. This can be seen when a dispute occurs, as a new player who has not internalised the rule set cannot distinguish between good and bad faith behaviour. Through a digital system you could make all rules explicit rather than assumed, even when using custom rules and variations, while also allowing social engagement to take place by using other programs for voice chat, and other methods of communication.

## *Online Rummy Platform (CardzMania):*

CardzMania is an online card game platform, which allows for players to play a large variety of card games, including rummy. It offers many variations of card games, along with some customisable options. It also provides features such as game history and leaderboards. Throughout the following section, analysis will be done on CardzMania's system architecture, user management, and game customisation abilities. Understanding of how this platform addresses limitations in the physical card system will also be pertinent.

### System Architecture:




As shown above, CardzMania seems to operate on a simple client server model, where clients send actions and requests to a central game server. This central game server validates moves against the game logic loaded into memory at session initialisation, reducing cheating. Additionally, this ensures that all clients are synchronised to the current game state, as the updates are only received by clients when they are validated, and there is only one source for the game state.

However, due to the client server model, there exists a major bottleneck when it comes to scalability. Due to the centralisation of rules validation, the server load increases with the player count, as it must perform continuous rule validation for all current sessions. According to CardzMania, they have 238,000[1] registered users, where if even 1% play rummy at one, (600 game states with 4 players), the server must handle continuous validation for the sessions, representing a significant load. Another issue is the fact that it acts as a single point of failure, where if the server has an issue, all games stop. To handle this load, CardzMania likely uses horizontal scaling, with multiple game servers around the world, allowing for both load balance and reduced latency for international clients, while also removing the single point of failure from the system.

A peer-to-peer approach could distribute the load as well as eliminate the single point of failure. However, this would introduce extra complexity and latency, as the clients must instead

act as the validators. For a casual card game platform, where anti-cheat is critical, and players expect reliable gameplay, a client server model is the appropriate choice despite the more expensive infrastructure required.

## Authentication and User Management:

Account creation is used as well, requiring entering a username, password, and email address. To keep the system secure, passwords are hashed before storage, such that if the database is compromised, plaintext passwords are not retrievable. The email authentication seems to be a way to mitigate spam bots from attempting to overwhelm the server with fake accounts, requiring them to be verified with an email address. When a game session begins, players are validated against the user database, where either they are compared to a user-id if signed in, or a temporary session key if using a guest account. This system ensures that both quick and easy play with a guest account is possible, while also allowing the more dedicated players to keep track of their statistics as they play the game. This dual authentication balances user acquisition and retention, as guest accounts minimize friction for casual players, not requiring them to do anything other than open the website, while the

## Data Persistence and History:

Through the fact that clients can view previously played games with no limit, therefore the game database holds every game that is played. The most obvious method that this can be done is record all the player actions as a sequence of moves, as well as the rule set used. When viewing a previous game, it simply plays those moves in order in a private game session. This approach is relatively storage-efficient, as it only stores the move sequences. This contrasts with storing the full game state after every move, which would be far more inefficient, especially for longer games. However, in longer games, going to a specific point might cause noticeable delay, due to the system having to replay all prior moves. A system could be implemented where it saves the state of the game every so often, perhaps every 20 moves. This would both be storage efficient, while not sacrificing replay performance.

## Game Customisability and Rules Flexibility:

In general, CardzMania offers customisability in terms of the types of games you are able to play on the site. It handles this by offering pre-coded variations of the game that get loaded into memory at game state initialisation. These presets can then be fine-tuned further, via a few options, such as whether jokers are in play, point limits, and hand sizes. This approach offers the player some limited customisability while maintaining rule validation. This offers a much higher amount of customisability compared to other digital systems, while still taking advantage of the digital format, by using a centralised validation system.

However, the preset based approach imposes significant limitations on customization. While specific features can be toggled, more fundamental game systems cannot be altered, such as the minimum card limit for melds, or whether completed melds get added back to the draw pile. As such, this system particularly limits players who already have established house rules are unable to replicate their experience. Instead, they must settle for a “good enough” preset and options combination, compromising their experience and reducing the system's appeal for experienced players with established preferences.

In contrast, physical card play offers all the flexibility through its lack of a centralised validation system, as the players are able to agree to any modification, even in the middle of a game, no matter how unconventional. CardzMania trades this flexibility for a simpler system that is more consistent and has better cheat prevention.

An opportunity to improve this system would be to allow a truly customisable system, that allows the player to modify core rule components, such as meld size minimums, wild cards,

turn structure, while maintaining the automated validation. The major challenge for this system would be how it can validate arbitrary rules consistently. CardzMania sidesteps this problem by using presets, but this represents an opportunity for innovation for a truly customisable platform.

## Interviews:

Interviews were conducted with potential users of a customisable rummy platform. All participants regularly play cards physically and have experience with digital adaptations. Therefore, they are well suited to identify the common issues and limitations with these adaptations.

Across the three interviews conducted, the questions focused on several key areas: learning curves and complexity, customisation, frustrations with current card games, social elements, and User Interface features. These interviews were conducted to gather qualitative user data, such that they may be analysed to identify the problems and in previous attempts.

The findings from these interviews are used to inform the design of the proposed solution, focusing on addressing the concerns that users have expressed.

### *Interview Transcript 1: Gunmay Bhalla:*

The interviewee is a close friend of mine, and regularly plays a large variety of games, particularly multiplayer games with friends. As well, he frequently has a deck of cards on him and plays physical card games regularly.

This interview was done over a google meet on the 13th of September 2025.

Faisal Mustafa: Question one (1): Do you ever feel like you'd like to play a quick game with friends but don't have enough time to learn a completely new set of rules? Any particular game you learned recently that took much time to learn? How was the learning experience?

Gunmay Bhalla: Yes, 100%. Especially when I'm recommending a game and I know that there's a certain part where we are all going to have fun, but you know the time it will take for them to get there is just not worth it. Personally, I find that games with a lot of mechanical skill like Team Fortress Two or games with a large volume of information like dungeons and dragons take the most time to learn. I still feel like I still haven't properly learned either.

Faisal Mustafa: Question two (2): How often do you play card games with friends or family, either in person or online?

Gunmay Bhalla: I play in person with friends reasonably often, as it is the easiest way I find to get everyone in the mood to banter and talk. I usually carry a deck of cards with me most of the time though, which I acknowledge is not exactly normal.

Faisal Mustafa: Question three (3): When playing card or board games, do you usually introduce or use house rules? If yes, please give an example.

Gunmay Bhalla: Yeah, we obviously do have house rules for something. We just end up doing stuff and it becomes a house rule. Most obviously UNO and monopoly, but also when we play Dungeons and Dragons, we often just change the ruleset on a whim. Like allowing strength to intimidate as well as charisma.

Faisal Mustafa: question four: Would you be interested in an online card game platform that allows you to create and apply your own house rules in private lobbies?

Gunmay Bhalla: Yes, I would be interested in that. Especially because so often, as the version of the game you know isn't available. Though it should be a sort of “hidden” option. It would just distract from the game sometimes.

Faisal Mustafa: question five (5): What are your biggest frustrations or challenges with existing online card and board games (e.g., complexity, lack of features, poor matchmaking, paywalls)?

Gunmay Bhalla: Often the rules are not the classic version of the game. Someone will make some one specific version and if it's not what you want to play, then screw you. Secondly is that for most card games the good versions are paywalled and you can't access them.

Faisal Mustafa: Question six (6): How important are audio and visual effects (animations, sound effects, themes) to your overall enjoyment of an online card game?

Gunmay Bhalla: I feel it adds a lot to the immersion of the game. Otherwise, you just have a lot of pixels on a screen being teleported around. I find it very hard to track what’s happening without some sort of sound or visual effects. It just makes it difficult to play I feel.

Faisal Mustafa: Question seven (7): While playing online card games, what information is most important for you to see on screen (e.g., cards in hand, opponent moves, turn order, score)?

Gunmay Bhalla: I think it’s better when there is less information that the computer gives you easily, but all the information that is there should be displayed. For example, it should show the opponent's hand, like the back of it, and the number of cards in the hand should be accurate, but it shouldn’t show “x number of cards” underneath it. Like if someone has eight (8) cards, it should show that in the graphics, but it shouldn’t have like eight (8) cards in opponents hand written in the UI. It’s a double-edged sword because you don’t want someone to lose simply because they missed some information because they aren’t used to playing virtually. Maybe as an accessibility feature? Like a cheat sheet you can bring up while playing or something? Something like turn order though would need to be very explicit.

Faisal Mustafa: Question eight (8): Which devices would you most likely use to play an online card game? (e.g., smartphone, tablet, laptop, desktop)

Gunmay Bhalla: I think that desktop or laptop would be the one I’d use most often. Mostly because the only time I would use a phone or a tablet would be when I’m out and about, where I would just play with a real deck instead. Plus, card games are quite long in general.

Faisal Mustafa: Question nine (9): If you had to choose, which would you prefer:

· Online matchmaking with strangers. Why?

· Private lobbies with friends. Why?

Gunmay Bhalla: It depends on the type of card games you are playing. For example, poker I would say would prefer having online skill-based matchmaking, as it is a competitive game. But, for more communal games? I would prefer private lobbies, as the games are often very long and most fun when you are with your friends and able to banter.

Faisal Mustafa: Question Ten (10): How often do you play card games online, and what websites or applications do you use.

Gunmay Bhalla: I don’t often play card games online, but when I do it's usually on a website that hosts lots of flash games, that have some options for card games. The issue is that for most card games, like bluff or goldfish, there aren’t good online versions. If there are any, they are paywalled and not exactly worth it.

## *Interview Transcript 2: Alan Golovanov:*

The person I am interviewing is a colleague of mine, Alan Golovanov.

This interview was done in person on the 24th of September 2025.

Faisal Mustafa: Question one (1): Do you ever feel like you'd like to play a quick game with friends but don't have enough time to learn a completely new set of rules? Any particular game you learned recently that took much time to learn? How was the learning experience?

Alan Golovanov: Definitely, most games have a huge learning curve. The skill ceiling is too high most of the time, which completely eliminates the ability to play for most players due to the number of players who have been playing for far too long.

Faisal Mustafa: Question two (2): How often do you play card games with friends or family, either in person or online?

Alan Golovanov: Not as often as I would like, mostly due to how long it takes to set up the game. You know, sorting the deck, dealing the cards, teaching everyone the rules. It just takes a while. Also getting everyone together is just a massive pain.

Faisal Mustafa: Question three (3): When playing card or board games, do you usually introduce or use house rules? If yes, please give an example.

Alan Golovanov: Definitely, definitely. I think house rules definitely make the games more fun. If it is implemented right, they'd definitely change the game in a more positive way. The main time I add house rules is for games like UNO, stacking cards and things like that.

Faisal Mustafa: Question four (4): Would you be interested in an online card game platform that allows you to create and apply your own house rules in private lobbies?

Alan Golovanov: Yeah, definitely. Uno does this, but not well. It's definitely hard to play and use the custom rules. They definitely propagate their own idea of the game, which kind of shows that most games nowadays have a bias towards their own creator's opinion. And I think that the players should be able to decide what the rules are.

Faisal Mustafa: Follow-up: If I may ask, what version of Uno are you talking about?

Alan Golovanov: Uh, the android one, you can just download it. There are some changes you can make to the rules, but most of the time it's just variations of rules they made. Not house rules.

Faisal Mustafa: Question five (5): What are your biggest frustrations or challenges with existing online card and board games (e.g., complexity, lack of features, poor matchmaking, paywalls)?

Alan Golovanov: Um, again, a big learning curve. And, um, just overall, I think the UX is a big issue. Uno lags a lot. I'm using Uno as my main example here. Uno lags a lot. It needs a specific OS to run, which limits people from being able to access it. Plus, there's just not a lot of options overall. Yeah, lack of features is what I would say.

Faisal Mustafa: Question six (6): How important are audio and visual effects (animations, sound effects, themes) to your overall enjoyment of an online card game?

Alan Golovanov: The visuals are definitely important. Like, I think just the basics they have to like function well, more than look good. I just want it to be smooth. But audio, I would say for audio… as long as you get a good sounding audio that's satisfying to your ears, you don't need to invest into a lot of audio, you know? It's important, but not that important to enjoy the game.

Faisal Mustafa: Question seven (7): While playing online card games, what information is most important for you to see on screen (e.g., cards in hand, opponent moves, turn order, score)?

Alan Golovanov: The most important information I need to know? I'd imagine it would be nice to know how many cards are left in the deck, you know? For some games it's important to know, it's just to have a feel for it. It has to be implemented in a way that's visually pleasing as well, not just a giant number, an implementation so that you can somewhat see them, it's specific to the game. I think turn order would be important to know, as well a s obviously the number of cards in people's hand.

Faisal Mustafa: Question eight (8): Which devices would you most likely use to play an online card game? (e.g., smartphone, tablet, laptop, desktop)

Alan Golovanov: Probably a laptop or a phone. Realistically, a laptop or desktop, because it's much easier to access. For a quick game, I'd like to be able to just click out of my tab that I'm working on, play a game with my friends, and call it a day, get back to work.

Faisal Mustafa: Question nine (9): If you had to choose, which would you prefer:

· Online matchmaking with strangers. Why?

· Private lobbies with friends. Why?

Alan Golovanov: I'd say a mix of both, you know? Most probably I would use private lobbies more because with card and board games, most of the enjoyment comes from talking with your friends as you play the game. I would use online matchmaking sometimes, just because if I'm bored it's convenient, but otherwise I would use private lobbies.

Faisal Mustafa: Question ten (10): How often do you play card games online, and what websites and applications do you use?

Alan Golovanov: I've answered this a bit. I play Uno sometimes on my phone. It's comfortable on my phone, easy to set up. Adding a friend is so annoying though. It takes so long. You have to send a link to the person instead of just searching for a username, because the usernames are always so long.

## *Interview Transcript 3: Abiel Tenorio:*

The person I am interviewing is a friend of mine, Abiel. He is no stranger to complex card and board games, and equally so for weird online versions, as an avid dungeons and dragons player. As such, he is used to more demanding and complex initial user experiences.

This interview was done in person on the 25th of September 2025.

Faisal Mustafa: Question one (1): Do you ever feel like you'd like to play a quick game with friends but don't have enough time to learn a completely new set of rules? Any particular game you learned recently that took much time to learn? How was the learning experience?

Abiel: No. Most of the time me and my friends have enough free time where the learning curve doesn't really matter. We can just hop on and spend a whole day learning without it really affecting anything. I think the strongest learning curve I've gone through recently is the card game Magic: The Gathering. Just because it is such an old game with so much to learn.

Faisal Mustafa: Question two (2): How often do you play card games with friends or family, either in person or online?

Abiel: I play a bit, not as much as I want to, but I would love to play more card games with my friends or family. I would most of the time like to play in person, but most of the time that's not possible.

Faisal Mustafa: Question three (3): When playing card or board games, do you usually introduce or use house rules? If yes, please give an example.

Abiel: Most of the time yes, such as with UNO, with 07 rulings and stacking. Monopoly as well, with how you can buy land and houses.

Faisal Mustafa: Question four (4): Would you be interested in an online card game platform that allows you to create and apply your own house rules in private lobbies?

Abiel: I feel like I would like it, as I like customizing things in general. User customizability is good, but if I was so concerned with it, I would just use something like Tabletop Simulator where it has infinite customizability.

Faisal Mustafa: Question five (5): What are your biggest frustrations or challenges with existing online card and board games (e.g., complexity, lack of features, poor matchmaking, paywalls)?

Abiel: I don't have many, mostly because I don't play many online card games or board games, except Dungeons and Dragons. Maybe if I was to point to something it would be the friction of actually starting to play. Dungeons and Dragons has this issue if you try to play with an online tabletop.

Faisal Mustafa: Question six (6): How important are audio and visual effects (animations, sound effects, themes) to your overall enjoyment of an online card game?

Abiel: For an online card game, I wouldn't rank the sound effects and such that highly, because it doesn't change much in the end. It's not like a horror game where the atmosphere is incredibly important.

Faisal Mustafa: Question seven (7): While playing online card games, what information is most important for you to see on screen (e.g., cards in hand, opponent moves, turn order, score)?

Abiel: For a card game, it'd be just the basic information of the game. Turn order, the steps in a turn, what can be played, what can't, score, et cetera.

Faisal: Question eight (8): Which devices would you most likely use to play an online card game? (e.g., smartphone, tablet, laptop, desktop)

Abiel: Either a desktop or laptop. Maybe a tablet? Not a smartphone or a console though, it just isn't made for that.

Faisal Mustafa: Question nine (9): If you had to choose, which would you prefer:

· Online matchmaking with strangers. Why?

· Private lobbies with friends. Why?

Abiel: It depends on the game obviously. For most games I would choose online matchmaking because it allows more progression and consistent playing. There are a lot of games though that would be better with friends, especially if it is more than 2 people.

Faisal Mustafa: Question ten (10): How often do you play card games online, and what websites and applications do you use?

Abiel: No, I don't usually play card games online. The most I have is Magic: The Gathering Arena and some uno on random websites.

## *Interview Summaries:*

From the interviews, some potential users with different desires are revealed. Gunmay frequently plays using physical cards, Alan is focused on convenience, while Abiel is more of an enthusiast, willing to play more complex games.

More specifically, an issue identified by the interviewees was the lack of flexibility in existing online card games, especially regarding rule variations. As Gunmay put it, “the version of the game you know isn't available”. As well, Alan describes the options given to him in Uno were "just variations of rules they made. Not house rules.".

However, users expressed concern that could negatively impact the learning curve of the game, especially for casual players. Gunmay suggests customisation as a “Hidden option”, to avoid turning away casual players. Abiel instead mentions that players who want extreme customisability will use a platform such as Tabletop Simulator.

Setup friction is another issue that was experienced by multiple people is the setup friction. Alan expressed this for physical cards, "sorting the deck, dealing the cards, teaching everyone the rules". As such, a digital platform must provide a streamlined game setup and clear explanations of the rules to minimise the friction.

In terms of platform preferences, all interviewees answered that the primary platform they would use would be a desktop or laptop, citing the longer sessions present in online games, as well as having better interfaces compared to mobile devices. However, they all expressed technical frustrations with the current systems. This included performance issues, along with platform dependency issues, such as when Alan mentioned “Uno lags a lot. It needs a specific OS to run, which limits people from being able to access it.”. Further, Alan expressed annoyance with the friending system in UNO, where “Adding a friend is so annoying...”, as it requires a link be sent to the other client, instead of it working in the app.

As well, social experience was highlighted to be particularly important for this genre of game. All the interviewees called attention to the importance of social interaction in card games, as Alan puts it, “most of the enjoyment comes from talking with your friends as you play the game”. Furthermore, all interviewees preferred private lobbies over public matchmaking. As such, easy player interaction must be a focus of the system. With streamlined social features, instead of a competitive ranking system.

In terms of the user interface and experience, the interviewees expressed a want for information to be displayed implicitly, rather than explicitly through text. As Gunmay mentions, "it should show the opponent's hand, like the back of it, and the number of cards in the hand should be accurate, but it shouldn't show 'x number of cards' underneath it." The reason this method was preferred was to maintain the tactile feel of physical cards, requiring players to observe and still do some parts manually.

However, it was also mentioned that this could be a slight barrier to players unfamiliar with digital card games, as they could miss digital information. As such, a solution given by Gunmay

was "An accessibility feature? Like a cheat sheet you can bring up while playing or something?". However, it was also expressed that some information would need to be explicit, such as turn order.]

Regarding animations and sound effects, there was a spectrum of answers given, with Gunmay stressing its importance, "I find it very hard to track what's happening without some sort of sound or visual effects. It just makes it difficult to play.". However, Abiel and Alan considered it less critical, saying that it “doesn’t change much in the end”.

# Objectives:

Through the current system analysed and the interviews conducted, clear objectives have been identified such that the game is a minimum viable product.

## Central Validation System:

In order to address the issues found in the physical card system, where players must rely on memory and their interpretation of rules, creating disputes over validity, a central validation system would be used. This is supported by the interview findings, where Gunmay highlighted the ned for reference material during gameplay, suggesting that players, especially new players, struggle to track rules consistently. As such a system that programmatically enforce rules will reduce ambiguity and prevent invalid actions from entering the game state.

*Performance Criteria:*

1. Perfect accuracy in detecting the validity of moves during testing
2. The system can handle all standard rummy rules, while allowing for custom rule configurations that can still be used to validate moves.
3. Error messages must be present when a move is invalid, such as "Cannot meld 7♠-8♥-9♠: mixed suits"
4. Rule Validation must occur within 100ms per move

## Client-Server Network Architecture:

As found in the analysis of CardzMania, a client-server system is the most efficient system for communication. This is supported by the interviews, where all users highlighted the need for private lobbies, requiring a constant synchronised game state. A client-server model ensures a single source of truth, so all players receive identical updates and gameplay remains consistent.

*Performance Criteria:*

1. All clients must be synchronised to the updated game state within 800ms
2. The system supports multiple concurrent game sessions.
3. Disconnected clients can reconnect within an allotted time without data loss.

## Account Creation and Authentication:

Through the analysis of CardzMania, it is revealed that users want to track their games. As well, all interviewees expressed a want for private lobbies and friends lists. This is reinforced by Alan’s frustration with existing systems where adding friends is unnecessarily difficult. Therefore, account creation is required to enable persistent user data, simplify social interactions, and support friend lists.

*Performance Criteria:*

1. Account creation completes in under 30s for the user.
2. Passwords must be hashed and must be done quickly. (Under 500ms)
3. Users can search and add friends through usernames.

## User Interface Design:

Analysis of the interviews identified the preferences for UI design, a balance of implicit and explicit information. Gunmay preferred information to be shown visually rather than textually, with optional aids such as a “cheat sheet”. This is compounded with Alan’s emphasis on smooth and clear UX and Abiel’s highlighting that important information such, such as turn order, must be as clear as possible. Therefore, the interface should balance clarity with simplicity, while supporting accessibility features.

*Performance Criteria:*

1. The default UI should display information implicitly
2. Optional Accessibility toggle to be done in 2 actions
3. Turn order and other critical information to be clearly visible

## Game Rule Customisation:

A system to modify the ruleset granularly had been shown through the interviews conducted, with Gunmay raising a complaint that “the version of the game you know isn’t available”. This was also mentioned by Alan, as developers offered "variations of rules they made. Not house rules". This demonstrates a clear demand for flexibility, so the system must allow users to define and apply their own rulesets while maintaining validation.

*Performance Criteria:*

1. Custom rule configurations to named and saved to a local account.
2. Rules to be loaded from local account in under 5 seconds.
3. Generate a shareable code to load a ruleset, e.g. "RMY-A7B3-X9K2".
4. The system validates that rules are valid through detecting and preventing contradictory rule combinations, such as the minimum meld size being more than the maximum meld size.
5. Rule Configuration UI should be simple, with options split up into recognisable categories in the UI.
6. Allow customisation of core rules, such as meld restrictions and requirements, Ace positioning, etc.
7. Allow customisation of basic rules, such as scoring system, joker count, discard pile reuse, etc.

# Limitations and constraints:

## Technical Constraints:

The system is limited by the complexity of implementing a fully customisable rule validation system. By allowing users to define arbitrary rule combinations introduces significant logical complexity, and some edge cases may not be fully validated. Furthermore, not every rule can be changed, as that would have even more overhead on the validation system, so only some rules can be changed. Furthermore, the client-server architecture introduces dependency on

server availability, as if a server fails, all active games are interrupted. Network latency can also be an issue, affecting the synchronisation, causing slight delays.

## Project Constraints:

The development of the system is constricted by time and resource limitations, due to the fixed timeframe of the project. Furthermore, time was allocated to the independent learning of web technologies, which further constrained the development time. As such, the system focuses on a minimum viable product, meaning that some advanced features, such as mobile implementation will not be implemented. Testing is limited to a small group of users, which may not represent the full target audience

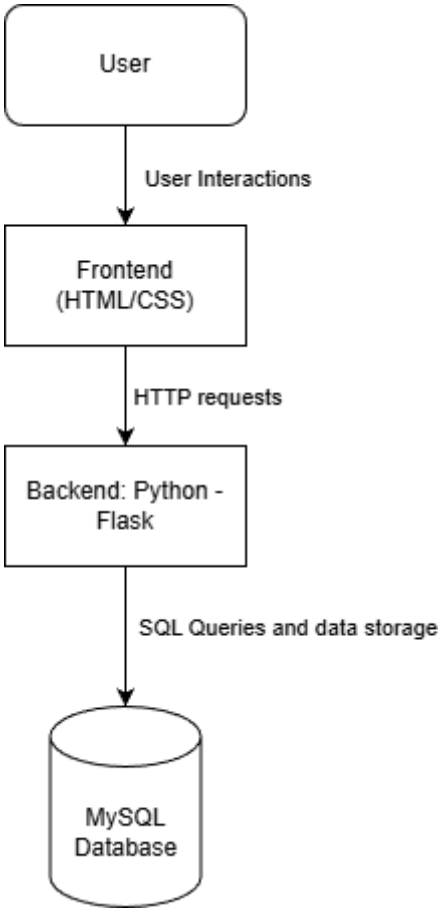
## User and Environmental Constraints:

The system must accommodate users with varying levels of experience with digital card games. Efforts are made to balance usability and customisability; some users will still find it intimidating or difficult to understand. Furthermore, the system is designed for desktop use, which will limit access for users on mobile devices. The social experience central to card games also cannot be replicated, and requires a separate application, such as discord or google meets, to fully encapsulate it.

# DESIGN:

## System Overview:

Throughout the following section, a high-level overview will be provided, showing the major components of House Rummy's architecture, and how the components interact with each other.




As outlined in the high-level system architecture, the system will contain 3 main components, frontend, backend, and the database.

The front end will be comprised of HTML, CSS, along with JavaScript. These are used as they are simple and well documented. It was also chosen for its interaction with the backend server. Furthermore, Socket.io will be used to maintain a persistent WebSocket connection for real-time game state synchronisation. The backend server will be a python app, using flask and Flask-Socket IO. Flask is a web application framework made for python to interact with web servers.

The database will be using MySQL, an RDBMS based on SQL. MySQL boasts exceptional performance for high frequency operations. Furthermore, it is extremely well supported with python, allowing for an easily deployable way to manage the database.

## IPSO Chart:

| IPSO | | | |
|---|---|---|---|
| **Input** | **Process** | **Storage** | **Output** |
| - Username & Password<br><br>- Ruleset Configuration<br><br>- Player Actions (Draw, Discard, Lay off, play meld)<br><br>- Player Decisions (Card Selection, Meld Formation) | - User Authentication<br><br>- Ruleset Validation<br><br>- Game Setup<br><br>- Move Validation Against Ruleset<br><br>- Game State Management.<br><br>- Automatic Score Calculation against Ruleset. | - MySQL Dtabase:<br>Users Table<br>Games Table<br>Friends List Table<br>GameHistory Table<br><br>- Game Objects<br><br>- Session Data | - Gamestate representation<br><br>- Validation Feedback<br><br>- Player state updates<br><br>- Score Representation. |

Inputs: The application takes user credentials for authentication, ruleset configurations to define the rules used in a specific lobbies, player actions, and player decisions. Player actions include actions such as drawing, discarding, creating melds, adding to melds, etc. Player decisions include which cards those actions are done to, and whether to do them or not.

Processes: The application authenticates users, validates the ruleset, and handles game setup. Furthermore, during gameplay it validates all actions against the ruleset. This includes melding, laying off, discarding cards, drawing card from either deck of discard pile, or shuffling the deck. The ruleset also contributes in the automatic score calculation. The system also manages the game state ensuring that all players are synchronized to the same game state.

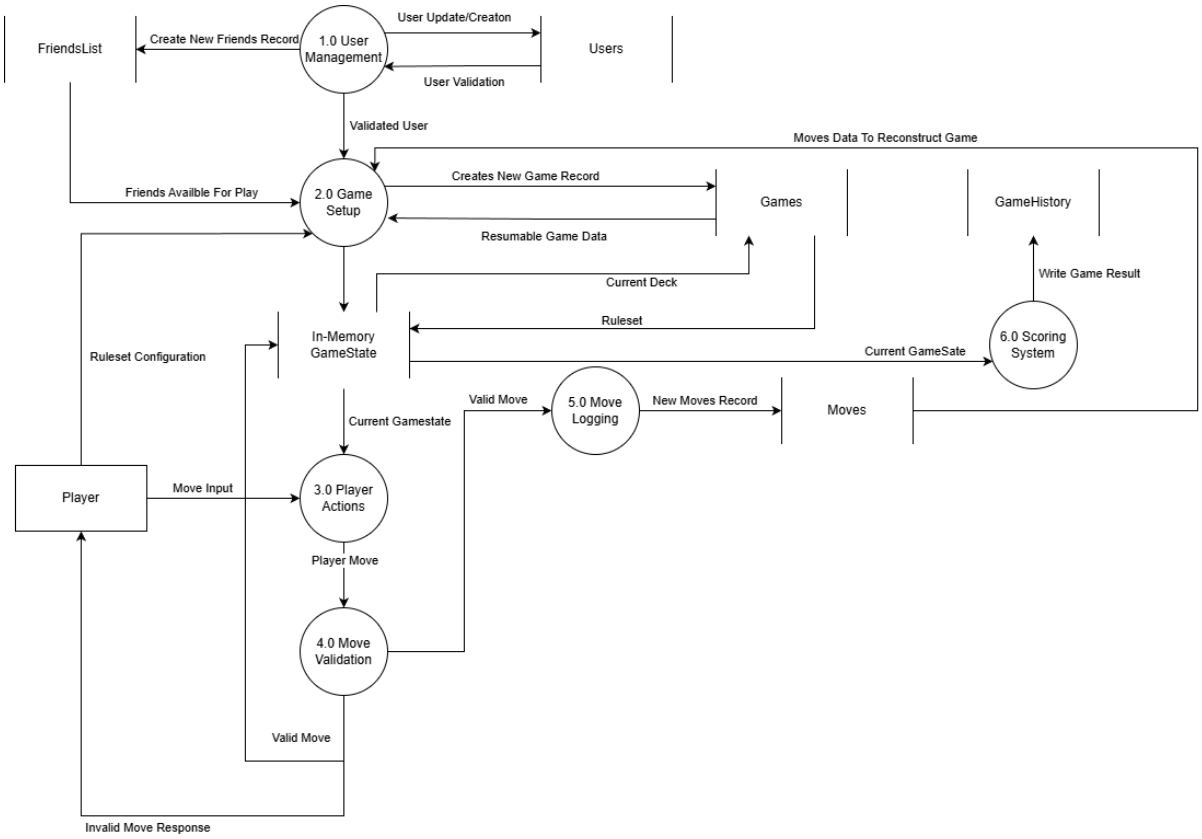
Storage: The application stores all users, moves, games, and friend connections to a MySQL database. Furthermore, all game objects such as the deck, hand, and discard pile are stored in memory, alongside any session data to maintain the current game state.

Output: The application outputs a representation of the current game state that is synchronised with all players, validation results, and updates to each player's score.

# Design and Objectives Link:

| Design Feature | Objective Linked | Justification |
|---|---|---|
| Validation Class and Move Validation Algorithms | Central Validation System | Ensures all player actions are checked against the ruleset before being applied, preventing invalid moves and reducing rule disputes. |
| Centralised Game State Management | Central Validation System | Maintains a single authoritative game state, ensuring consistency and enabling controlled updates after validation. |
| Client-Server Architecture (Flask + Sokcet.IO) | Client-Server Network Architecture | Allows real-time communication between players and ensures all clients receive synchronised updates from a central server. |
| MySQL database | Account Creation and Authentication | Stores persistent user data, enabling account management, friend systems, and game history tracking. |
| FriendsList System and Lobby Design | Account Creation and Authentication | Enables users to add friends and create private lobbies, supporting social gameplay identified in interviews. |
| Frontend UI (HCI) | User Interface Design | Provides clear visual representation of the game state, balancing implicit and explicit information for usability. |
| Ruleset Class + JSON configuration system | Game Rule Customisation | Allows users to define, save, and load custom rulesets, supporting flexibility beyond preset game modes. |
| Ruleset validation algorithms | Game Rule Customisation | Ensures custom rules are logically valid, preventing contradictory configurations. |

# Level 1 DFD Diagram:




The Dataflow diagram represents how data flows through backend system of House Rummy, showing interactions between the user and the system components.

The user is validated against the Users datastore if attempting to login, or the registered as a new entry when signing up. The FriendsList datastore is updated by sending and accepting friend requests, allows for game invites based on connections in the FriendsList.

During Game Setup the user inputs in their custom ruleset configuration through either UI controls or by uploading a JSON file ruleset file. The JSON file is validated against the data types, data ranges, and allowed values.

Users can invite friends from the FriendsList to join the lobby. When the game starts a new record in the Games datastore is created, which contains the ruleset JSON and game metadata, such as the status and timestamp it is made. The ruleset is initialised into the In-Memory In-Memory GameState datastore, which maintains the active game state (hands, deck, table melds, etc), allowing for quick access in gameplay

The player actions process includes all the actions that can be performed in a game, such as drawing, melding, and discarding. This triggers the Validate Moves process, which checks the move against the active ruleset in the In-Memory Gamestate datastore. Invalid moves are returned as error messages to the user; valid moves are passed to the In-Memory Gamestate datastore, and the Moves Logging process. This process logs the move into the Moves

datastore as JSON notation including full card data. Further, the current deck state, which contains the cards in order, is serialized and saved to the games datastore.

Through this system, where both moves and the current deck state are stored, there are two complementary methods of recovery:

Historical replays can be done through the moves datastore, as it contains the complete move history along with card data, allowing for complete recreation any past game position, for replays.

Games can also be resumed through the Games datastore as they store the most recent deck, which allows for the game to be replayed up to the most recent moves to the moves list, enabling a complete recreation of the game up to the point of resuming.

The Scoring System process checks if a win condition has been detected after every valid move and the, from the In-Memory Gamestate datastore. Results are stored in the GameHistory datastore, that connects the Games and Users datastores. That is expanded on with the database design.

# Data Structures:

There are two main types of data structures withing House rummy, persistent relational structures in MySQL and runtime structures in Python.

## Database Structures:

- Users: Stores user credentials and details
- Games: Stores ruleset, timestamp, and seed
- GameHistory: Links Games and Users, stores the result of a game for the user
- FriendsList: Self referential connections between users, stores status of friend request and friends' online status
- Moves: Stores all actions in a game

## Runtime Structures (Backend – Python)

In the implementation of rummy, 6 different data structures are used: lists, dictionaries, Enumerations, tuples, stacks, and queues.

### *Lists:*

- Cards in hand = list(cards)
  Used to store the cards in hand for a certain player.

- Deck = list(cards)
  Used to store the current cards in the deck.

- Discard pile = list(cards)
  Stores the current cards in the discard pile.

- Melds = list(cards)
  Used to store the cards in a meld.

- Table Melds = list(melds)

Used to store all melds currently in play

For the above examples, lists are used instead of arrays because of the flexibility they provide in dynamic sizing, and for the constant insertions and deletions into and out of the lists.

## Dictionaries:

- Wild Assignments: mapping the wild card with the card it is taking the place of inside a meld.
- Rank Index: mapping ranks to their overall index, used for sorting and run validation
- Index Rank: a flipped Rank Index dictionary, used for scoring melds, and validating wild replacement.
- Ruleset: In the process from going JSON files to a ruleset class and in reverse.

Dictionaries are used for a multitude of reasons, including their O(1) key value lookup, improving the efficiency for operations that access them frequently. In sorting algorithms this is useful, as the index of a card must be looked up repeatedly, where the O(n) time complexity of a list would increase the total time complexity of the sorting algorithm. Further, the ruleset parameters are stored within a dictionary, which provides one to one mapping with JSON files. The use of a dictionary also enables partial configurations, where the user would only specify non default values.

However, dictionaries have higher overhead than lists due to hash tables, but this is negligible due to the small datasets in use. (Maximum 30 key value pairs.)

## Enumerations:

There are two Enums used in the project, Suit and Moves.

- Suits: Defines the four suits as fixed values with index.
- Moves: Defines the 6 actions that can occur in a game.
- Melds: Defines the two different meld types, runs and sets.

The enumerations are used in both to improve validation, as they reduce the risk of invalid values, and only allow the specific elements outlined in the enumeration. This improves the reliability of the system, as only those recognised values can be used. Furthermore, the codebase is more readable, as ambiguous strings are replaced with clear identifiers.

## Tuples:

Tuples will be used throughout the code as return values, using the consistent structure of Boolean, Data. This allows each operation to indicate its success, while also returning any resulting data or an appropriate error message, Furthermore, operation failure is expected for validation functions, so exception handling would not be a valid method.

Overhead is also minimised as python’s exception handling method is slow an uses more resources than a simple tuple, keeping execution more efficient.

## Stacks:

Stacks are used for the discard pile and the deck, as both follow the Last-In First-Out structure in the game, because only the top card in either pile is interactable. The use of a stack also ensures a O(1) time complexity for the actions of pushing or popping from the stack, increasing the efficiency of the application, while keeping the logic consistent with the game behaviour.

## *Circular Queue:*

A circular queue is used for the turn order as it allows players to be processed in a continuous loop, without changing the structure. This maintains the loop, while allowing for enqueue and dequeue processes that are more efficient than equivalent list-based operations, making advancing turns be accomplished with minimal overhead.

## *Wilds:*

In the ruleset, wilds are stored as a list of tuples, with the first value of the tuple being the rank/name of the card, and the second value being the number of that card in the deck. This was chosen instead of linked lists or dictionaries for its smaller memory usage, while condensing all the values into one list.

# OOP Design:

Object oriented programming is used to manage the complexity of the project by encapsulating the data and methods into a single object, increasing code usability and maintainability by separating responsibilities. Furthermore, the ease of testing important given that customisability is a key requirement of the application.

## Objects:

(+ indicates a public attribute or method, # indicates a private attribute or method.)

| Card Class | |
|---|---|
| Attributes | # Suit (Enumerator)<br># Rank (String)<br># Ruleset (Object) |
| Methods | # __init__: Initialises the card object<br># __repr__: Defines the representation of the card in the form, “rank of suits”<br># __eq__: Defines the method to evaluate if two card objects are equal.<br>+ return_value: Returns the “value” of the card, e.g. King = 10.<br>+ return_rank_index: Returns the index of a card in a run, e.g. Jack = 11<br>+ to_dict: Creates a dictionary with value pairs of rank and suit to be exported to JSON<br>+ from_dict: reconstructs the class from a dictionary.<br>+ to_json: uses the JSON library and to_dict to create a JSON file with the card info<br>+ as data using to_dict.<br>+ from_json: reconstructs the class from the JSON file using from_dict. |

The card is defined by its rank and suit, with the ruleset is used for validation, as it stores what cards are valid.

| Deck Class | |
|---|---|
| Attributes | + cards (Stack)<br># ruleset (Object)<br># seed (Integer) |
| Methods | # __init__: Initialises the deck object, creates all the cards into the cards attribute. Creates random seed if no seed is given<br>+ shuffle: Uses a shuffling algorithm to shuffle all the cards, dependent on seed.<br>+ pop: pushes the top card from the stack |

| | |
|---|---|
| | + get_size: returns the size of the deck<br>+ empty_check: checks if the deck is empty<br>+ push: pushes a card or a stack of cards to the deck |

The deck is defined by the cards inside the deck, along with the ruleset to determine the number of standard 52 card decks, along with the wild cards.

| Hand Class | |
|---|---|
| Attributes | + cards (List)<br>+ selected_cards (List) |
| Methods | # __init__: Initialises the hand object.<br># __repr__: Defines the representation of the hand object as “card, card, card, …” and “selected cards: card, card, ...”<br>+ add_card: adds a card to the cards list<br>+ remove_card: deletes card from the card list<br>+ select_card: adds card to the selected_cards attribute<br>+ deselect_card: removes card from selected_cards attribute<br>+ sort_by_rank: Sorts cards list by rank<br>+ sort_by_suit: Sorts the cards by suits, then by rank.<br>+ is_empty: Checks if hand is empty<br>+ calculate_deadwood: calculates the score of all the cards in the hand. |

| Meld Class | |
|---|---|
| Attributes | + cards (List)<br># meld_type (Enumerator)<br># wild_assignments (Nested Dictionary)<br># ruleset (Ruleset Object) |
| Methods | # __init__: Initialises the meld object, assigning wild cards the attributes of the card they are replacing.<br># __repr__: Defines the representation of the meld object as “card, card, card, …”<br># wild_assignment_run: Used during initialisation to assign wild cards their attributes if the meld_type is a run.<br># wild_assignment_set: Used during initialisation to assign wild cards their attributes if the meld_type is a set.<br># split_meld: used in wild assignments, splitting non wilds and wilds<br># is_ace_high_in_run: If an ace exists, checks whether it is high or low to know its score<br>+ get_card_value: Gets the value of any card in the meld, including wilds<br>+ get_meld_value: Gets the full value of the meld.<br>+ add_card: Adds card to the meld list<br># wild_card_comparison: Checks if a card has the same attributes as the card the wild is taking the place of<br>+replace_wild: replaces the wild card with a card with the same attributes, returning the wild card as well. |

| Discard Pile Class | |
|---|---|
| Attributes | + cards (Stack) |
| Methods | # __init__: Initialises the discard pile.<br># __repr__: Defines the representation of the discard pile, acts equal to peek in a stack. |

| | |
|---|---|
| | + push: adds card to the top of the discard pile.<br>+ pop: pops the top card of the discard pile.<br>+ split_discard_pile: Separates the top and rest of the discard pile, returning the bottom to be reshuffled<br>+ is_empty: Checks if the discard pile is empty |

| Validator Class | |
|---|---|
| Attributes | # ruleset (Ruleset Object) |
| Methods | # __init__: Initialises the validator.<br># draw_valdiation: Checks if the deck is not empty, and if drawing from the discard pile is currently allowed by the rules.<br># discard_validation: Checks if the card is in the players hand and if discarding the card is valid under ruleset.<br># run_validation: Validates if a run follows all the rules.<br># set_validation: Validates if a set follows all the rules.<br># meld_validaiton: Abstracts run_validation and set_validation<br># lay_off_validation: Validates if a card can be added to a meld<br># end_turn_validation: Validates if has completed all required moves to end a turn |

| Player Class | |
|---|---|
| Attributes | + player_id (Integer)<br>+ name (String)<br>+ hand (Hand Object)<br># stored_melds (List of meld objects)<br>+ score (Integer)<br># has_melded (Boolean) |
| Methods | # __init__: Initialises the player object.<br>+ store_meld: stores a valid meld into the stored_melds attribute<br>+ push: adds card to the top of the discard pile.<br>+ clear_stored_melds: clears the stored_melds list.<br>+ get_stored_melds_score: gets the total score of all the melds stored<br>+ add_to_score: adds to player score |

| Ruleset Class | |
|---|---|
| Attributes | + allow_sets (Boolean)<br>+ allow_runs (Boolean)<br>+ min_meld_size (Integer)<br>+ max_meld_size_set (Integer)<br>+ max_meld_size_run (Integer)<br>+ min_initial_meld_score (Integer)<br>+ initial_meld_increment (Integer)<br>+ wilds (Array of Tuples)<br>+ wild_deadwood_score (Integer)<br>+ scoring_method (string)<br>+ ace_low (Boolean)<br>+ ace_high (Boolean)<br>+ ace_both (Boolean)<br>+ ace_wrap_around (Boolean)<br>+ ace_high_score (Integer)<br>+ initial_hand_size (Integer) |

| | |
|---|---|
| | + num_decks (Integer)<br>+ require_melding_to_draw_from_dicard (Boolean)<br>+ allow_wild_replacement (Boolean)<br>+ allow_wild_only_melds (Boolean)<br>+ prevent_discard_same_card (Boolean) |
| Methods | - __init__: Initialises the player object<br>- get_validated: validates whether the provided variable for each attribute. Allows defaults, expected types, allowed values, and min/max values for integers<br>+ push: adds card to the top of the discard pile.<br>+ clear_stored_melds: clears the stored_melds list.<br>+ get_stored_melds_score: gets the total score of all the melds stored<br>+ add_to_meld: adds card to meld on the table after being validated by Validator.<br>+ calculate_deadwood: calculates the score of all the cards in your hand and adds that to your deadwood.<br>+ add_to_score: adds to player score |

| GameState Class | |
|---|---|
| Attributes | # ruleset (ruleset object)<br># game_state (Enumerator)<br># players (Circular Queue of Player Objects)<br># deck (Deck Object)<br># discard_pile (Discard Pile Object)<br># table_melds (List of Melds)<br># winner (Player Object)<br># current_required_meld_score (Integer)<br># current_player_index (Integer) |
| Methods | + __init__ : Initialises the game state, with deck, discard pile, and table melds set to None until game starts<br>+ update_ruleset: Updates the used ruleset, if the game hasn't started.<br>+ start_game: Initialises the deck, deals cards, and places a top card onto the discard pile<br>+ return_current_player: Returns the player whose turn it is<br>+ nex_turn: Advances to next player, maintaining the circular queue.<br>+ draw_from_deck: Uses the Validator method to validate, before drawing a card from the deck and adding it to the player's hand.<br>+ draw_from_discard: Uses the Validator method to validate, before drawing a card from the discard pile and adding it to the player's hand.<br>+ discard: Uses the Validator method to validate, removing a card from the players hand and adding it to the discard pile.<br>+ lay_off: Uses the Validator method to validate, removing a card from the players hand and adding it to a meld on the table.<br>+ return_game_state: returns the current GameStatus enum value<br>+ return_winner: returns winning player object, or none if the game isn't over.<br># end_game: calculates all the scores<br>- check_win_condition: Uses the ruleset to check if a player has won this turn, calls end_game if it has.<br>- update_required_meld_score: Checks the ruleset and most recent move to see if the required score to meld has increased. |

# Algorithms:

Many Algorithms are used throughout the program for validation and sorting. All the algorithms shown here are assuming that the above objects exist with their attributes.

Some standard helper functions are used throughout the following section, and as such, a quick explanation of them will be put here.

1) ruleset.is_wild(card): Card object as the argument, returns a Boolean expression depending on if the card is a wild cardor not.
2) rank_sort(cards): List of card objects as the argument, uses a quicksort algorithm to sort them by rank.

## Utilities:

### *Sorting:*

```
FUNCTION quicksort(items)
   IF LENGTH(items) <= 1 THEN
      RETURN items

   pivot <-- last item in items
   items_greater <-- empty list
   items_lesser <-- empty list

   FOR item IN ITEMS
      IF item > pivot THEN
         ADD item TO items_greater
      ELSE
         ADD item TO items_lesser

   RETURN quicksort(items_lesser) + pivot + quicksort(items_greater)
```

Quicksort is used for both run validation and for sorting player hands. In run validation it is used to ensure that the list is al sequential with no breaks.

```
FUNCTION sort_suit(cards)

   hearts_cards <-- empty list
   clubs_cards <-- empty list
   diamonds_cards <-- empty list
   spades_cards <-- empty list

   FOR card IN cards
      IF card.suit IS hearts THEN
         ADD card TO hearts_cards
      ELSE IF card.suit IS clubs THEN
         ADD card TO clubs_cards
```

```
        ELSE IF card.suit IS diamonds THEN
            ADD card TO diamonds_cards
        ELSE IF card,suit IS spades THEN
            ADD card TO spades_cards

    RETURN sort_rank(hearts_cards) + sort_rank(clubs_cards)
    + sort_rank(diamonds_cards) + sort_rank(spades_cards)
```

This is used for sorting the hand to more easily notice runs, while the quicksort is used to sort the hand to more easily notice sets.

### *Split wilds:*

The two arguments used in the function are:

1) cards: List of card objects.
2) ruleset: Ruleset object.

```
FUNCTION split_wilds(cards, ruleset)

    wilds <-- empty list
    non_wilds <-- empty list

    FOR card IN cardds

    IF ruleset.is_wild(card) THEN
        ADD card TO wilds
    ELSE
        ADD card TO non_wilds

    RETURN wilds, non_wilds
```

This is a general algorithm that splits melds and non-wilds from a list of cards, returning a tuple. This is used in both validation and in assigning wild cards the attributes of the card they are taking the place of.

## Validation Algorithms:

The two algorithms below are the algorithms used for validating melds in the game. Assume the MeldType Enumeration exists.

### *Set Validation:*

The two arguments used in the function are:

1) cards: List of card objects.
2) ruleset: Ruleset object.

```
FUNCTION validate_set(cards, ruleset)
```

```
    IF ruleset.max_meld_size_set IS NOT NONE
    AND LENGTH(meld) > ruleset.max_meld_size_set THEN
        RETURN FALSE, "Meld Too Large"

    IF LENTH(cards) < ruleset.min_meld_size THEN
        RETURN FALSE, "Meld Too Small"

    suits ← empty list
    ranks ← empty list

    FOR card IN cards
        IF NOT ruleset.is_wild(card) THEN
            ADD card.rank TO ranks
            ADD card.suit TO suits

    IF LENGTH(SET(ranks)) != 1 THEN
        RETURN FALSE, "Different Ranks IN Set"

    IF LENGTH(SET(suits)) != LENGTH(suits) THEN
        RETURN FALSE, "Multiple of the Same Suits in 1 Set"

    RETURN TRUE, MeldType.SET
```

The above pseudocode documents how sets are validated against both the general rules of the game and any specific rules in the ruleset. First the size of the meld is checked against the max and minimum meld size specified in the ruleset. If it passes, then it creates a list of all the ranks and suits of the non-wild cards. Wild cards are not considered here as they act as substitutes and do not affect the validity of any set. By using the length of the set of ranks, we can see the number of different ranks in use, which if it is not equal to 1, means the set is invalid. Further, as every suit must be different, the length of the set of suits must equal to the length of suits, so if that fails, the set is invalid. Finally, if it passes all validation steps, it returns true. Finally, all return operations are Tuples with the structure of (False, error_message) or (True, cards)

The algorithm is used when a set is attempted to be made.

## *Run Validation:*

The two arguments used in the function are:

3) cards: List of card objects.
4) ruleset: Ruleset object.

```
FUNCTION validate_run(cards, ruleset)

    IF ruleset.max_meld_size_run IS NOT NONE
    AND LENGTH(meld) > ruleset.max_meld_size_run THEN
```

```
        RETURN FALSE, "Meld Too Large"

    IF LENTH(cards) < ruleset.min_meld_size THEN

        RETURN FALSE, "Meld Too Small"

    wilds, non_wilds <-- split_wilds(cards)

    IF NOT ruleset.allow_wild_only_melds AND LENGTH(ranks) == 0 THEN
        RETURN FALSE, "Meld Contains Only Wild Cards"

    suits <-- empty list
    indices <-- empty list

    FOR card IN non_wilds
        ADD card.suit TO suits
        ADD card.return_rank_index() TO indices

    indices <-- quicksort(indices)

    IF LENGTH(SET(suits)) != 1 THEN
        RETURN FALSE, "Multiples Suits in 1 Run"

    IF LENGTH(SET(indices)) != LENGTH(indices) THEN
        RETURN FALSE, "Multiples of the Same Ranks"

    IF ruleset.ace_low OR ruleset.ace_both THEN

        min_index <-- MINIMUM(indices)
        max_index <-- MAXIMUM(indices)
        span <-- max_index - min_index + 1
        gaps <-- span - LENGTH(non_wilds)

        IF gaps <= LENGTH(wilds) THEN
            RETURN TRUE

        has_ace <-- FALSE
        has_king <-- FALSE

    FOR index IN indices
        IF index == 0 THEN
            has_ace <-- TRUE
        ELSE IF index == 12 THEN
            has_king == TRUE


    IF ruleset.ace_high OR ruleset.ace_both THEN
```

```
    adjusted_indices = []
    FOR index IN indices
      IF index == 0 THEN
        ADD 13 TO adjusted_indices
      ELSE
        ADD index TO adjusted_indices

    adjusted_indices = quicksort(adjusted_indices)

    min_index <-- MINIMUM(adjusted_indices)
    max_index <-- MAXIMUM(adjusted_indices)
    span <-- max_index - min_index + 1
    gaps <-- span - LENGTH(non_wilds)

    IF gaps <= LENGTH(wilds) THEN
      RETURN TRUE, MeldType.RUN

    IF ruleset.ace_wrap_around AND has_king THEN
      low_indices <-- empty list
      high_indices <-- empty list

      FOR index IN indices
        IF index < 9 THEN
          ADD index TO low_indices
        ELSE IF index >= 10 THEN
          ADD index TO high_indices

        IF LENGTH(low_indices) > 0 AND LENGTH(high_indices) >0 THEN
          low_span <-- MAX(low_indices) - MIN(low_indices)
          high_span <-- MAX(high_indices) - MIN(high_indices)

          total_gaps <-- (low_span - LENGTH(low_indices))
          + (high_span - LENGTH(high_indices))

          IF total_gaps <= LENGTH(wilds) THEN
            RETURN TRUE, MeldType.RUN

  RETURN FALSE, "Invalid Run"
```

The above algorithm validates whether a group of cards constitutes a run for a given ruleset. The first validation step is for the size of the run, ensuring that that no unnecessary computation is expended if a rule is clearly violated. Then, wild and non-wilds are separated because wild cads can act as substitutes, so only the non-wilds are needed. The next important part of validation uses the SET function to confirm that all non-wild have the same suit and no duplicate ranks exist.

Next is the gap calculation, which calculates the number of gaps between the non-wilds. It then validates if the number of gaps is equal to the number of wilds. Further, for ace_high it changes the index of ace from 0 to 13, and then does the gaps validation. For ace_wrap_around, it validates sequences going King, Ace, 2, etc.

As with the previous validation, it returns tuples with the patterns of either True, cards or False, error message.

## *Playing Melds Validation:*

Four arguments are used for validation:

1) melds: List of meld objects
2) has_melded: Boolean if player has already melded
3) ruleset: Ruleset object
4) required_score: Integer of the required score to meld for the first time.

```
FUNCTION validate_play_melds(melds, has_melded, ruleset, required_score)

  IF LENGTH(melds) == 0:
    RETURN FALSE, "No Melds"

  IF NOT has_melded THEN
    total_score <-- 0

    FOR meld IN melds
      total_score <-- total_score + meld.return_meld_value(ruleset)
    IF total_score < required_score THEN
      RETURN FALSE, "Minimum Score Not Met"

  RETURN TRUE, NONE
```

The above algorithm validates if the player can play the melds currently in their hand, using the current required score.

## *Laying Off Validation:*

Three arguments are used in the following function:

1) card: card object that is being added to meld.
2) meld: meld object that the card is being added to.
3) ruleset: ruleset object.

The validate_meld function is an abstraction of the previous two meld validation functions.

```
FUNCTION validate_lay_off(card, meld, ruleset, has_melded)
  test_cards <-- meld.cards + [card]
```

```
    valid, error = validate_meld(test_cards, ruleset)

    IF ruleset.require_melding_to_lay_off AND NOT has_melded THEN
        RETURN FALSE, "You Require to Have Melded to Lay Off"

    IF valid THEN
        RETURN TRUE, NONE
    RETURN FALSE, "Card Doesn't Fit in Meld"
```

The above algorithm tests if the player can lay off a card according to the ruleset, and then if the card is a valid addition to the meld. If it is, it returns TRUE, NONE. If it doesn't pass, it returns FALSE, ERROR.

## *Discard Validation:*

Five arguments are used in the following function:

1) card: card object that is being discarded
2) cards: list of card objects that represents the hand
3) has_drawn: Boolean of whether the player has drawn a card this turn.
4) drawn_from_discard: card object that was immediately drawn from the discard pile, resets to None after every turn
5) ruleset: ruleset object.

```
FUNCTION validate_discard(card, cards, has_drawn, drawn_from_discard, ruleset):
    IF card NOT IN cards THEN
        RETURN FALSE, "Card Not in Hand"

    IF NOT has_drawn THEN
        RETURN FALSE, "Must First Draw a Card"

    IF ruleset.prevent_discard_same_card AND drawn_from_discard == card THEN
        RETURN FALSE, "Cannot Discard Card Immedietly Drawn from Discard Pile"

    RETURN TRUE, NONE
```

The above algorithm validates if the selected card can be discarded. First it checks if the card is in the hand, then it checks if the player has drawn a card yet. If both pass, it checks if the card was a card that was in the discard pile immediately before, and if that is allowed in the ruleset. If all validation steps pass, it return TRUE, NONE.

## *Draw Validation:*

Two Arguments are used for validation for drawing from the deck:

1) deck: deck object that is being drawn from
2) has_drawn: Boolean if player has drawn this turn

```
FUNCTION validate_draw(deck, has_drawn)
   IF deck.is_empty() THEN
      RETURN FALSE, "Deck is Empty"
   IF has_drawn THEN
      RETURN FALSE, "Player has Drawn"

   RETURN TRUE, NONE
```

The above function validates if the player can draw from the deck, by first checking if the deck is empty, then if the player has already drawn. If it fails, it returns FALSE, ERROR, and if it succeeds it returns TRUE, NONE

5 arguments are used in the validation for drawing from the discard pile:

1) card: card object being drawn from the discard pile
2) discard_pile: list of card objects representing the discard pile
3) has_drawn: Boolean if the player has drawn this turn
4) has_melded: Boolean if the player has melded this game
5) ruleset: ruleset object

```
FUNCTION validate_draw_discard(card, discard_pile, has_drawn, has_melded, ruleset)
   IF discard_pile.empty THEN
      RETURN FALSE, "Discard Pile is Empty"
   IF has_drawn THEN
      RETURN FALSE, "Player Has Drawn Card"
   IF card NOT IN discard_pile THEN
      RETURN FALSE, "Card isn't in Discard Pile"

   IF ruleset.require_melding_to_draw_from_disc AND NOT has_melded THEN
      RETURN FALSE, "Melding Required to Draw from Discard Pile"

   RETURN TRUE, NONE
```

The above algorithm validates if the player can draw from the discard pile, validating if the discard pile isn't empty, if the card they are drawing is in the discard pile, if they have already drawn this turn, and if the ruleset says that the player must have melded previously to draw from the discard pile. If validation fails, it returns FALSE, ERROR. If validation succeeds it returns TRUE, NONE.

## Deck Algorithms:

There are two algorithms used in the Deck: Deck creation and shuffling.

### *Deck Creation:*

This creates a deck of cards based on the ruleset provided.

```
FUNCTION create_deck(ruleset)
```

```
cards <-- empty list

FOR deck <-- 1 IN RANGE(ruleset.num_decks)
    FOR rank IN rank_index
        IF rank NOT IN ruleset.wilds THEN
            FOR suit IN Suit
                card <-- create_card(rank, suit, ruleset)
                ADD card TO cards

FOR wild, num_wild IN ruleset.wilds
    FOR i <-- 1 IN RANGE(num_wild)
        card <-- create_card(wild, NONE, ruleset)
        ADD card TO cards

RETURN cards
```

The first for loop creates all non-wild cards, by using nested for loops. The second for loop creates all wild cards, using the tuple given in the ruleset.wilds, before adding them to the deck. The deck is then returned.

### *Deck Shuffling:*

```
FUNCTION shuffle(cards)

FOR i FROM last index OF cards DOWN TO 1
    roll = random integer between 0 and i (inclusive)
    SWAP cards[i] WITH cards[roll]
```

This is a fischer-yates algorithm is used to swap two cards from the deck, ensuring a perfect shuffle in O(n) time complexity. This is used rather than picking a random card from the deck when drawing, to reduce it to an algorithm only used once when a deck is created.

## Wild Assignments:

The two below algorithms are how wild cards are assigned the attributes of the card they are taking the place of. Both use the split_meld function, so that will be explored first.

### *Wild Assignment set:*

```
FUNCTION wild_assignment_set(cards, ruleset)

wilds, non_wilds <-- split_wilds(cards, ruleset)

non_wild_suits <-- empty list

FOR card IN non_wilds
    ADD card.suit TO non_wild_suits

missing_suits <-- empty list
```

```
FOR suit IN Suit
    IF suit NOT IN non_wild_suits THEN
        ADD suit TO missing_suits

FOR i <-- 0 TO LENGTH(wilds) - 1
    wild_assignments[wild] <-- {
        'SCORE' : non_wilds[0].return_value(),
        'RANK_INDEX' : non_wilds[0].return_rank_index(),
        'RANK' : non_wilds[0].rank,
        'SUIT' : missing_suits[i]
        }
```

This finds the missing suit(s) in the set, and then add to the wild_assignments dictionary another dictionary outlining the attributes of the wild card, with everything being the same as the non-wild cards, except for the suit, which takes one of the missing suits.

## *Wild Assignment Run:*

```
FUNCTION wild_assignment_run(cards, ruleset)

    wilds, non_wilds <-- split_wilds(cards, ruleset)

    indices <-- empty list

    FOR card IN non_wilds
        ADD card.return_rank_index() TO indices

    indices <-- quicksort(indices)

    range_indeces <-- FIRST(indices) TO LAST(indices)

    missing_indices <-- empty list

    FOR index IN range_indices
        IF index NOT IN indices THEN
            ADD index TO missing_indices

    assigned_wilds <-- empty list

    FOR wild, missing_index IN ZIP(wilds, missing_indices)
        wild_assignments[wild] <-- {
            SCORE : score of missing_index rank,
            RANK_INDEX : missing_index,
            RANK : rank of missing_index,
            SUIT : suit used in the run}
        ADD wild TO assigned_wilds

    FOR wild IN assigned_wilds
```

```
        REMOVE wild FROM wilds

    IF LENGTH(wilds) > 0 THEN
        FOR i <-- 0 TO LENGTH(wilds) - 1
            offset <-- MIN(indices) - (LENTH(wilds) - i)
            wild_assignments[wilds[i] <-- {
                SCORE : Score of Rank at the offset,
                RANK_INDEX : offset,
                RANK : Rank of offset index,
                SUIT : suit used in the run}
```

This finds any missing indices in a run, adding to the wild_assignments dictionary another dictionary outlining the attributes of the wild card, with only the suit being the same as the non-wild cards. The first part finds any missing index between 2 indices and placing the wild card there. The second part assigns any wild cards used to the lowest part of the run.

## Ruleset Validation:

Seven arguments are used for the ruleset validation:

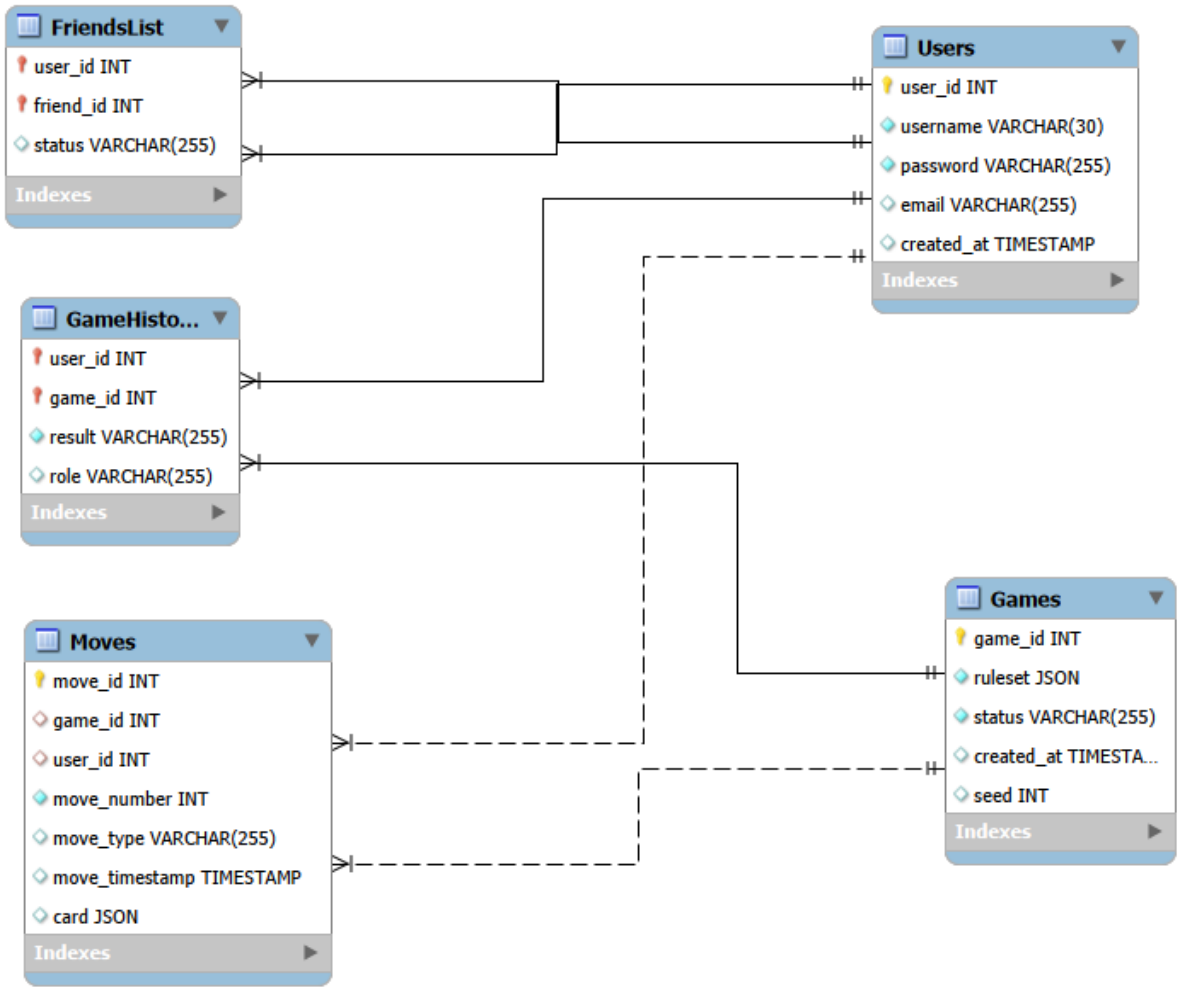
1) config: A dictionary with key value pairs
2) key: the specific key being validated
3) default: the default of this rule
4) expected_value: The expected value type
5) allowed_values: A list of all allowed values, if one exists
6) min_value: Minimum value, if one exists
7) max_value: Maximum value, if one exists

```
FUNCTION ruleset_validation(
    config,
    key,
    default,
    expected_value,
    allowed_values,
    min_value,
    max_value)

    value <-- config[key] IF key EXISTS, ELSE default

    IF value IS NOT NONE AND value IS NOT OF expected_tupe THEN
        RETURN default

    IF allowed_values IS NOT NONE AND value NOT IN allowed_values THEN
        RETURN default

    IF min_value IS NOT NONE AND value < min_value THEN
        RETURN default

    IF max_value IS NOT NONE AND value > max_value THEN
        RETURN default
```

RETURN value

The function retrieves a value from the config dictionary, before validating it. If validation fails, it falls back onto the default value, else the value given is returned. The series of checks used are checking if the value is the correct type, if it is withing a list of allowed values, if it exceeds any maximum or minimum bounds.

# Database Design:




By utilising a relational database, the natural relationships in House Rummy can be represented with the structured data it can provide, while enforcing the referential integrity of the system. As mentioned previously, MySQL is used for its maturity and support with my preferred back-end Flask, with the library Flask-SQLAlchemy.

The database is normalised to the third normal form, reducing data redundancy and ensuring each attribute is dependent on one primary key. This is except for the deliberate use of the JSON file format in the database, specifically:

- The ruleset field in the Games table (JSON)
- The card field in the Moves table (JSON)

While strictly speaking these do not adhere to the first normal form, as they are non-atomic elements, due to the practical system requirements they are the optimal choice. JSON files solve the issue of storing complex data, by having a file with variable structure that can be processed as a unit. This is useful because it avoids the needs of additional tables and fields, keeping the schema simple. The ruleset field especially benefits from the varied structure, as with updates new rule parameters can be added or removed without changing the database schema. This introduces a new problem, where the amount of complex relational queries are reduced, which could be useful, such as looking for a move that played a specific card. However, the general access pattern does not rely on such queries.

The database schema is made up of 5 tables:

- Users: Stores credentials and the time stamp for when the account was created.
- Games: Stores the ruleset used, the timestamp that the game was created, and the seed used by the game.
  The seed initialises the random number generator, which when combined with the Moves table, allows for full reconstruction at any point within a game.
- GameHistory: Links users to games, showing the role they played, along with the personal result of the user.
- FriendsList: Self-referential connections between users, along with status for whether the friend request is pending, denied, or accepted.
- Moves: Stores every action performed, including the move type, and who performed the action. The card field stores the card used in the move if one exists, or the field is null if the action does not include a specific card, e.g. Shuffling, allowing for both player and system actions to be represented in this structure.
  System actions are recorded with an identifier in the user_id field, rather than a user, allowing for them to be separated while retaining consistency in the structure.

The reconstruction approach mentioned above is used to reduce the amount of redundant data that will be stored, as it creates a single source of data, the Moves table. Furthermore, inconsistency is reduced as there is no saved states that can conflict with the moves list. However, this reconstruction requires to retrieve moves from the database, before replaying them sequentially, which has a time complexity of O(n) while a save state has a time complexity of O(1).

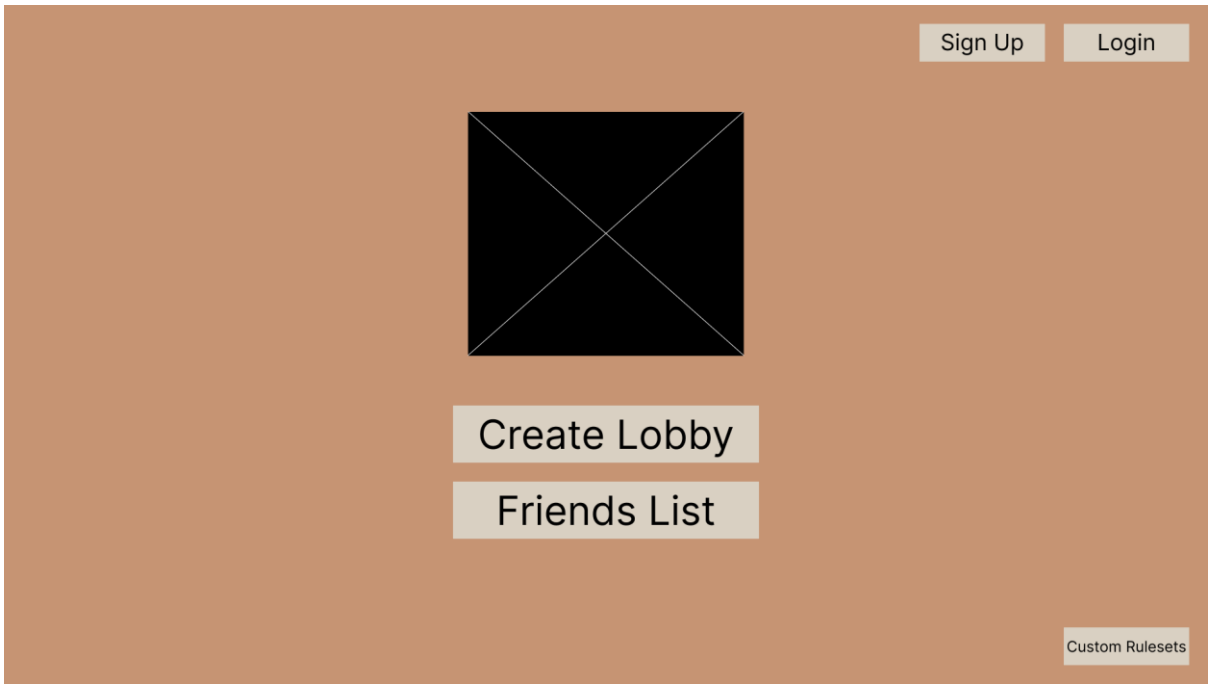
Foreign key constraints exist for all tables that use foreign keys, which prevent records from referencing non-existent foreign keys, therefore preventing orphaned records.

# Human Computer Interface:

The user interface was designed to do be simple, while displaying all crucial information. It is also designed to in game information displayed implicitly rather than explicitly. The colour scheme is designed to be as minimally distracting as possible.

In the technical solution, the logo will be made, in the meanwhile a placeholder is used.

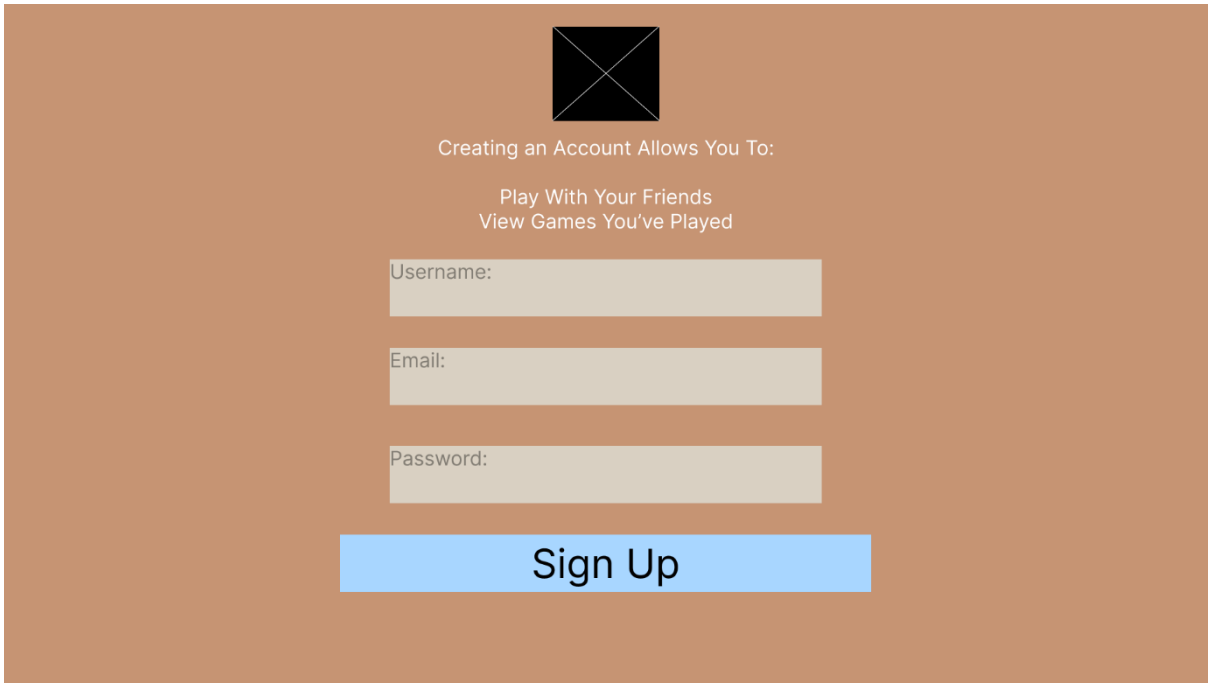
## Homepage:

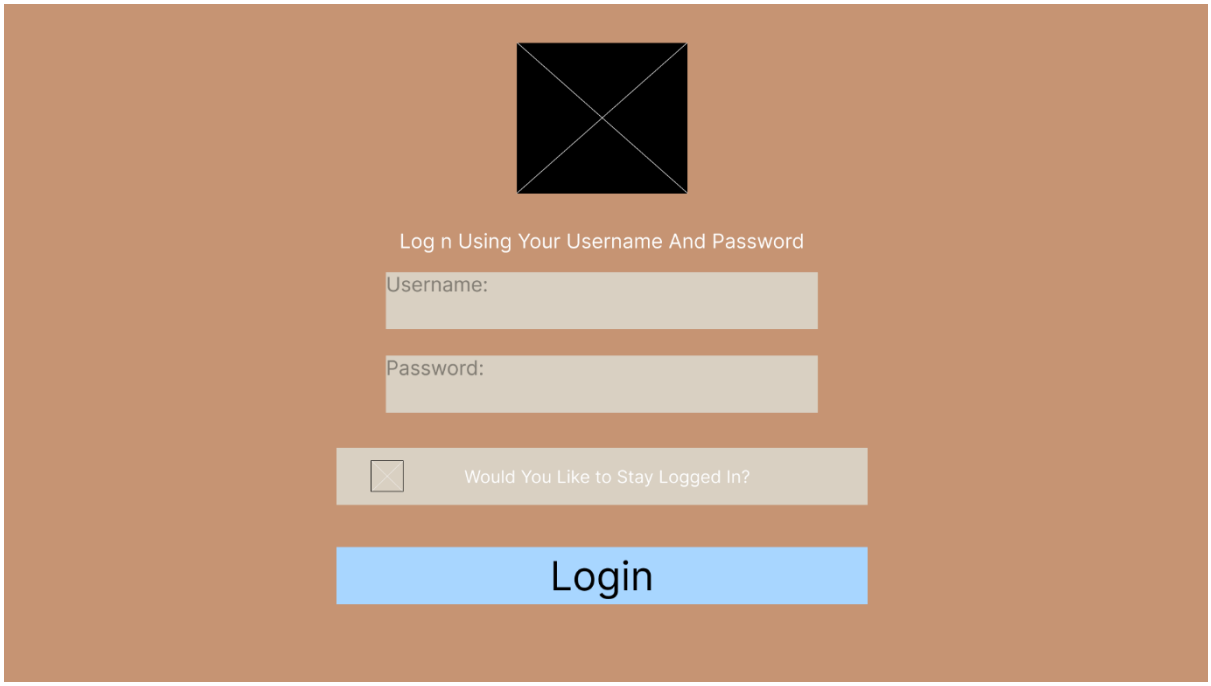



There are 5 options that the user can decide on in the main menu:

1) Sign up: Hyperlink to the sign-up page, allowing first time users to make an account.
2) Login: Hyperlink to the login page, allowing recurring users to log into the system.
3) Custom Rulesets: Hyperlink to the ruleset's configuration page and allows the user to export and import JSON files.
4) Create Lobby: Hyperlink to the lobby page.
5) Friends List: Hyperlink to the friends list page.

## Sign-up/Login:






Users can create lobbies and invite people to them even if not signed up, but they do not gain access to friends and game histories. Furthermore, users are asked for their preferred username, and the email they will use to verify their account. Finally, a password for authentication.

The Login screen also allows the user to stay logged in perpetually, using a cookie.

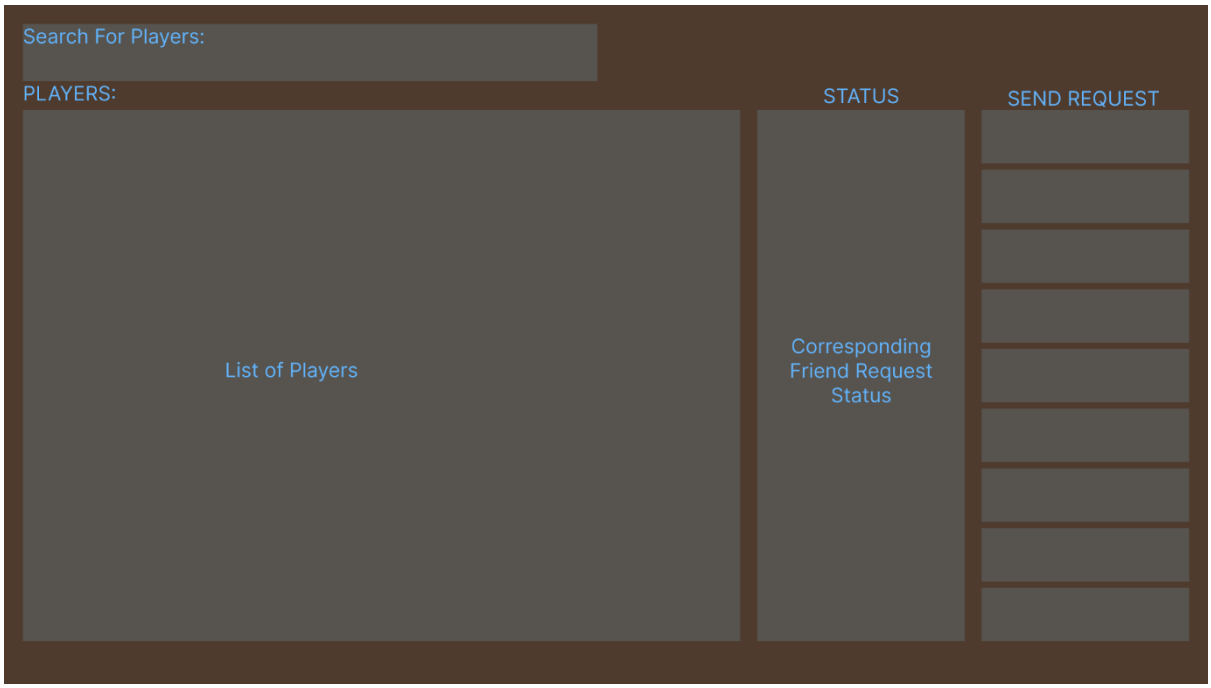
## Game Lobby:

The game lobby is made up of 5 distinct parts:

1) The players in the lobby, so that all users can confirm who is in currently.

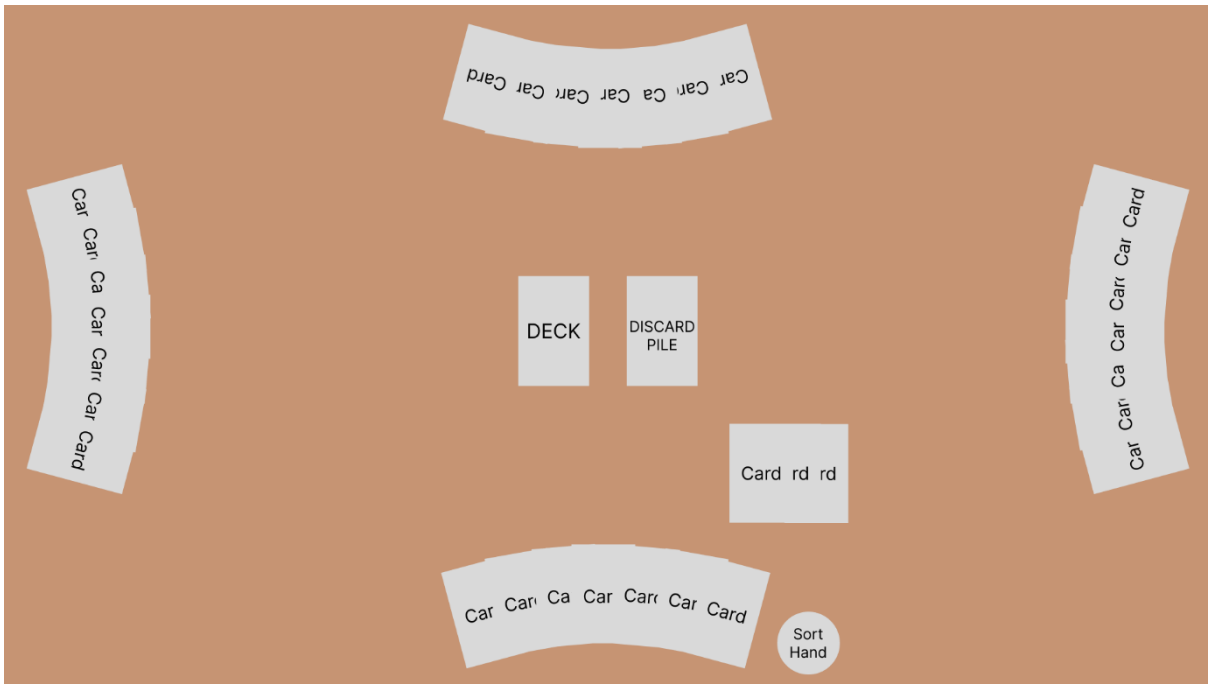
2) The INVITE hyperlink which allows them to invite a player based on their user Id or their username.
3) Ready! Allows the user to “ready up”, only when all players are ready can the game begin.
4) START begins the game.
5) Ruleset: This outlines the rules used in the game. The host can change rules, or import a JSON file.

## Friends List:




The friends list allows you to search for players, based on user Id or username, and send a friend request. It also lets you see the status of the request, either “not sent”, “pending”, “accepted”, or “declined”.

## GameState:

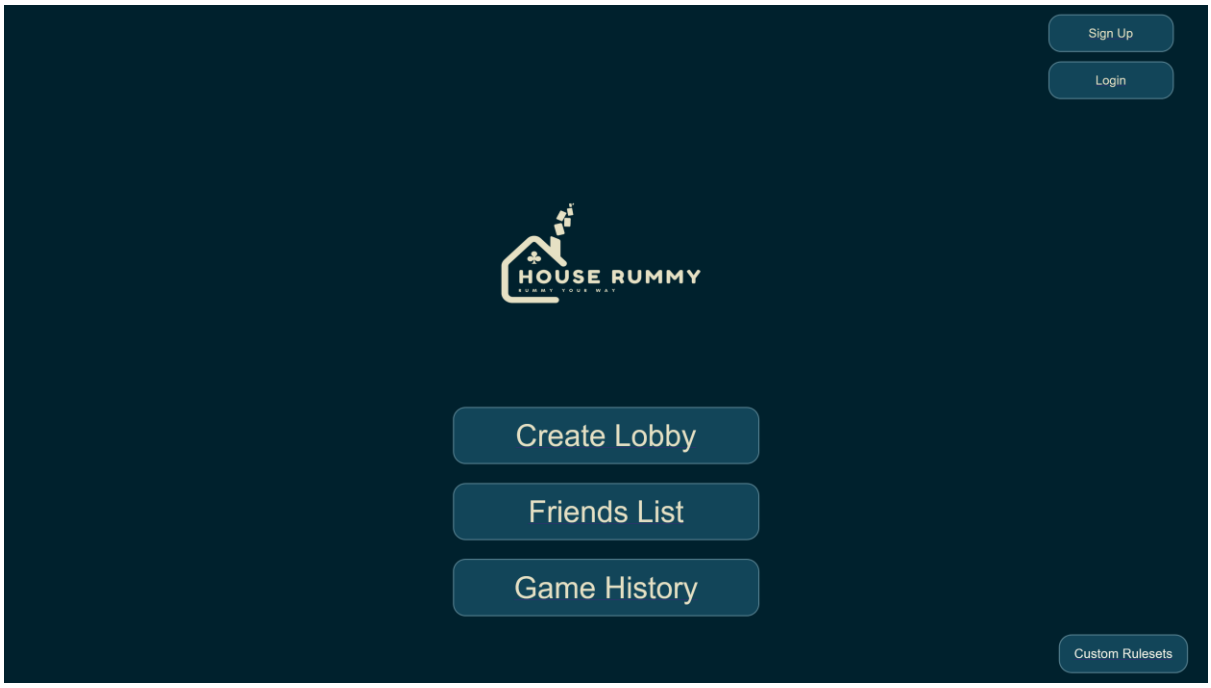



The gamestate allows players to see the current game board. Other players hands are shown face down, while the user's cards are shown face up. The sort-hand button allows the user to automatically sort their hand by either rank or suit. Furthermore, when hovering over other players hands, a small text box with the number of cards in that players hands is visible. You cand draw from the deck or discard pile by clicking on them. By clicking on a card in the hand, it is selected, either to be discarded, be put into a meld, or added to a meld on the table. Melds are place faced up on the table, where everyone can see or interact with them.

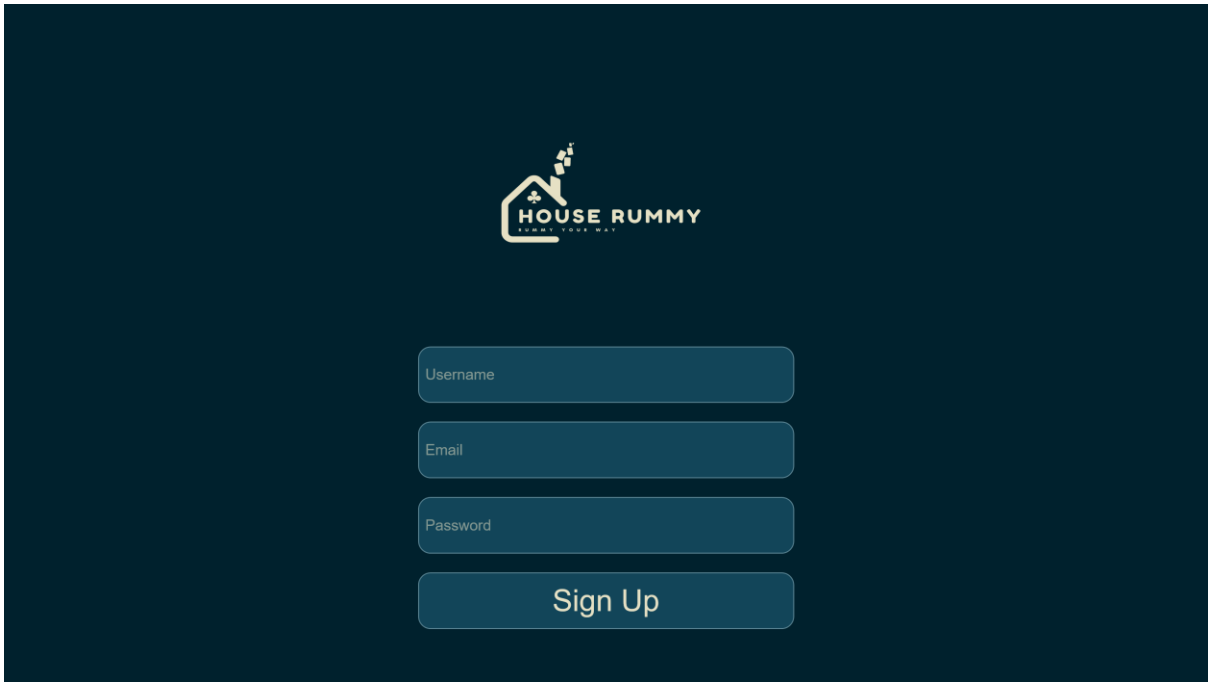
The cards in the hand are in a fan pattern, maximizing space while still allowing for all cards to be seen.

# Final Designs:

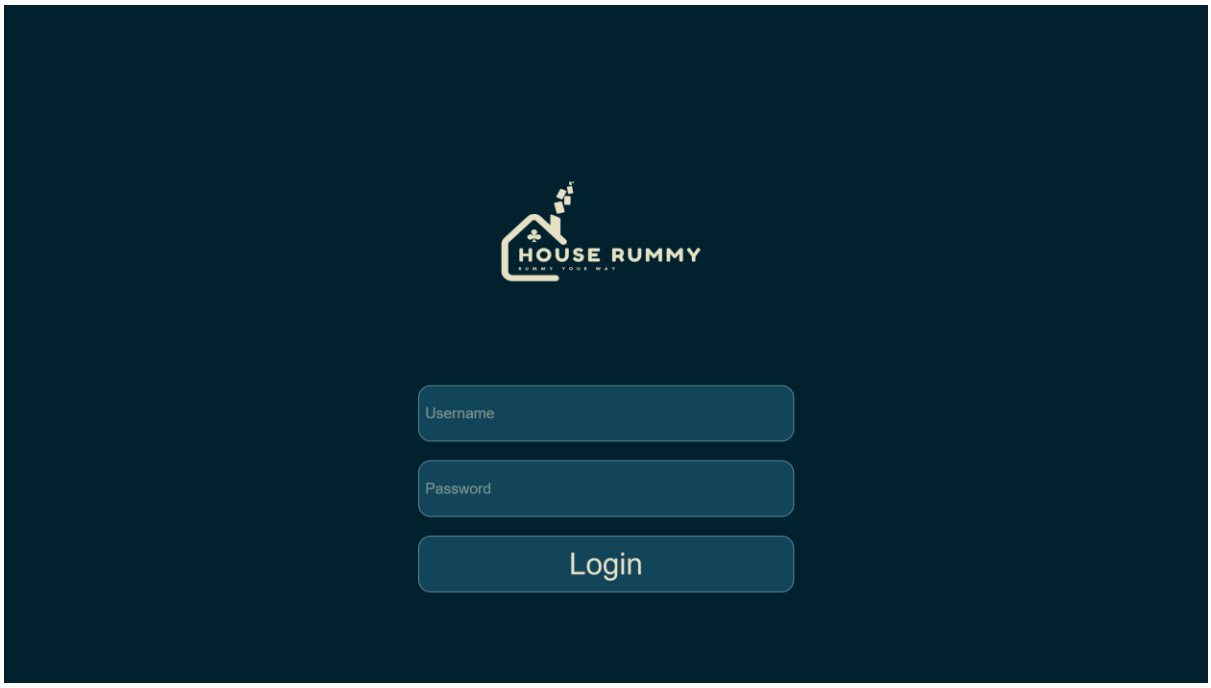
## Main Menu:




## Sign-up:



## Login:




## Custom Rulesets:

## Friends List:

# Technical Solution:

## Files & Skills used:

The following table summarises the files used in the project and the techniques used in each.

### Game Logic:

| File | Skills Used – (Group) | Evidence (Methods/Attributes) |
|---|---|---|
| utils.py | Mergesort or similarly efficient sorting algorithm (A)<br>Dictionaries (B) | Quicksort function, rank mappings. |
| card.py | Appropriate choice of simple data types (C)<br>Records/ Dictionaries (B) | Rank and suit attributes, to_dict method, JSON conversion |
| deck.py | Lists (A/B)<br>Stack Operations( (A) | Cards list. draw(), peel(), and add_card() allow it to act as a stack. |
| discard_pile.py | Stack Operations (A) | Cards list. push(). draw_top_card(), peek() allows it to act as a stack. |
| hand.py | List operations (A) | Adding, removing, and sorting algorithms: add_card(), remove_card(), select_card(), deselect_card(), sort_by_suit(). |
| meld.py | Complex user-defined algorithms (A)<br>Lists (A/B)<br>Dictionaries (B) | Wild card assignment, scoring, run and set logic, using wild_assignment_run() and wild_assignment_set(), and return_card_value(). The wild_assignments dictionary stores the assigned rank/suit/score data. |
| validator.py | Complex user-defined algorithms (A) | Meld validation and rule enforcement logic, such as validate_run(), validate_set(), validate_lay_off(), etc. |
| ruleset.py | Complex control models (A)<br>Dictionaries (B) | The ruleset is a configurable rule system with input validation and fail safes if it doesn't work. |
| player.py | Complex user-defined use of an OOP model (A)<br>Dynamic generation of objects (A) | Stores melds, interactions with hand and meld objects, using the __init__() method. The store_meld() method creates new meld objects at runtime. |
| game_state.py | Complex user-defined use of OOP model (A)<br>Complex user defined algorithms (A) | Coordinates between all previously defined objects, ruleset, deck, discard_pile, player, valiator, and melds. Further, the apply_move(), play_stored_melds are complex user-defined algorithms. The start_game() methods also dynamically creates the deck and discard_pile oblects |

| | | |
|---|---|---|
| | Dynamic generation of objects based on complex OOP model. (B) | |
| GameService.py | Complex client-server model (A)<br>Dynamic generation of objects based on the OOP model (A)<br>Dictionaries (B) | Manages active games and connects with the database layer to do everything. The create_game method dynmaically creates game_state objects dependinng on the rulesets and players in. Further, dictionaries are used to stores the active players and games. |

## Database Layer:

| File | Skills Used – (Group) | Evidence (Methods/Attributes) |
|---|---|---|
| DBConnections.py | Complex client-server model (A) | Abstracts the databse connection to execute queries efficiently. Execute() method runs those queries and the __init__() method establishes the DB connection. |
| Hashing.py | Hashing (A)<br>Complex user-defined algorithms (A) | simple_hash() is iterative bitwise and arithmetic hashing.<br>hash_password() is salted and uses iterative hashing with 1000 iterations. |
| UserRepository.py | Single parameterised SQL table (B)<br>Dictionaries/records (B) | CRUD operations and authentication querues, such as create_user(), update_username(), get_user_by_username(), etc. |
| GameRepository.py | Single parameterised SQL table (B) | CRUD operations and JSON serialisation, such as create_game(), get_game(), change_status(), and get_ruleset() |
| MoveRepository.py | Aggregate SQL functions (A), single table SQL (B)<br>Reading/writing structured data (B) | CRUD operations, and aggregate functions such as add_move(), which is inserting and JSON serialisation. get_move_count() is an aggregate function, as well as get_moves_by_game() which is an ordered select. |
| LinkRepository.py | Complex data model in a database (A)<br>Cross-table relational model (A)<br>Single table parameterised SQL (B) | Create_game_history links users with games, get_game_history_by_game/get_game_history_by_game() is relational retrieval for the game_history links. Create_friends_list() creates a relationship between two users, and update_friend_status() updates and inserts reciprocal relationship if accepted. |
| DatabaseService.py | Complex data model in a database(A)<br>Cross-table parameterised SQL (A)<br>Aggregate SQL functions (A) | get_pause_games_by_user() uses a JOIN query across tables.<br>add_move uses the move count before inserting<br>get_all_users_except uses sets and filers across the friendslist table.<br>delete_user() cascades across multiple tables to delete the user. |

| | | |
|---|---|---|
| | Complex client-server model (A) | |

## Frontend (Routes, sockets, and decorators):

| File | Skills Used – (Group) | Evidence (Methods/Attributes) |
|---|---|---|
| auth_decorators.py | Server-side scripting using requests and response objects (A) Server-side extensions for a complex client server model (A) | Creates decorators to restring access using registerd_only() and user_required(), using session checks and JSON responses. |
| game_decorators.py | Server-side scripting using requests and response objects (A) Server-side extensions for a complex client server model (A) | Creates decorators to restring access using in_game() and not_in_game(), using session checks. |
| auth_routes.py | Server-side scripting using requests and response objects (A) Server-side extensions for a complex client server model (A) | Using sign_up(), login(), guest_login(), etc, using session and JSON responses. |
| game_routes.py | Server-side scripting using requests and response objects (A) Server-side extensions for a complex client server model (A) | Using create_game(), join_game(), start_game(), and pause game(), using session and JSON responses, and returns JSON responses. |
| social_routes.py | Server-side scripting using requests and response objects (A) Server-side extensions for a complex client server model (A) | Using get_friends(), get_requests() and get_player_history() using session and JSON responses, and returns JSON responses. |
| game_events.py | Server-side scripting using requests and response objects (A) Server-side extensions for a complex client server model (A) | Using on_move(data), emit_game_state(), on_disconnect() ajd on_resume_game() using socket-io and server emits. |
| social_events.py | Server-side scripting using requests and response objects (A) Server-side extensions for a complex client server model (A) | Using on_friend_request(data), on_get_socials() amd on_invite_to_lobby(data) to handle real-time client events, using socket-io and JSON responses. |

# Game Logic Layer:

The python classes and utilites defined for the game logic, arranged from the lowest to highest layers of abstraction.

## Utilities:

The utility functions provide reusable algorithms, such as sorting and card classification, which are used by the other systems. It also provides records to be used in other systems.

```
from enum import Enum


def sort_rank(cards):
    """
    Return cards sorted by rank.

    Args:
        cards
    """
    return quicksort(cards, key=lambda card: card.return_rank_index())

def quicksort(items, key=lambda x: x):
    """
    Sort a list using quicksort with an optional key function.

    Args:
        items (list): Items to sort.
        key (function): Function to extract comparison value.

    Returns:
        list: Sorted list.
    """
    items = items.copy()

    if len(items) <= 1:
        return items

    pivot = items.pop()
    pivot_value = key(pivot)

    lesser = []
    greater = []

    for item in items:
        if key(item) > pivot_value:
            greater.append(item)
        else:
            lesser.append(item)

    return quicksort(lesser, key) + [pivot] + quicksort(greater, key)
```

```
def split_wilds_non_wilds(cards, ruleset):
    """
    Split a list of cards into wild and non-wild cards.

    Args:
        cards (list): The cards to split.
        ruleset: The active ruleset used to determine whether a card is wild.

    Returns:
        tuple: A tuple containing:
            - list: all wild cards
            - list: all non-wild cards
    """
    wilds = []
    non_wilds = []

    for card in cards:
        if ruleset.is_wild(card):
            wilds.append(card)
        else:
            non_wilds.append(card)

    return wilds, non_wilds

# Map each card rank to its index position.
# This is mainly used for meld validation and run checking.
rank_index = {
    "Ace": 0,
    "2": 1,
    "3": 2,
    "4": 3,
    "5": 4,
    "6": 5,
    "7": 6,
    "8": 7,
    "9": 8,
    "10": 9,
    "Jack": 10,
    "Queen": 11,
    "King": 12
}

# Reverse the rank_index dictionary so a rank name can be retrieved
# from its index. This is useful when assigning wild cards in runs.
index_rank = {value: key for key, value in rank_index.items()}

# Store the score value for each rank.
rank_score = {
    "Ace": 1,
    "2": 2,
    "3": 3,
    "4": 4,
    "5": 5,
    "6": 6,
    "7": 7,
```

```
    "8": 8,
    "9": 9,
    "10": 10,
    "Jack": 10,
    "Queen": 10,
    "King": 10
}

class Suit(Enum):
    """
    Represent the four card suits as an immutable enumeration.
    """
    Hearts = 0
    Clubs = 1
    Diamonds = 2
    Spades = 3

class Moves(Enum):
    """
    Represent the possible player actions in the game.

    Each enum value stores the move name used by the system.
    """
    Draw_Deck = "Draw_Deck"
    Draw_Discard = "Draw_Discard"
    Discard = "Discard"
    Store_Meld = "Store_Meld"
    Meld = "Meld"
    Store_Card = "Store_Card"
    Deselect_all = "Deselect_all"
    Lay_Off = "Lay_Off"
    Deck_Shuffle = "Deck_Shuffle"
    Sort_Rank = "Sort_Rank"
    Sort_Suit = "Sort_Suit"

class MeldTypes(Enum):
    """
    Represent the two possible meld types in the game.
    """
    RUN = "run"
    SET = "set"
```

## Card Class:

The Card class provides the underlying data structure used by the rest of the system, including the validation system, storing rank, suit, and scoring information.

```
from game_logic.utils import Suit, rank_index, rank_score

import json
```

```
class Card:
    def __init__(self, rank, suit, ruleset):
        """
        Initialises the card object, validating it against the known ranks known.
        Wild cards are validated differently than normal cards, as they do not require a suit.

        Args:
            rank (string): rank of the card, e.g. "Ace", or "8".
            suit (enum): suit of the card as a Suit enum value,
                Can be None for wildcards.
            ruleset (ruleset object): fcurrent ruleset, used for general validation.

        Returns:
            ValueError: If the rank is not valid
            ValueError: If  a non-wild card has an invalid suit.
        """

        wild_ranks = [wild[0] for wild in ruleset.wilds]

        if rank not in rank_index and rank not in wild_ranks:
        #A card must be either a standard rank or a wild rank.

            raise ValueError(f"{rank} is not a valid rank")

        if rank not in ruleset.wilds:
        #Non-wild cards must have a valid suit

            if suit not in Suit:
            #Checks if suit is a valid suit enum

                raise ValueError(f"{suit} is not a valid suit")


        self.rank = rank
        self.suit = suit
        self.ruleset = ruleset

        # Standard cards have an index used for ordering and run validation.
        # Wild cards do not have a fixed numeric index.
        if rank in rank_index:
            self.index = int(rank_index[rank])
        else:
            self.index = None

    def return_rank_index(self):
        """
        Returns the index of the rank of the card object, used for run validation


        Returns:
            Integer: index of rank
        """
        return int(rank_index[self.rank])

    def __repr__(self):
```

```
        """
        Returns a readable string representation for a card.

        Useful for debugging and testing,
        so that a card object can be printed in a human readable form.

        Returns:
            String: Rank of Suits, e.g. "King of Clubs"
        """
        if self.suit is None:
            return f"{self.rank}"
        return f"{self.rank} of {self.suit.name}"
    def __eq__(self, other):
        """
        Defines the equality of two card objects.
        Two cards are considered equal if they have the same rank and same suit

        Args:
            other (card object): The other card object being compared to.

        Returns:
            Boolean: True if equal, otherwise is false
        """

        if not isinstance(other, Card):
            return False
        return self.rank == other.rank and self.suit == other.suit

    def return_value(self):
        """
        Returns the default value of a card, used for deadwood calculations.

        Returns:
            Integer: Default Rank Value
        """

        if  self.ruleset.is_wild(self):
            return self.ruleset.wild_deadwood_socre
        else:
            return rank_score[self.rank]

    def to_dict(self):
        """
        Converts the card into a dictionary representation

        Used for serialising any move used so that card data can be sent
        ti the frontend or written to a file

        Returns:
            Dictionary: rank and suit key value pairs.
        """

        return {
            'rank': self.rank,
            'suit': self.suit.name if self.suit is not None else None
```

```
        }

    @classmethod
    def from_dict(cls, data, ruleset):
        """
        Classmethod, creates a card object from a dictionary,
        requires the same format as to_dict method.

        Args:
            Dictionayr: rank and suit key value pairs.

        Returns:
            Card object.
        """
        rank = data['rank']
        suit = Suit[data.get('suit')]
        return cls(rank, suit, ruleset)

    def to_json_file(self, filename):
        """
        Writes the card data to JSON file

        Args:
            self, filename

        Returns:
            JSON file
        """
        with open(filename, 'w') as file:
            json.dump(self.to_dict(), file, indent=4)
```

## Deck class:

The Deck class manages card generation dynamically based on the ruleset, ensuring that gameplay reflects the configurations. It also manages the retrieval of cards using stack-like operations.

```
from game_logic.utils import rank_index, Suit
from game_logic.card import Card
import random

local_rank_index = rank_index.copy()

class Deck:

    """
    Represents a deck of playing cards used in the game.

    The deck is constructed based on the current ruleset, including:
    - number of standard decks
    - wildcard types and counts

    A seeded random number generator is used to ensure deterministic
    shuffling when required.
    """
```

```
    def __init__(self, ruleset, seed=None):
        """
        Initialise a deck of cards based on the provided ruleset.

        The deck consists of:
        - Standard cards (rank + suit combinations)
        - Wildcards (with no suit, depending on ruleset)

        A random seed is used to allow reproducible shuffling.

        Args:
            ruleset (Ruleset): The ruleset defining deck composition.
            seed (int, optional): Seed for deterministic randomness.
                If None, a secure random seed is generated.
        """

        if seed is None:
            #Generates a secure 32bit seed using OS random generator
            seed = random.SystemRandom().randint(0, 2**32 -1)

        self.cards = []
        self.seed = seed
        self.rng = random.Random(seed)

        wild_ranks = [wild[0] for wild in ruleset.wilds]

        #Creates all standard cards
        for deck in range(ruleset.num_decks):
            for rank in rank_index:
                if rank not in wild_ranks:
                    for suit in Suit:
                        self.cards.append(Card(rank, suit, ruleset))

        #Add wildcard cards to deck
        for wild, num_wild in ruleset.wilds:
            for i in range(num_wild):
                self.cards.append(Card(wild, None, ruleset))



    def shuffle(self):
        """
        Shuffles the deck using a Fisher-Yates style algorithm.

        Ensures an unbiased shuffle using the seeded random generator.
        """
        for i in range(len(self.cards) -1, 0, -1):
            roll = self.rng.randint(0, i)
            self.cards[i], self.cards[roll] = self.cards[roll], self.cards[i]

    def peek(self):
        """
        Return the top card of the deck, without removing it.
```

```
        Returns:
            Card: top card of the deck
        """
        return self.cards[-1]

    def draw(self):
        """
        Remove and return the top card of the deck, simulating drawing a card.

        Returns:
            Card: The card removed from the  top of the deck.
        """
        return self.cards.pop()

    def size(self):
        """
        Returns the number of cards currently remaining in the deck.

        Returns:
            int: the number of cards in the deck
        """
        return len(self.cards)

    def empty_check(self):
        """
        Checks whether the deck is empty.

        Returns:
            bool: True if the deck contains no cards, otherwise False.
        """
        return self.size() == 0

    def add_cards(self, cards):
        """
        Add multiple cards to the deck.

        This is typically used when recycling cards from a discard pile
        back into the deck.

        Args:
            cards (list[Card]): A list of Card objects to add to the deck.
        """
        self.cards.extend(cards)
```

## Discard Pile class:

The Discard Pile class stores discarded cards and manages the retrieval of the cards through stacked base methods. This supports the validation system and gameplay flow.

```
class Discard_pile:
    """
    Represents the discard pile used during gameplay.

    The discard pile stores cards that have been discarded by players.
    The top card may be viewed or drawn, while the rest of the pile can
```

```
    be split off and reshuffled back into the deck when required.
    """

    def __init__(self):
        """
        Initialise an empty discard pile.
        """
        self.cards = []

    def __repr__(self):
        """
        Return a readable string representation of the discard pile.

        If the discard pile is empty, a message is returned instead.
        Otherwise, the top card is shown.

        Returns:
            str: A description of the top card, or a message if empty.
        """
        if self.is_empty():
            return "Empty discard pile"

        return f"Top card: {self.cards[-1]}"

    def push(self, card):
        """
        Add a card to the top of the discard pile.

        Args:
            card (Card): The card being discarded.
        """
        self.cards.append(card)

    def draw_top_card(self):
        """
        Remove and return the top card from the discard pile.

        Returns:
            tuple: (True, Card) if a card was successfully drawn,
                or (False, error_message) if the pile is empty.
        """
        if self.is_empty():
            return False, "Cannot draw from empty discard pile"

        return True, self.cards.pop()

    def split_discard_pile(self):
        """
        Remove all but the top card from the discard pile.

        This is used when the discard pile needs to be recycled back into
        the deck, while keeping the current top card visible.

        Returns:
            list[Card]: All cards except the top card.
```

```
        """

        if len(self.cards) <= 1:
            return []

        top_card = self.cards[-1]
        cards_to_reshuffle = self.cards[:-1]

        self.cards = [top_card]

        return cards_to_reshuffle

    def is_empty(self):
        """
        Check whether the discard pile is empty.

        Returns:
            bool: True if the discard pile contains no cards, otherwise False.
        """
        return len(self.cards) == 0

    def peek(self):
        """
        Return the top card of the discard pile without removing it.

        Returns:
            Card: The top card of the discard pile, None if pile is empty
        """
        if self.is_empty():
            return None
        return self.cards[-1]
```

## Hand class:

The Hand class manages the cards in control on the player, supporting user ineteraction, sorting, and move preparation.

```
from game_logic.utils import sort_rank, Suit

class Hand:
    """
    Represents a player's hand of cards.

    The hand stores both:
    - all cards currently held by the player
    - any cards currently selected for actions such as meld building

    It also provides utility methods for sorting, swapping, selecting,
    and calculating the total deadwood score of the hand.
    """

    def __init__(self):
        """
        Initialise an empty hand.
```

```
        Attributes:
            cards (list[Card]): All cards currently in the player's hand.
            selected_cards (list[Card]): Cards currently selected by the player.
        """
        self.cards = []
        self.selected_cards = []

    def __repr__(self):
        """
        Return a readable string representation of the hand.

        If cards have been selected, they are shown separately beneath the
        main hand contents. This is mainly useful for debugging and testing.

        Returns:
            str: A formatted string showing the hand contents.
        """
        card_strings = [str(card) for card in self.cards]

        if len(self.selected_cards) > 0:
            selected_card_strings = [str(card) for card in self.selected_cards]
            return ', '.join(card_strings) + "\nSelected Cards: " + ', '.join(selected_card_strings)

        return ', '.join(card_strings)

    def add_card(self, card):
        """
        Add a card to the player's hand.

        Args:
            card (Card): The card to add.
        """
        self.cards.append(card)

    def remove_card(self, card):
        """
        Remove a card from the player's hand.

        Args:
            card (Card): The card to remove.
        """
        self.cards.remove(card)

    def select_card(self, card):
        """
        Mark a card as selected.

        Selected cards are stored separately so that the player can build
        melds or perform actions without immediately changing the hand.

        Args:
            card (Card): The card to select.
        """
        if card not in self.selected_cards:
            self.selected_cards.append(card)
```

```
    def deselect_card(self, card):
        """
        Remove a card from the selected cards list.

        Args:
            card (Card): The card to deselect.
        """
        if card in self.selected_cards:
            self.selected_cards.remove(card)

    def deselect_all(self):
        """
        Clear all currently selected cards.
        """
        self.selected_cards = []

    def sort_by_rank(self):
        """
        Sort the hand by rank.

        The sorting logic is delegated to the shared sort_rank utility
        function so that card ordering is consistent across the program.
        """
        self.cards = sort_rank(self.cards)

    def sort_by_suit(self):
        """
        Sort the hand by suit, then by rank within each suit.

        This groups cards by suit first, then sorts each suit group by rank.
        """
        self.cards = self._sort_suit(self.cards)

    def swap_card(self, card1, card2):
        """
        Swap the positions of two cards in the hand.

        This may be useful for manual card ordering in future UI features.

        Args:
            card1 (Card): The first card to swap.
            card2 (Card): The second card to swap.
        """
        index1 = self.cards.index(card1)
        index2 = self.cards.index(card2)

        self.cards[index1], self.cards[index2] = self.cards[index2], self.cards[index1]

    def _sort_suit(self, items):
        """
        Sort a list of cards by suit, then by rank.

        Cards are first split into separate suit groups, then each group is
        sorted by rank before the full list is recombined.
```

```
        Args:
            items (list[Card]): The cards to sort.

        Returns:
            list[Card]: A new list of cards sorted by suit then rank.
        """

        #Splits the cards into lists, each containing one suit
        items_hearts = []
        items_clubs = []
        items_diamonds = []
        items_spades = []

        for card in items:
            if card.suit == Suit.Hearts:
                items_hearts.append(card)
            elif card.suit.value == Suit.Clubs:
                items_clubs.append(card)
            elif card.suit.value == Suit.Diamonds:
                items_diamonds.append(card)
            elif card.suit.value == Suit.Spades:
                items_spades.append(card)

        #Sorts each list of cards individually, before adding them together
        return (
            sort_rank(items_hearts)
            + sort_rank(items_clubs)
            + sort_rank(items_diamonds)
            + sort_rank(items_spades)
        )

    def is_empty(self):
        """
        Check whether the hand contains any cards.

        Returns:
            bool: True if the hand is empty, otherwise False.
        """
        return len(self.cards) == 0

    def calculate_deadwood(self, ruleset):
        """
        Calculate the deadwood score of the hand.

        Wild cards use the wildcard deadwood value from the ruleset.
        All other cards use their standard deadwood value.

        Args:
            ruleset (Ruleset): The active ruleset used for scoring.

        Returns:
            int: The total deadwood score of the hand.
        """
        deadwood_score = 0
```

```
        for card in self.cards:
            if ruleset.is_wild(card):
                deadwood_score += ruleset.wild_deadwood_score
            else:
                deadwood_score += card.return_value()

        return deadwood_score
```

## Meld class:

The meld class represents valid card combinations for melds and handles wildcard assignment and scoring, supporting the central validation and ruleset system.

```
from game_logic.utils import index_rank, Suit, rank_score, MeldTypes,
split_wilds_non_wilds, quicksort

class Meld:
    """
    Represent a single meld in the rummy game.

    A meld can be either a set or a run. The class stores the cards in the meld,
    its type, and any wild card assignments needed to make the meld valid.

    Attributes:
        cards (list): The cards currently in the meld.
        meld_type: The type of meld, usually set or run.
        wild_assignments (dict): Maps each wild card to the card details it is
            representing in the meld.
    """

    def __init__(self, cards, meld_type, ruleset):
        """
        Initialise a Meld object.

        Args:
            cards (list): The cards that make up the meld.
            meld_type: The meld type, such as set or run.
            ruleset: The ruleset object controlling allowed meld behaviour.
        """
        self.cards = cards
        self.meld_type = meld_type
        self.wild_assignments = {}

        # Only assign wild cards if the meld type is allowed by the ruleset.
        if self.meld_type == MeldTypes.RUN:
            if ruleset.allow_runs:
                self._wild_assignment_run(ruleset)
        else:
            if ruleset.allow_sets:
                self._wild_assignment_set(ruleset)

    def __repr__(self):
        """
        Return a string representation of the meld.
```

```
        Returns:
            str: A comma-separated string of the cards in the meld.
        """
        card_strings = [str(card) for card in self.cards]
        return ",".join(card_strings)

    def _wild_assignment_run(self, ruleset):
        """
        Assign values to wild cards when the meld is a run.

        Wild cards are first used to fill gaps between non-wild cards.
        Any remaining wild cards are then assigned to extend the run,
        defaulting to the lower end.

        Args:
            ruleset: The active ruleset used to identify wild cards.
        """
        wilds, non_wilds = split_wilds_non_wilds(self.cards, ruleset)

        # Build and sort the rank indices of all non-wild cards.
        indices = []
        for card in non_wilds:
            indices.append(card.return_rank_index())

        indices = quicksort(indices)

        # Create the full expected range of indices across the run.
        range_indexes = range(indices[0], indices[-1] + 1)

        # Identify any missing ranks between the lowest and highest cards.
        missing_indices = []
        for index in range_indexes:
            if index not in indices:
                missing_indices.append(index)

        # Assign wild cards to fill missing ranks inside the run.
        assigned_wilds = []

        for wild, missing_index in zip(wilds, missing_indices):
            self.wild_assignments[wild] = {
                'score': rank_score[index_rank[missing_index]],
                'rank_index': missing_index,
                'rank': index_rank[missing_index],
                'suit': non_wilds[0].suit
            }
            assigned_wilds.append(wild)

        # Remove wild cards that have already been assigned.
        for wild in assigned_wilds:
            wilds.remove(wild)

        # Assign any remaining wild cards to extend the run downward.
        if len(wilds) > 0:
            for i, wild in enumerate(wilds):
```

```
            self.wild_assignments[wild] = {
                'score': rank_score[index_rank[min(indices) - (len(wilds) - i)]],
                'rank_index': min(indices) - (len(wilds) - i),
                'rank': index_rank[min(indices) - (len(wilds) - i)],
                'suit': non_wilds[0].suit
            }

    def _wild_assignment_set(self, ruleset):
        """
        Assign values to wild cards when the meld is a set.

        Wild cards take the same rank and score as the non-wild cards,
        and are assigned suits that are not already represented.

        Args:
            ruleset: The active ruleset used to identify wild cards.
        """
        wilds, non_wilds = split_wilds_non_wilds(self.cards, ruleset)

        # Collect the suits already present in the non-wild cards.
        non_wild_suits = []

        for card in non_wilds:
            non_wild_suits.append(card.suit)

        # Find suits not yet used in the set.
        missing_suits = []

        for suit in Suit:
            if suit not in non_wild_suits:
                missing_suits.append(suit)

        # Assign each wild card one of the missing suits.
        for i, wild in enumerate(wilds):
            self.wild_assignments[wild] = {
                'score': non_wilds[0].return_value(),
                'rank_index': non_wilds[0].return_rank_index(),
                'rank': non_wilds[0].rank,
                'suit': missing_suits[i]
            }

    def return_card_value(self, card, ruleset):
        """
        Return the score value of a card within this meld.

        Wild cards use their assigned replacement value.
        Aces may be scored as high or low depending on meld type and rules.

        Args:
            card: The card whose value is being calculated.
            ruleset: The active ruleset controlling scoring.

        Returns:
            int: The score value of the card in this meld.
        """
```

```
        if ruleset.is_wild(card):
            return self.wild_assignments[card]['score']

        if card.rank == "Ace":
            if self.meld_type == "set":
                if ruleset.ace_both or ruleset.ace_high:
                    return ruleset.ace_high_score
                else:
                    return 1

            if self.meld_type == "run":
                if self._is_ace_high_in_run(card, ruleset):
                    return ruleset.ace_high_score
                else:
                    return 1

        return card.return_value()

    def _is_ace_high_in_run(self, ace_card, ruleset):
        """
        Determine whether an Ace should be treated as high in a run.

        An Ace is treated as high if the run contains a King, either as a
        normal card or through a wild card assignment.

        Args:
            ace_card: The Ace card being checked.
            ruleset: The active ruleset used to identify wild cards.

        Returns:
            bool: True if the Ace should be scored high, otherwise False.
        """
        other_cards = [
            card for card in self.cards
            if card.rank != "Ace" and not ruleset.is_wild(card)
        ]

        has_king = any(card.rank == "King" for card in other_cards)

        for wild in self.wild_assignments:
            if self.wild_assignments[wild]['rank'] == "King":
                has_king = True
                break

        return has_king

    def return_meld_value(self, ruleset):
        """
        Calculate the total score of the meld.

        Args:
            ruleset: The active ruleset controlling scoring.

        Returns:
            int: The total value of all cards in the meld.
```

```
        """
        score = 0

        for card in self.cards:
            score += self.return_card_value(card, ruleset)

        return score

    def add_card(self, card):
        """
        Add a card to the meld.

        Args:
            card: The card to add.
        """
        self.cards.append(card)

    def wild_card_comparison(self, card, wild, ruleset):
        """
        Check whether a card can replace a wild card in the meld.

        Replacement is only allowed if the ruleset permits it, and if the
        card matches the rank and suit currently assigned to the wild card.

        Args:
            card: The card being compared.
            wild: The wild card currently in the meld.
            ruleset: The active ruleset controlling replacement behaviour.

        Returns:
            bool: True if the card can replace the wild card, otherwise False.
        """
        if ruleset.allow_wild_replacement:
            wild_assignment = self.wild_assignments[wild]
            return (
                card.rank == wild_assignment['rank']
                and card.suit == wild_assignment['suit']
            )
        else:
            return False

    def replace_wild(self, card, wild):
        """
        Replace a wild card in the meld with a normal card.

        The wild card is removed from the meld, the new card is added,
        and the stored wild assignment is deleted.

        Args:
            card: The card replacing the wild card.
            wild: The wild card being removed.

        Returns:
            The removed wild card.
        """
```

```
        self.cards.remove(wild)
        self.cards.append(card)

        if wild in self.wild_assignments:
            del self.wild_assignments[wild]

        return wild
```

## Player class:

The Player class stores player identity, the hand, score, and meld state. This supports game state management and move processing.

```
from game_logic.validator import Validator

from game_logic.meld import Meld


class Player:
    """
    Represent a player in the game.

    This class stores the player's identity, hand, score, and any melds or
    cards they are currently working with during their turn.

    Attributes:
        user_id: A unique identifier for the player.
        name (str): The player's name.
        hand: The player's hand object.
        current_stored_melds (list): Melds created during the current turn
            that are not yet fully committed.
        completed_stored_melds (list): Melds that have been completed and stored.
        stored_cards (list): Cards currently selected for creating a meld.
        ready (bool): Whether the player is ready to begin.
        score (int): The player's current score.
        has_melded (bool): Whether the player has made their opening meld.
        has_drawn (bool): Whether the player has drawn a card this turn.
        drawn_from_discard: Stores whether or what was drawn from the discard pile.
    """

    def __init__(self, user_id, name, hand):
        """
        Initialise a Player object.

        Args:
            user_id: A unique identifier for the player.
            name (str): The player's name.
            hand: The player's hand object.
        """
        self.user_id = user_id
        self.name = name
        self.hand = hand
        self.current_stored_melds = []
        self.completed_stored_melds = []
        self.stored_cards = []
```

```
        self.ready = False
        self.score = 0
        self.has_melded = False
        self.has_drawn = False
        self.drawn_from_discard = None

    def store_meld(self, ruleset):
        """
        Validate the currently stored cards and create a meld if valid.

        The selected stored cards are checked using the validator. If valid,
        a Meld object is created and added to the player's current stored melds.

        Args:
            ruleset: The active ruleset used to validate the meld.

        Returns:
            tuple:
                - (False, meld_type) if the cards do not form a valid meld.
                - (True, meld) if the meld is created successfully.
        """
        is_valid, meld_type = Validator.validate_meld(self.stored_cards, ruleset)

        if not is_valid:
            return False, meld_type

        meld = Meld(self.stored_cards, meld_type, ruleset)
        self.current_stored_melds.append(meld)

        # Clear all selected cards after successfully storing the meld.
        self.deselect_all()
        return True, meld

    def select_card(self, card):
        """
        Select a card from the player's hand.

        Args:
            card: The card to select.
        """
        self.hand.select_card(card)

    def deselect_card(self, card):
        """
        Deselect a card from the player's hand.

        Args:
            card: The card to deselect.
        """
        self.hand.deselect_card(card)

    def deselect_all(self):
        """
        Deselect all currently selected cards in the player's hand.
        """
```

```
        self.hand.deselect_all()

    def reset_current_stored_melds(self):
        """
        Clear all melds currently stored for this turn.
        """
        self.current_stored_melds = []

    def return_stored_melds_score(self, ruleset):
        """
        Calculate the total score of the player's current stored melds.

        Args:
            ruleset: The active ruleset used to score melds.

        Returns:
            int: The combined score of all current stored melds.
        """
        score = 0
        for meld in self.current_stored_melds:
            score += meld.return_meld_value(ruleset)

        return score

    def sort_rank(self):
        """
        Sort the player's hand by rank.
        """
        self.hand = self.hand.sort_by_rank()

    def sort_suit(self):
        """
        Sort the player's hand by suit.
        """
        self.hand = self.hand.sort_by_suit()

    def add_to_score(self, ruleset, points):
        """
        Update the player's score according to the scoring method.

        In negative scoring, points are added. Otherwise, points are subtracted.

        Args:
            ruleset: The active ruleset controlling score behaviour.
            points (int): The number of points to apply.
        """
        if ruleset.scoring_method == 'negative':
            self.score += points
        else:
            self.score -= points

    def get_stored_cards(self):
        """
        Return the cards currently stored by the player.
```

```
        Returns:
            list: The player's stored cards.
        """
        return self.stored_cards

    def reset_player(self):
        """
        Reset the player's game state for a new round or game.

        This clears the player's hand, stored melds, score, and turn-related
        status flags.
        """
        self.hand.cards = []
        self.completed_stored_melds = []
        self.current_stored_melds = []
        self.score = 0
        self.has_melded = False
        self.has_drawn = False
        self.drawn_from_discard = False
```

## Ruleset class:

The Ruleset class stores configurable game rules and validates those configurations. This is the main class supporting the customisable ruleset objective.

```
import json

class Ruleset:
    """
    Store and validate all configurable game rules for the rummy platform.

    A Ruleset object is created from an optional configuration dictionary.
    Each setting is validated before being stored, and defaults are used
    whenever a value is missing or invalid.

    Attributes:
        allow_sets (bool): Whether set melds are allowed.
        allow_runs (bool): Whether run melds are allowed.
        max_meld_size_run (int | None): Maximum size allowed for a run meld.
        min_initial_meld_score (int): Minimum score needed for the initial meld.
        initial_meld_increment (bool): Whether the initial meld requirement
            increases over time.
        wilds (list): A list of cards treated as wild cards.
        wild_deadwood_score (int): Deadwood score assigned to wild cards.
        scoring_method (str): The scoring system used in the game.
        ace_low (bool): Whether aces can be used as low cards.
        ace_high (bool): Whether aces can be used as high cards.
        ace_both (bool): Whether aces can be used as both high and low.
        ace_wrap_around (bool): Whether aces can wrap around in runs.
        ace_high_score (int): Score value used when an ace is high.
        initial_hand_size (int): Number of cards dealt to each player.
        min_meld_size (int): Minimum number of cards required in a meld.
        max_meld_size_set (int): Maximum size allowed for a set meld.
        num_decks (int): Number of decks used in the game.
        require_melding_to_draw_from_disc (bool): Whether a player must meld
```

```
        before drawing from the discard pile.
    require_melding_to_lay_off (bool): Whether a player must meld before
        laying off cards.
    allow_wild_replacement (bool): Whether wild cards can be replaced.
    allow_wild_only_melds (bool): Whether melds made entirely of wild cards
        are allowed.
    prevent_discard_same_card (bool): Whether the card just drawn can be
        discarded immediately.
    points_for_winning (int): Bonus points awarded for winning.
    max_deck_shuffle (int | None): Maximum number of times the deck can be
        reshuffled.
    winner_deadwood (int): Deadwood score awarded or counted for the winner.
    max_players (int): Maximum number of players allowed.
"""

def __init__(self, config=None):
    """
    Initialise a Ruleset object from a configuration dictionary.

    Args:
        config (dict | None): A dictionary of rule values. If None,
            default values are used.
    """
    config = config or {}

    # Control which meld types are allowed. At least one should be enabled
    # or the game cannot be completed.
    self.allow_sets = self._get_validated(
        config,
        'allow_sets',
        True,
        bool
    )

    self.allow_runs = self._get_validated(
        config,
        'allow_runs',
        True,
        bool
    )

    # Define meld size restrictions and initial meld requirements.
    self.max_meld_size_run = self._get_validated(
        config,
        'max_meld_size_run',
        None,
        int,
        None,
        1,
        None
    )

    self.min_initial_meld_score = self._get_validated(
        config,
        'min_initial_meld_score',
```

```
        0,
        int,
        None,
        0,
        None
    )

    self.initial_meld_increment = self._get_validated(
        config,
        'initial_meld_increment',
        False,
        bool
    )

    # Store the cards that count as wild cards.
    self.wilds = self._get_validated(
        config,
        'wilds',
        [("Joker", 0)],
        list
    )

    self.wild_deadwood_score = self._get_validated(
        config,
        'wild_deadwood_score',
        25,
        int,
        None,
        0,
        None
    )

    # Choose the scoring method for the game.
    self.scoring_method = self._get_validated(
        config,
        'scoring_method',
        'negative',
        str,
        ['negative', 'positve']
    )

    # Control how aces behave in runs and scoring.
    self.ace_low = self._get_validated(
        config,
        'ace_low',
        False,
        bool
    )

    self.ace_high = self._get_validated(
        config,
        'ace_high',
        False,
        bool
    )
```

```
        self.ace_both = self._get_validated(
            config,
            'ace_both',
            True,
            bool
        )

        self.ace_wrap_around = self._get_validated(
            config,
            'wrap_around',
            False,
            bool
        )

        self.ace_high_score = self._get_validated(
            config,
            'ace_high_score',
            10,
            int,
            None,
            1
        )

        # Set core deck, hand, and meld sizing rules.
        self.initial_hand_size = self._get_validated(
            config,
            'initial_hand_size',
            14,
            int,
            None,
            1
        )

        self.min_meld_size = self._get_validated(
            config,
            'min_meld_size',
            3,
            int,
            None,
            1,
            self.initial_hand_size
        )

        self.max_meld_size_set = self._get_validated(
            config,
            'max_meld_size_set',
            4,
            int,
            None,
            1,
            self.initial_hand_size
        )

        self.num_decks = self._get_validated(
```

```
        config,
        'num_decks',
        2,
        int,
        None,
        1
    )

    # Control turn and meld interaction rules.
    self.require_melding_to_draw_from_disc = self._get_validated(
        config,
        'require_melding_to_draw_from_disc',
        True,
        bool
    )

    self.require_melding_to_lay_off = self._get_validated(
        config,
        'require_melding_to_lay_off',
        True,
        bool
    )

    self.allow_wild_replacement = self._get_validated(
        config,
        'allow_wild_replacement',
        True,
        bool
    )

    self.allow_wild_only_melds = self._get_validated(
        config,
        'allow_wild_only_melds',
        False,
        bool
    )

    self.prevent_discard_same_card = self._get_validated(
        config,
        'prevent_discard_same_card',
        True,
        bool
    )

    # Set scoring and game flow limits.
    self.points_for_winning = self._get_validated(
        config,
        'points_for_winning',
        25,
        int
    )

    self.max_deck_shuffle = self._get_validated(
        config,
        'max_deck_shuffle',
```

```
        None,
        int
    )

    self.winner_deadwood = self._get_validated(
        config,
        'winner_deadwood',
        25,
        int
    )

    self.max_players = self._get_validated(
        config,
        'max_players',
        4,
        int
    )

  def _get_validated(
    self,
    config,
    key,
    default,
    expected_type,
    allowed_values=None,
    min_value=None,
    max_value=None
  ):
    """
    Retrieve and validate a configuration value.

    If the value is missing, the default is used. If the value has the
    wrong type, is not in the allowed values list, or falls outside the
    allowed numeric range, the default is also used.

    Args:
        config (dict): The configuration dictionary.
        key (str): The key to retrieve.
        default: The fallback value if validation fails.
        expected_type: The required type for the value.
        allowed_values (list | None): Specific allowed values.
        min_value (int | None): Minimum allowed numeric value.
        max_value (int | None): Maximum allowed numeric value.

    Returns:
        The validated value, or the default if validation fails.
    """
    value = config.get(key, default)

    if value is not None and not isinstance(value, expected_type):
        return default

    if allowed_values is not None and value not in allowed_values:
        return default
```

```
        if min_value is not None and value < min_value:
            return default

        if max_value is not None and value > max_value:
            return default

        return value

    def is_wild(self, card):
        """
        Check whether a card is treated as a wild card.

        Args:
            card: The card to test.

        Returns:
            bool: True if the card's rank matches one of the configured
            wild cards, otherwise False.
        """
        return any(card.rank == t[0] for t in self.wilds)

    def to_dict(self):
        """
        Export the ruleset as a dictionary.

        This is useful for saving the ruleset or passing it between parts
        of the program.

        Returns:
            dict: A dictionary representation of the ruleset.
        """
        return {
            'allow_sets': self.allow_sets,
            'allow_runs': self.allow_runs,
            'min_meld_size': self.min_meld_size,
            'max_meld_size_set': self.max_meld_size_set,
            'max_meld_size_run': self.max_meld_size_run,
            'min_initial_meld_score': self.min_initial_meld_score,
            'initial_meld_increment': self.initial_meld_increment,
            'initial_hand_size': self.initial_hand_size,
            'num_decks': self.num_decks,
            'wilds': self.wilds,
            'scoring_method': self.scoring_method,
            'ace_high': self.ace_high,
            'ace_both': self.ace_both,
            'ace_high_score': self.ace_high_score
        }

    def to_json_file(self, filename):
        """
        Save the ruleset to a JSON file.

        Args:
            filename (str): The name of the file to write to.
        """
```

```
        with open(filename, 'w') as file:
            json.dump(self.to_dict(), file, indent=4)

    @classmethod
    def from_json_file(cls, filename):
        """
        Create a Ruleset object from a JSON file.

        Args:
            filename (str): The name of the JSON file to read.

        Returns:
            Ruleset: A new Ruleset object built from the file data.
        """
        with open(filename, 'r') as file:
            data = json.load(file)
        return cls(data)

    @classmethod
    def from_dict(cls, dictionary):
        """
        Create a Ruleset object from a dictionary.

        Args:
            dictionary (dict): A dictionary containing ruleset values.

        Returns:
            Ruleset: A new Ruleset object built from the dictionary.
        """
        return cls(dictionary)
```

## Validator Class:

The validator class contains all central rule checking logic, directly achieving the central validation system objective.

```
from game_logic.utils import quicksort, split_wilds_non_wilds, MeldTypes

class Validator:
    """
    Provide static validation methods for all game actions.

    This includes validating melds (sets and runs), player actions such as
    drawing and discarding, and rule-based constraints.
    """

    @staticmethod
    def validate_set(meld, ruleset):
        """
        Validate whether a meld forms a valid set.

        A valid set must:
        - Be of type SET
        - Meet size constraints
        - Contain cards of the same rank
```

```
        - Contain no duplicate suits (excluding wilds)

        Args:
            meld (list): Cards in the meld.
            ruleset: Active ruleset.

        Returns:
            tuple: (True, MeldTypes.SET) if valid, otherwise (False, reason).
        """
        if meld.meld_type != MeldTypes.SET:
            return False, "Not Valid Meld Type"

        if ruleset.max_meld_size_set is not None and len(meld) > ruleset.max_meld_size_set:
            return False, "Meld Too Large"
        if len(meld) < ruleset.min_meld_size:
            return False, "Meld Too Small"

        suits = []
        ranks = []

        # Collect rank and suit information for non-wild cards only.
        for card in meld:
            if not ruleset.is_wild(card):
                ranks.append(card.rank)
                suits.append(card.suit)

        # Prevent sets made entirely of wild cards if not allowed.
        if not ruleset.allow_wild_only_melds and len(ranks) == 0:
            return False, "Meld Contains Only Wild Cards"

        # All non-wild cards must have the same rank.
        if len(set(ranks)) != 1:
            return False, "Multiple Ranks in 1 Set"

        # All suits must be unique.
        if len(suits) != len(set(suits)):
            return False, "Multiple of the Same Suit in 1 Set"

        return True, MeldTypes.SET

    @staticmethod
    def validate_run(meld, ruleset):
        """
        Validate whether a meld forms a valid run.

        A valid run must:
        - Be of type RUN
        - Meet size constraints
        - Have cards of the same suit
        - Form a consecutive sequence, with wilds filling gaps
        - Respect ace rules (low, high, wrap-around)

        Args:
            meld (list): Cards in the meld.
            ruleset: Active ruleset.
```

```
    Returns:
        tuple: (True, MeldTypes.RUN) if valid, otherwise (False, reason).
    """
    if meld.meld_type != MeldTypes.RUN:
        return False, "Not Valid Meld Type"

    if ruleset.max_meld_size_run is not None and len(meld) > ruleset.max_meld_size_run:
        return False, "Meld Too Large"

    # FIXED: this was '>' instead of '<'
    if len(meld) < ruleset.min_meld_size:
        return False, "Meld Too Small"

    wilds, non_wilds = split_wilds_non_wilds(meld, ruleset)

    # Prevent all-wild runs if not allowed.
    if not ruleset.allow_wild_only_melds and len(non_wilds) == 0:
        return False, "Meld Is All Wilds"

    indices = []
    suits = []

    # Extract suit and rank indices for non-wild cards.
    for card in non_wilds:
        suits.append(card.suit)
        indices.append(card.return_rank_index())

    indices = quicksort(indices)

    # All cards must be the same suit.
    if len(set(suits)) != 1:
        return False, "Multiple suits"

    # No duplicate ranks allowed.
    if len(indices) != len(set(indices)):
        return False, "Multiples of the same rank"

    # Calculate gaps between lowest and highest rank.
    min_index = min(indices)
    max_index = max(indices)
    span = max_index - min_index + 1
    gaps = span - len(non_wilds)

    # Check if wilds can fill the gaps.
    if gaps <= len(wilds):
        return True, MeldTypes.RUN

    # Handle ace-related edge cases.
    has_ace = 0 in indices
    has_king = 12 in indices

    if has_ace:
        if ruleset.ace_high or ruleset.ace_both:
```

```
            # Treat Ace as high (index 13 instead of 0).
            adjusted_indices = []
            for index in indices:
                if index == 0:
                    adjusted_indices.append(13)
                else:
                    adjusted_indices.append(index)

            adjusted_indices = quicksort(adjusted_indices)

            min_index = min(adjusted_indices)
            max_index = max(adjusted_indices)
            span = max_index - min_index + 1
            gaps = span - len(non_wilds)

        if gaps <= len(wilds):
            return True, MeldTypes.RUN

        # Handle wrap-around runs (e.g. King, Ace, 2).
        if ruleset.ace_wrap_around and has_king:

            low_indices = [index for index in indices if index < 5]
            high_indices = [index for index in indices if index >= 10]

            if len(low_indices) > 0 and len(high_indices) > 0:
                low_span = max(low_indices) - min(low_indices) + 1
                high_span = max(high_indices) - min(high_indices) + 1

                total_gaps = (
                    (low_span - len(low_indices)) +
                    (high_span - len(high_indices))
                )

                if total_gaps <= len(wilds):
                    return True, MeldTypes.RUN

    return False, "Invalid Run"

@staticmethod
def validate_meld(meld, ruleset):
    """
    Validate a meld by delegating to the appropriate method.

    Args:
        meld: The meld to validate.
        ruleset: Active ruleset.

    Returns:
        tuple: Validation result.
    """
    if meld.meld_type == MeldTypes.SET:
        return Validator.validate_set(meld, ruleset)

    elif meld.meld_type == MeldTypes.RUN:
        return Validator.validate_run(meld, ruleset)
```

```
        else:
            return False, "Invalid Meld Type"

    @staticmethod
    def validate_play_melds(stored_melds, has_melded, ruleset, required_score):
        """
        Validate whether a player can play their stored melds.

        Ensures:
        - At least one meld exists
        - Initial meld meets minimum score requirement if not yet melded

        Args:
            stored_melds (list): Melds to validate.
            has_melded (bool): Whether the player has already melded.
            ruleset: Active ruleset.
            required_score (int): Minimum score required.

        Returns:
            tuple: Validation result.
        """
        if len(stored_melds) == 0:
            return False, "No Melds Stored"

        if not has_melded:
            total_score = 0
            for meld in stored_melds:
                total_score += meld.return_meld_value(ruleset)

            if total_score < required_score:
                return False, f"Minimum Score {ruleset.min_initial_meld_score}. Current Score
{total_score}"

        return True, stored_melds

    @staticmethod
    def validate_lay_off(card, meld, ruleset):
        """
        Validate whether a card can be laid off onto an existing meld.

        Args:
            card: Card to add.
            meld: Existing meld.
            ruleset: Active ruleset.

        Returns:
            tuple: Validation result.
        """
        test_cards = meld.cards + [card]

        valid, error_or_meld_type = Validator.validate_meld(test_cards, ruleset)

        if valid:
            return True, None
```

```
        return False, error_or_meld_type

    @staticmethod
    def validate_discard(card, cards, has_drawn, drawn_from_discard, ruleset):
        """
        Validate whether a discard action is legal.

        Args:
            card: Card to discard.
            cards (list): Player's hand.
            has_drawn (bool): Whether player has drawn this turn.
            drawn_from_discard: Card drawn from discard pile.
            ruleset: Active ruleset.

        Returns:
            tuple: Validation result.
        """
        if card not in cards:
            return False, "Card Not In Hand"

        if not has_drawn:
            return False, "Not Drawn a Card"

        if ruleset.prevent_discard_same_card and drawn_from_discard == card:
            return False, "Can't Discard Card Immediately Drawn From Discard Pile"

        return True, None

    @staticmethod
    def validate_draw(deck, has_drawn):
        """
        Validate drawing from the deck.

        Args:
            deck: The deck object.
            has_drawn (bool): Whether player has already drawn.

        Returns:
            tuple: Validation result.
        """
        if deck.is_empty():
            return False, "Deck is Empty"
        if has_drawn:
            return False, "Player has Drawn"

        return True, None

    @staticmethod
    def validate_draw_discard(card, discard, has_drawn, has_melded, ruleset):
        """
        Validate drawing a card from the discard pile.

        Args:
            card: Card being drawn.
            discard: Discard pile object.
```

```
        has_drawn (bool): Whether player has already drawn.
        has_melded (bool): Whether player has already melded.
        ruleset: Active ruleset.

    Returns:
        tuple: Validation result.
    """
    if discard.is_empty():
        return False, "Discard Pile is Empty"

    if has_drawn:
        return False, "Player has Drawn"

    if card not in discard.cards:
        return False, "Card isn't in Discard Pile"

    if ruleset.require_melding_to_draw_from_disc and not has_melded:
        return False, "Melding Required to Draw From Discard Pile"

    return True, None
```

## GameState Class:

The GameState class manages the full state of a game, including players, turns, deck, discard pile, moves, scoring, and win conditions. This supports validation and client-server synchronisation.

```
from game_logic.card import Card
from game_logic.deck import Deck
from game_logic.ruleset import Ruleset
from game_logic.discard_pile import Discard_pile
from game_logic.validator import Validator
from game_logic.utils import Moves, quicksort

from enum import Enum

class GameStatus(Enum):
    """Represents the current stage of the game."""

    LOBBY = "Lobby"
    IN_PROGRESS = "In Progress"
    GAME_OVER = "Game Over"
    PAUSED = "Paused"

class GameState:
    """
    Stores and manages the full state of a Rummy game.

    This class is responsible for:
    - tracking players
    - storing the deck and discard pile
    - managing turn order
    - applying player moves
    - checking win conditions
    - controlling game flow from lobby to game over
```

```
    """

    def __init__(self, player, ruleset=None, seed=None):
        """
        Create a new game state.

        Args:
            player: The player who creates the lobby.
            ruleset (Ruleset, optional): The ruleset to use for the game.
                If none is provided, a default Ruleset is created.
            seed (optional): Seed used for deterministic deck shuffling.
        """
        self.ruleset = ruleset or Ruleset()
        self.game_state = GameStatus.LOBBY

        # List of player objects currently in the game.
        # Initially, this only contains the player who created the lobby.
        self.players = [player]

        # Core game components. These are created when the game starts.
        self.deck = None
        self.discard_pile = None
        self.table_melds = []
        self.winner = None

        # Stores the score a player must reach for their initial meld.
        self.current_required_meld_score = self.ruleset.min_initial_meld_score

        # Optional random seed for reproducible shuffling.
        self.seed = seed or None

        # Tracks whose turn it currently is.
        self.current_player_index = 0

    def update_ruleset(self, new_ruleset):
        """
        Update the ruleset before the game starts.

        Args:
            new_ruleset (Ruleset): The new ruleset to apply.

        Returns:
            tuple[bool, str]: Success status and message.
        """
        if self.game_state != GameStatus.LOBBY:
            return False, "Can't Change Rules After Game Began"

        self.ruleset = new_ruleset
        return True, "Rules Updated"

    def add_player(self, new_player):
        """
        Add a new player to the game lobby.

        Players can only be added while the game is still in the lobby.
```

```
        Args:
            new_player: The player object to add.

        Returns:
            str | None: Error message if the player cannot be added.
        """
        if self.game_state == GameStatus.LOBBY:
            if len(self.players) < self.ruleset.max_players:
                self.players.append(new_player)
            else:
                return "Game Has Begun"

    def ready(self, player):
        """
        Mark a player as ready.

        Args:
            player: The player to mark as ready.
        """
        player.ready = True

    def start_game(self, ruleset):
        """
        Start the game once all players are ready.

        This method:
        - checks that the game has not already started
        - checks that all players are ready
        - loads the chosen ruleset
        - creates and shuffles the deck
        - deals cards to all players
        - places the first card in the discard pile

        Args:
            ruleset (dict): Ruleset data to convert into a Ruleset object.

        Returns:
            tuple[bool, str]: Success status and message.
        """
        if self.game_state != GameStatus.LOBBY:
            return False, "Game Already Started"

        for player in self.players:
            if not player.ready:
                return False, "Not All Players Ready"

        # Convert dictionary data into a Ruleset object.
        new_ruleset = Ruleset.from_dict(ruleset)
        self.update_ruleset(new_ruleset)

        self.game_state = GameStatus.IN_PROGRESS

        # Create and shuffle the deck.
        self.deck = Deck(self.ruleset, self.seed)
```

```
        self.deck.shuffle()

        # Create an empty discard pile.
        self.discard_pile = Discard_pile()

        # Deal the starting hand to each player.
        for player in self.players:
            for _ in range(self.ruleset.initial_hand_size):
                player.hand.add_card(self.deck.draw())

        # Place the first card from the deck into the discard pile.
        self.discard_pile.add_card(self.deck.draw())

        return True, "Game Started"

    def pause_game(self):
        """Pause the game."""
        self.game_state = GameStatus.PAUSED

    def return_current_player(self):
        """
        Return the player whose turn it currently is.

        Returns:
            player: The current player object.
        """
        return self.players[self.current_player_index]

    def next_turn(self):
        """
        Advance to the next player's turn.

        Uses modulus so that turn order wraps back to the first player
        after the final player has taken their turn.
        """
        self.current_player_index = (self.current_player_index + 1) % len(self.players)

    def draw_from_deck(self, player):
        """
        Allow a player to draw a card from the deck.

        Args:
            player: The player attempting to draw.

        Returns:
            tuple[bool, str | None]: Success status and optional error message.
        """
        valid, error = Validator.validate_draw(
            self.deck.peek(),
            self.deck,
            player.has_drawn
        )

        if valid:
            player.add_card(self.deck.draw())
```

```
            player.has_drawn = True
            return True, None
        else:
            return valid, error

    def draw_from_discard_pile(self, player):
        """
        Allow a player to draw the top card from the discard pile.

        Args:
            player: The player attempting to draw.

        Returns:
            tuple[bool, str | None]: Success status and optional error message.
        """
        valid, error = Validator.validate_draw_discard(
            self.discard_pile.peek(),
            self.discard_pile,
            player.has_drawn,
            player.has_melded,
            player.ruleset
        )

        if valid:
            card = self.discard_pile.draw_top_card()
            player.add_card(card)
            player.has_drawn = True
            player.drawn_from_discard_pile = card
            return valid, None
        else:
            return valid, error

    def play_stored_melds(self, player):
        """
        Play all melds currently stored by a player onto the table.

        This method validates the stored melds first. If valid, the cards
        are removed from the player's hand, added to the table melds, and
        the player's required meld score is updated if needed.

        Args:
            player: The player attempting to play melds.

        Returns:
            tuple[bool, str | list]: Success status and either an error
            message or validation result.
        """
        success, error_or_stored_melds = Validator.validate_play_melds(
            player.current_stored_melds,
            player.has_melded,
            self.ruleset,
            self.current_required_meld_score
        )

        if not success:
```

```
            return success, error_or_stored_melds

        # Remove melded cards from the player's hand.
        for meld in player.stored_melds:
            for card in meld.cards:
                player.hand.remove_card(card)

        # Add the validated melds to the shared table area.
        self.table_melds += player.current_stored_melds

        # Update the required meld score for future turns if needed.
        self.update_required_meld_score(
            player.return_stored_meld_score(self.ruleset)
        )

        # Move current stored melds into completed meld history.
        player.completed_stored_melds += player.current_stored_melds

        # Clear the player's temporary meld storage.
        player.reset_current_stored_melds()

        return success, error_or_stored_melds

    def discard(self, player, card):
        """
        Discard a card from the player's hand to the discard pile.

        If the discard is valid, the card is removed from the player's hand,
        added to the discard pile, and the turn passes to the next player.

        Args:
            player: The player discarding the card.
            card (Card): The card to discard.

        Returns:
            tuple[bool, str | None]: Success status and optional error message.
        """
        valid, error = Validator.validate_discard(
            card,
            player.hand.cards,
            self.discard_pile,
            player.has_drawn,
            player.drawn_from_discard,
            self.ruleset
        )

        if valid:
            player.hand.remove_card(card)
            self.discard_pile.add_card(card)
            self.next_turn()
            return True, None
        else:
            return False, error

    def lay_off(self, player, card, meld):
```

```
        """
        Add a card from a player's hand onto an existing meld on the table.

        Args:
            player: The player laying off the card.
            card (Card): The card to add.
            meld: The table meld to add the card to.

        Returns:
            tuple[bool, str | None]: Success status and optional error message.
        """
        valid, error = Validator.validate_lay_off(card, meld, self.ruleset)

        if valid:
            player.hand.remove_card(card)
            meld.add_card(card)
            return True, None
        else:
            return False, error

    def update_required_meld_score(self, score):
        """
        Update the current required initial meld score.

        Only increases the score if the ruleset allows incremental meld scores.

        Args:
            score (int): The score to compare against the current requirement.
        """
        if self.ruleset.initial_meld_increment:
            if self.current_required_meld_score < score:
                self.current_required_meld_score = score

    def game_end(self, winning_player):
        """
        Calculate end-of-game scores and placements.

        All non-winning players lose points based on their deadwood score.
        The winning player gains the winner bonus defined by the ruleset.
        Players are then sorted into placements.

        Args:
            winning_player: The player who won the game.
        """
        deadwoods = []

        for player in self.players:
            if player != winning_player:
                deadwood = player.hand.calculate_deadwood(self.ruleset)
                player.add_to_score(self.ruleset, -deadwood)
                deadwoods.append((player, deadwood))

        winning_player.add_to_score(self.ruleset, self.ruleset.winner_deadwood)
        deadwoods.append((winning_player, self.ruleset.winner_deadwood))
```

```
        # Sort players differently depending on the scoring system.
        if self.ruleset.scoring_method == 'negative':
            deadwoods = quicksort(deadwoods, key=lambda x: x[1])
        else:
            deadwoods = quicksort(deadwoods, key=lambda x: x[1], reverse=True)

        placement_results = {}

        # Assign placement numbers starting from 1st place.
        for place, (player, score) in enumerate(deadwoods):
            placement_results[place + 1] = (player, score)

        return placement_results

    def check_win_condition(self, player):
        """
        Check whether a player has won the game.

        A player wins when their hand is empty.

        Args:
            player: The player to check.

        Returns:
            player | None: The winning player if they have won, otherwise None.
        """
        if player.hand.is_empty():
            self.game_state = GameStatus.GAME_OVER
            self.winner = player
            return self.winner
        else:
            return None

    def return_game_state(self):
        """
        Return the current game status.

        Returns:
            GameStatus: The current state of the game.
        """
        return self.game_state

    def return_winner(self):
        """
        Return the winning player.

        Returns:
            player | None: The winner if the game is over, otherwise None.
        """
        return self.winner

    def get_player_by_id(self, user_id):
        """
        Find a player in the game using their ID.
```

```
        Args:
            user_id: The ID of the player to find.

        Returns:
            player | None: The matching player, or None if not found.
        """
        for player in self.players:
            if player.user_id == user_id:
                return player
        return None

    def _select_card(self, player, card):
        """
        Mark a card as selected for a player.

        Args:
            player: The player selecting the card.
            card (Card): The card to select.
        """
        player.select_card(card)

    def deselect_all(self, player):
        """
        Deselect all currently selected cards for a player.

        Args:
            player: The player whose cards should be deselected.
        """
        player.deselect_all()

    def _deselect_card(self, player, card):
        """
        Deselect a specific card for a player.

        Args:
            player: The player deselecting the card.
            card (Card): The card to deselect.
        """
        player.deselect_card(card)

    def _get_card_from_dict(self, card_dict):
        """
        Convert a dictionary representation of a card back into a Card object.

        Args:
            card_dict (dict): Dictionary form of a card.

        Returns:
            Card: The reconstructed card object.
        """
        return Card.from_dict(card_dict, self.ruleset)

    def apply_move(self, move):
        """
        Apply a move sent by a player.
```

```
        This acts as the main dispatcher for all move types. It reads the move
        data, identifies the player, and calls the relevant method.

        Args:
            move (dict): A dictionary containing move information, including:
                - move_type
                - user_id
                - any extra data needed for that move

        Returns:
            tuple[bool, str | None]: Success status and optional error message.
        """
        move_type = Moves[move["move_type"]]
        player = self.get_player_by_id(move["user_id"])

        if move_type == Moves.Draw_Deck:
            return self.draw_from_deck(player)

        elif move_type == Moves.Draw_Discard:
            return self.draw_from_discard_pile(player)

        elif move_type == Moves.Discard:
            card = self._get_card_from_dcit(move["card"])
            valid, error = self.discard(player, card)
            if not valid:
                return False, error
            self.check_win_condition(player)
            return True, None

        elif move_type == Moves.Store_Meld:
            return player.store_meld(self.ruleset)

        elif move_type == Moves.Meld:
            return self.play_stored_melds(player)

        elif move_type == Moves.Lay_Off:
            card = self._get_card_from_dcit(move["card"])
            meld = self.table_melds[move["meld_index"]]
            return self.lay_off(player, card, meld)

        elif move_type == Moves.Deck_Shuffle:
            self.deck.shuffle()
            return True, None

        elif move_type == Moves.Store_Card:
            card = self._get_card_from_dict(move["card"])
            self._select_card(player, card)
            return True, None

        elif move_type == Moves.Deselect_all:
            self.deselect_all(player)

        elif move_type == Moves.Sort_Rank:
            player.sort_rank()
```

```
            return True, None

        elif move_type == Moves.Sort_Suit:
            player.sort_suit()
            return True, None

        else:
            return False, "Unknown Move"
```

## GameService class:

The GameService class acts as the bridge between frontend, game logic, and the database. This supports the client-server architecture and persistent multiplayer.

```
import uuid
from game_logic.game_state import GameState, GameStatus
from game_logic.player import Player
from game_logic.hand import Hand

class GameService:
    """
    Handles the main game service logic for the application.

    This class manages:
    - active games currently running in memory
    - active players currently logged in
    - communication between the game logic and the database service
    - communication between the game logic and the frontend
    """

    def __init__(self, db_service):
        """
        Create a new GameService object.

        Args:
            db_service: The database service used to store and retrieve data.
        """
        self.active_games = {}
        self.active_players = {}
        self.db = db_service

    def create_game(self, user_id, ruleset, seed=None):
        """
        Create a new game for a player.

        The method first checks that the player is active and not already in
        another game. It then creates the game in the database and stores the
        game in memory.

        Args:
            user_id: The ID of the user creating the game.
            ruleset: The ruleset that will be used for the game.
            seed: Optional seed used for predictable deck generation.
```

```
        Returns:
            tuple: (game_id, None) if successful, otherwise (False, error_message).
        """
        success, player = self.get_active_player(user_id)
        if not success:
            return False, player

        in_game, game_id = self.get_player_current_game(user_id)
        if in_game:
            return False, "Player In Game"

        success, game_id = self.db.create_game(ruleset, seed)
        if not success:
            return False, "Could Not Create Game"

        game = GameState(player, ruleset, seed)
        self.active_games[game_id] = game
        return game_id, None

    def add_player(self, game_id, user_id):
        """
        Add an active player to a game lobby.

        A player can only be added if the game is still in the lobby state.

        Args:
            game_id: The ID of the game to join.
            user_id: The ID of the player joining the game.

        Returns:
            tuple: (True, game_id) if successful, otherwise (False, error_message).
        """
        success, player = self.get_active_player(user_id)
        if not success:
            return False, player

        game = self.active_games[game_id]
        if game.game_state != GameStatus.LOBBY:
            return False, "Game Not Available"

        game.add_player(player)
        return True, game_id

    def ready(self, user_id, game_id):
        """
        Mark a player as ready in a lobby.

        The player must exist in the lobby and the game must still be available.

        Args:
            user_id: The ID of the player marking themselves as ready.
            game_id: The ID of the game lobby.

        Returns:
```

```
        tuple: (True, game_id) if successful, otherwise (False, error_message).
    """
    success, player = self.get_active_player(user_id)
    if not success:
        return False, player

    success, players = self.get_lobby_players(game_id)
    for p in players:
        if p.get("user_id") == user_id:
            game = self.active_games[game_id]
            if game.game_state != GameStatus.LOBBY:
                return False, "Game Not Available"

            game.ready(player)
            return True, game_id

    return False, "Player Not in Lobby"

  def get_lobby_players(self, game_id):
    """
    Get a list of all players currently in a lobby.

    Args:
        game_id: The ID of the game lobby.

    Returns:
        tuple: (True, players_list) if successful, otherwise (False, error_message).
    """
    game = self.active_games.get(game_id)
    if game is None:
        return False, "Game Not Found"

    players = []
    for p in game.players:
        players.append({
            "user_id": p.user_id,
            "username": p.username,
            "ready_status": p.ready
        })

    return True, players

  def start_game(self, game_id, ruleset):
    """
    Start a game from the lobby.

    This updates the game state, stores the game as started in the database,
    and records each player as part of the game.

    Args:
        game_id: The ID of the game to start.
        ruleset: The ruleset to apply.

    Returns:
        tuple: (True, None) if successful, otherwise (False, error_message).
```

```
        """
        game = self.active_games[game_id]
        success, error = game.start_game(ruleset)
        if not success:
            return False, error

        self.db.start_game(game.ruleset, game.deck.seed)

        for player in game.players:
            self.db.add_player_to_game(player.user_id, game_id, "Player")

        return True, None

    def pause_game(self, game_id):
        """
        Pause a currently active game.

        The game is saved through the database and then removed from active memory.

        Args:
            game_id: The ID of the game to pause.

        Returns:
            tuple: (True, None) if successful, otherwise (False, error_message).
        """
        game = self.active_games[game_id]
        if game is None:
            return False, "Game Not Found"

        self.db.pause_game(game_id)
        game.pause_game()
        del self.active_games[game_id]
        return True, None

    def load_paused_game(self, game_id, user_id):
        """
        Load a paused game back into active memory.

        The method checks that:
        - the user is active
        - the game exists and is paused
        - the user belongs to that game

        Args:
            game_id: The ID of the paused game.
            user_id: The ID of the player loading the game.

        Returns:
            tuple: (True, "Game Loaded") if successful, otherwise (False, error_message).
        """
        success, player = self.get_active_player(user_id)
        if not success:
            return False, player

        row, error = self.db.get_game_status(game_id)
```

```
        if row is None:
            return False, error

        if row["status"] != "Paused":
            return False, "Game is not paused"

        history, error = self.db.get_player_history(player.user_id)
        if not history:
            return False, error

        if not any(row["game_id"] == game_id for row in history):
            return False, "Player Does Not Belong to This Game"

        ruleset, _ = self.db.get_ruleset(game_id)
        seed, _ = self.db.get_seed(game_id)

        game = GameState(player, ruleset, seed)
        self.active_games[game_id] = game
        return True, "Game Loaded"

    def rejoin_game(self, game_id, user_id):
        """
        Allow a player to rejoin a game they previously belonged to.

        Args:
            game_id: The ID of the game to rejoin.
            user_id: The ID of the player rejoining.

        Returns:
            tuple: (True, None) if successful, otherwise (False, error_message).
        """
        success, player = self.get_active_player(user_id)
        if not success:
            return False, player

        history, error = self.db.get_player_history(player.user_id)
        if not history:
            return False, error

        game_ids = [row["game_id"] for row in history]
        if game_id not in game_ids:
            return False, "Player Doesn't Belong to This Game"

        self.add_player(game_id, user_id)
        return True, None

    def resume_game(self, game_id):
        """
        Resume a paused game by replaying saved moves.

        Args:
            game_id: The ID of the game to resume.

        Returns:
            tuple: (True, "Game Resumed") if successful, otherwise (False, error_message).
```

```
        """
        game = self.active_games[game_id]
        moves, error = self.db.get_moves(game_id)

        if not moves:
            return False, error

        # Reapply each saved move so the game returns to its previous state.
        for move in moves:
            game.apply_move(move)

        return True, "Game Resumed"

    def apply_move(self, game_id, move):
        """
        Apply a player's move to the game.

        If the move is valid, it is stored in the database. If the move causes
        the game to finish, the game is ended automatically.

        Args:
            game_id: The ID of the game being played.
            move: A dictionary containing move information.

        Returns:
            tuple: (success, error_message_or_None)
        """
        game = self.active_games[game_id]
        success, error = game.apply_move(move)

        if success:
            self.db.add_move(
                game_id,
                move["user_id"],
                move["move_type"],
                move.get("card"),
                move.get("cards"),
                move.get("meld_index")
            )

            # If the game is over after this move, save the final results.
            if game.return_game_state() == GameStatus.GAME_OVER:
                self.end_game(game_id)

        return success, error

    def end_game(self, game_id):
        """
        End a game and store the final results.

        Each player's final placing and score is recorded in the database before
        the game is removed from active memory.

        Args:
            game_id: The ID of the game to end.
```

```
        Returns:
            tuple: (True, results) if successful, otherwise (False, error_message).
        """
        game = self.active_games[game_id]
        if game is None:
            return False, "Game Not Found"

        results = game.game_end(game.winner)

        for place, (player, score) in results.items():
            self.db.record_game_result(player.user_id, game_id, place, score)

        del self.active_games[game_id]
        return True, results

    def get_game_state(self, game_id, user_id):
        """
        Build and return the full visible game state for a player.

        This includes information such as:
        - current turn
        - top discard card
        - deck size
        - player's hand
        - table melds
        - scores
        - winner if the game has ended

        Args:
            game_id: The ID of the game.
            user_id: The ID of the player requesting the game state.

        Returns:
            dict or tuple: Game state dictionary if successful, otherwise
            (False, error_message).
        """
        player, error = self.get_active_player(user_id)
        if player is None:
            return False, error

        game = self.active_games[game_id]

        game_status = game.game_state.value
        current_player = game.return_current_player().user_id
        discard_top = game.discard_pile.peek().to_dict()
        deck_size = len(game.deck.cards)
        has_drawn = player.has_drawn
        has_melded = player.has_melded
        required_meld_score = game.current_required_meld_score
        winner = game.winner.user_id if game.winner else None

        # Convert the player's hand into a list of dictionaries for easy use by the UI.
        hand = []
        for card in player.hand.cards:
```

```
            hand.append(card.to_dict())

        # Convert all melds on the table into nested lists of card dictionaries.
        table_melds = []
        for meld in game.table_melds:
            meld_cards = []
            for card in meld.cards:
                meld_cards.append(card.to_dict())
            table_melds.append(meld_cards)

        # Store summary information for each player in the game.
        players = []
        for p in game.players:
            players.append({
                "user_id": p.user_id,
                "username": p.username,
                "hand_size": len(p.hand.cards),
                "score": p.score
            })

        # Store the cards currently selected by the player.
        selected_cards = []
        for card in player.get_stored_cards():
            selected_cards.append(card.to_dict())

        state = {
            "game_status": game_status,
            "current_player": current_player,
            "discard_top": discard_top,
            "deck_size": deck_size,
            "has_drawn": has_drawn,
            "has_melded": has_melded,
            "required_meld_score": required_meld_score,
            "hand": hand,
            "selected_cards": selected_cards,
            "table_melds": table_melds,
            "players": players,
            "winner": winner
        }

        return state

    def get_active_player(self, user_id):
        """
        Find an active player currently stored in memory.

        Args:
            user_id: The ID of the user to find.

        Returns:
            tuple: (True, player) if found, otherwise (False, error_message).
        """
        player = self.active_players.get(user_id)
        if player is None:
            return False, "Player Not Found"
```

```
        return True, player

    def create_guest(self):
        """
        Create a guest account and log that guest into active memory.

        A random guest username is generated automatically.

        Returns:
            tuple: (True, user_data) if successful, otherwise (False, error_message).
        """
        username = f"Guest_{uuid.uuid4().hex[:8]}"
        success, user = self.db.sign_up(username, None, None, True)
        if not success:
            return False, "Could Not Create Guest"

        hand = Hand()
        player = Player(user["user_id"], username, hand)
        self.active_players[user["user_id"]] = player
        return True, user

    def get_lobbies(self):
        """
        Get all available game lobbies from the database.

        Returns:
            The result returned by the database service.
        """
        return self.db.get_lobbies()

    def get_paused_game(self, user_id):
        """
        Get paused games associated with a user.

        Args:
            user_id: The ID of the user.

        Returns:
            The result returned by the database service.
        """
        return self.db.get_paused_games_by_user(user_id)

    def sign_up(self, username, password, email):
        """
        Create a new registered user account.

        Args:
            username: The user's chosen username.
            password: The user's chosen password.
            email: The user's email address.

        Returns:
            The result returned by the database service.
        """
        return self.db.sign_up(username, password, email)
```

```
    def log_in(self, username, password):
        """
        Log a user in and create their active Player object.

        Args:
            username: The user's username.
            password: The user's password.

        Returns:
            tuple: (True, user_data) if successful, otherwise (False, error_message).
        """
        user, error = self.db.log_in(username, password)
        if user:
            hand = Hand()
            player = Player(user["user_id"], username, hand)
            self.active_players[user["user_id"]] = player
            return True, user
        return False, error

    def log_out(self, user_id, is_guest):
        """
        Log a user out of the system.

        If the user is a guest, their account is deleted as part of logout.

        Args:
            user_id: The ID of the user logging out.
            is_guest: Boolean value showing whether the user is a guest.

        Returns:
            tuple: (True, "Logged Out") if successful, otherwise (False, error_message).
        """
        user, _ = self.db.get_user_by_id(user_id)

        if user is None:
            return False, "No User To Log Out"

        if is_guest:
            self.delete_account(user_id)

        if user_id in self.active_players:
            del self.active_players[user_id]

        return True, "Logged Out"

    def delete_account(self, user_id):
        """
        Delete a user account.

        Args:
            user_id: The ID of the user account to delete.

        Returns:
            The result returned by the database service.
```

```
        """
        return self.db.delete_user(user_id)

    def change_password(self, user_id, old_password, new_password):
        """
        Change a user's password.

        Args:
            user_id: The ID of the user.
            old_password: The user's current password.
            new_password: The new password to set.

        Returns:
            The result returned by the database service.
        """
        return self.db.change_password(user_id, old_password, new_password)

    def change_username(self, user_id, new_username):
        """
        Change a user's username.

        Args:
            user_id: The ID of the user.
            new_username: The new username.

        Returns:
            The result returned by the database service.
        """
        return self.db.change_username(user_id, new_username)

    def send_friend_request(self, user_id, friend_id):
        """
        Send a friend request from one user to another.

        Args:
            user_id: The sender's user ID.
            friend_id: The recipient's user ID.

        Returns:
            The result returned by the database service.
        """
        return self.db.send_friend_request(user_id, friend_id)

    def accept_friend_request(self, user_id, friend_id):
        """
        Accept a friend request.

        Args:
            user_id: The user accepting the request.
            friend_id: The user who sent the request.

        Returns:
            The result returned by the database service.
        """
        return self.db.accept_friend_request(user_id, friend_id)
```

```
    def reject_friend_request(self, user_id, friend_id):
        """
        Reject a friend request.

        Args:
            user_id: The user rejecting the request.
            friend_id: The user who sent the request.

        Returns:
            The result returned by the database service.
        """
        return self.db.reject_friend_request(user_id, friend_id)

    def get_friends(self, user_id):
        """
        Get a user's list of friends.

        Args:
            user_id: The ID of the user.

        Returns:
            The result returned by the database service.
        """
        return self.db.get_friends(user_id)

    def get_pending_requests(self, user_id):
        """
        Get all pending friend requests for a user.

        Args:
            user_id: The ID of the user.

        Returns:
            The result returned by the database service.
        """
        return self.db.get_pending_requests(user_id)

    def get_all_users_except(self, user_id):
        """
        Get all users except the specified user.

        Args:
            user_id: The ID of the user to exclude.

        Returns:
            The result returned by the database service.
        """
        return self.db.get_all_users_except(user_id)

    def get_player_history(self, user_id):
        """
        Get the full game history of a player.

        Args:
```

```
        user_id: The ID of the user.

    Returns:
        The result returned by the database service.
    """
    return self.db.get_player_history(user_id)

def get_player_current_game(self, user_id):
    """
    Check whether a player is already in an active game.

    Args:
        user_id: The ID of the player.

    Returns:
        tuple: (True, game_id) if found, otherwise (False, error_message).
    """
    for game_id, game in self.active_games.items():
        if any(p.user_id == user_id for p in game.players):
            return True, game_id
    return False, "Could Not Find Game"

def get_user_guest_status(self, user_id):
    """
    Get whether a user account is a guest account.

    Args:
        user_id: The ID of the user.

    Returns:
        The result returned by the database service.
    """
    return self.db.get_user_guest_status(user_id)
```

# Database Layer:

## Hashing:

The hashing algorithms include password hashing and sating, supporting the authentication objective by protecting the stored passwords.

```
import os

def simple_hash(input_str, salt):
    """
    Generate a custom hash value from an input string and salt.

    The function combines the input text with the salt, then repeatedly
    applies bitwise and arithmetic operations to produce a fixed hash value.

    Args:
        input_str (str): The string to hash.
        salt (str): The salt added to the input before hashing.
```

```
    Returns:
        int: The resulting hash value as an integer.
    """
    hash_vector = 2166136261

    combined = input_str + salt

    for char in combined:
        # Mix each character into the hash using XOR and bit shifting.
        hash_vector ^= ord(char)
        hash_vector *= 53383619654664048229
        hash_vector &= 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
        hash_vector ^= (hash_vector >> 13)
        hash_vector ^= (hash_vector << 7) & 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF

    return hash_vector

def hash_password(password, salt=None):
    """
    Hash a password using a salt.

    If a salt is supplied, it is reused so the same password can be checked
    against a stored hash. If no salt is supplied, a new random salt is created.

    The password is hashed repeatedly to make the final value harder to reverse.

    Args:
        password (str): The password to hash.
        salt (str, optional): An existing salt. If omitted, a new salt is generated.

    Returns:
        tuple: The salt used and the final hash as a hexadecimal string.
    """
    if salt is not None:
        hash_value = simple_hash(password, salt)
    else:
        salt = os.urandom(8).hex()
        hash_value = simple_hash(password, salt)

    # Repeat the hashing process multiple times to increase security.
    for _ in range(1000):
        hash_value = simple_hash(str(hash_value), salt)

    return salt, format(hash_value, "032x")
```

## DBConnections Class:

The DBConnection class manages the MySQL connection and query execution, supporting the persistent storage of accounts, games, moves, and history.

```
import mysql.connector
```

```
class DBConnection:
    """
    Handles the connection to the MySQL database.

    This class is responsible for:
    - establishing a connection
    - executing SQL queries
    - committing transactions
    - closing the connection safely
    """

    def __init__(self):
        """
        Initialise a connection to the database and create a cursor.

        The cursor uses dictionary=True so query results are returned
        as dictionaries instead of tuples.
        """
        self.connection = mysql.connector.connect(
            host='127.0.0.1',
            user='root',
            password='pass123',
            database='house_rummy'
        )

        self.cursor = self.connection.cursor(dictionary=True)

    def execute(self, query, params=None):
        """
        Execute a SQL query.

        Args:
            query (str): The SQL query to run.
            params (tuple, optional): Parameters to safely insert into the query.

        Returns:
            cursor: The cursor after execution (used to fetch results).
        """
        self.cursor.execute(query, params or ())
        return self.cursor

    def commit(self):
        """
        Commit the current transaction to the database.

        This must be called after INSERT, UPDATE, or DELETE queries
        to save changes permanently.

        """
        self.connection.commit()

    def close(self):
        """
        Close the database cursor and connection.
```

```
        Should be called when the database is no longer needed
        to free resources.

        """
        self.cursor.close()
        self.connection.close()
```

## UserRepository Class:

The UserRepository manages user account data, supporting account creation, login, password changes, username changes, and user lookup.

```
from SQLConnections.Hashing import hash_password


class UserRepository:
    """
    Handles all database operations related to users.

    This repository is responsible for:
    - creating user accounts
    - retrieving user records
    - updating usernames and passwords
    - deleting users
    - checking login credentials
    - retrieving guest account status
    """

    @staticmethod
    def create_user(db, username, password, email, guest=False):
        """
        Create a new user account.

        The method first checks whether the username already exists.
        If not, the password is hashed and the new user is inserted
        into the Users table.

        Args:
            db: The database connection object.
            username (str): The username for the new account.
            password (str): The user's plain text password.
            email (str): The user's email address.
            guest (bool): Whether the account is a guest account.

        Returns:
            tuple: (True, user) if successful, otherwise (False, error_message).
        """
        if UserRepository._username_exists(db, username):
            return False, "Username Exists"

        salt, hashed_password = hash_password(password)

        db.execute(
```

```
        "INSERT INTO Users(username, password, email, salt, guest) VALUES (%s, %s, %s,
%s, %s)",
        (username, hashed_password, email, salt, guest)
    )
    db.commit()

    user, _ = UserRepository.get_user_by_username(db, username)
    return True, user

  @staticmethod
  def _username_exists(db, username):
    """
    Check whether a username is already stored in the database.

    Args:
        db: The database connection object.
        username (str): The username to check.

    Returns:
        bool: True if the username exists, otherwise False.
    """
    result = db.execute(
        "SELECT 1 FROM Users WHERE username = %s",
        (username,)
    )

    return result.fetchone() is not None

  @staticmethod
  def get_user_by_username(db, username):
    """
    Retrieve a user record using a username.

    Args:
        db: The database connection object.
        username (str): The username to search for.

    Returns:
        tuple: (row, None) if found, otherwise (None, error_message).
    """
    result = db.execute(
        "SELECT * FROM Users WHERE username = %s",
        (username,)
    )

    row = result.fetchone()
    if row is None:
        return None, "User Not Found"
    return row, None

  @staticmethod
  def get_user_by_id(db, user_id):
    """
    Retrieve a user record using a user ID.
```

```
        Args:
            db: The database connection object.
            user_id (int): The ID of the user.

        Returns:
            tuple: (row, None) if found, otherwise (None, error_message).
        """
        result = db.execute(
            "SELECT * FROM Users WHERE user_id = %s",
            (user_id,)
        )

        row = result.fetchone()
        if row is None:
            return None, "User Not Found"
        return row, None

    @staticmethod
    def change_password(db, user_id, password, new_password):
        """
        Change a user's password.

        The current password must be verified before the new password
        is hashed and stored.

        Args:
            db: The database connection object.
            user_id (int): The ID of the user.
            password (str): The user's current password.
            new_password (str): The new password to save.

        Returns:
            tuple: (True, None) if successful, otherwise (False, error_message).
        """
        result, error = UserRepository.verify_password(db, password, user_id)

        if result:
            salt, hashed_password = hash_password(new_password)
            db.execute(
                "UPDATE Users SET password = %s, salt = %s WHERE user_id = %s",
                (hashed_password, salt, user_id)
            )
            db.commit()
            return True, None

        return False, error

    @staticmethod
    def change_username(db, user_id, new_username):
        """
        Change a user's username.

        Args:
            db: The database connection object.
            user_id (int): The ID of the user.
```

```
        new_username (str): The new username to save.

    Returns:
        tuple: (True, None) after the update is completed.
    """
    db.execute(
        "UPDATE Users SET username = %s WHERE user_id = %s",
        (new_username, user_id)
    )
    db.commit()

    return True, None

@staticmethod
def delete_user(db, user_id):
    """
    Delete a user account from the database.

    Args:
        db: The database connection object.
        user_id (int): The ID of the user to delete.

    Returns:
        tuple: (True, None) after the user is deleted.
    """
    db.execute(
        "DELETE FROM Users WHERE user_id = %s",
        (user_id,)
    )
    db.commit()

    return True, None

@staticmethod
def verify_password(db, password, user_id):
    """
    Check whether a supplied password matches the stored password.

    The stored salt is retrieved from the database and used to hash
    the entered password for comparison.

    Args:
        db: The database connection object.
        password (str): The password entered by the user.
        user_id (int): The ID of the user.

    Returns:
        tuple: (True, None) if the password is correct,
               otherwise (False, error_message).
    """
    result = db.execute(
        "SELECT password, salt FROM Users WHERE user_id = %s",
        (user_id,)
    )
```

```
        row = result.fetchone()
        if row is None:
            return False, "User Not Found"

        salt, hashed_password = hash_password(password, row["salt"])

        if hashed_password == row["password"]:
            return True, None

        return False, "Password Incorrect"

    @staticmethod
    def get_all_users_except(db, user_id):
        """
        Retrieve all users except the specified user.

        Results are returned in alphabetical order by username.

        Args:
            db: The database connection object.
            user_id (int): The ID of the user to exclude.

        Returns:
            tuple: (True, rows) if users are found, otherwise (False, error_message).
        """
        result = db.execute(
            "SELECT * FROM Users WHERE user_id != %s ORDER BY username ASC",
            (user_id,)
        )

        rows = result.fetchall()

        if rows == []:
            return False, "No Users Found"
        return True, rows

    @staticmethod
    def get_user_guest_status(db, user_id):
        """
        Retrieve whether a user account is marked as a guest account.

        Args:
            db: The database connection object.
            user_id (int): The ID of the user.

        Returns:
            tuple: (True, row) if found, otherwise (False, error_message).
        """
        result = db.execute(
            "SELECT guest FROM Users WHERE user_id = %s",
            (user_id,)
        )

        row = result.fetchone()
```

```
        if row is None:
            return False, "User Not Found"
        return True, row

    @staticmethod
    def get_username_by_user_id(db, user_id):
        """
        Retrieve a username using a user ID.

        Args:
            db: The database connection object.
            user_id (int): The ID of the user.

        Returns:
            tuple: (True, username) if found, otherwise (False, error_message).
        """
        result = db.execute(
            "SELECT username FROM Users WHERE user_id = %s",
            (user_id,)
        )

        row = result.fetchone()

        if row is None:
            return False, "User Not Found"
        return True, row["username"]
```

## GameRepository Class:

The GameRepository class stores and retrieves game records, which includes rulesets, statuses, and seeds. This supports game persistence and rule customisation.

```
import json

class GameRepository:
    """
    Handles all database operations related to games.

    All methods return:
    (True, value) on success OR (False, error_message) on failure.
    """

    @staticmethod
    def create_game(db, ruleset, status, seed):
        """
        Create a new game record.

        The ruleset object is converted into JSON before storage.
        """
        ruleset_string = json.dumps(ruleset.to_dict())

        db.execute(
            "INSERT INTO Games (ruleset, status, seed) VALUES (%s, %s, %s)",
            (ruleset_string, status, seed)
        )
```

```
        db.commit()

        game_id = db.cursor.lastrowid
        return True, game_id

    @staticmethod
    def change_status(db, game_id, status):
        """
        Update the status of a game.
        """
        db.execute(
            "UPDATE Games SET status = %s WHERE game_id = %s",
            (status, game_id)
        )
        db.commit()

        return True, None

    @staticmethod
    def get_status(db, game_id):
        """
        Retrieve the status of a game.
        """
        result = db.execute(
            "SELECT status FROM Games WHERE game_id = %s",
            (game_id,)
        )

        row = result.fetchone()
        if row is None:
            return False, "Game Not Found"

        return True, row["status"]

    @staticmethod
    def get_game(db, game_id):
        """
        Retrieve full game record.
        """
        result = db.execute(
            "SELECT * FROM Games WHERE game_id = %s",
            (game_id,)
        )

        row = result.fetchone()
        if row is None:
            return False, "Game Not Found"

        return True, row

    @staticmethod
    def get_ruleset(db, game_id):
        """
        Retrieve and decode the stored ruleset.
        """
```

```
        result = db.execute(
            "SELECT ruleset FROM Games WHERE game_id = %s",
            (game_id,)
        )

        row = result.fetchone()
        if row is None:
            return False, "Game Not Found"

        return True, json.loads(row["ruleset"])

    @staticmethod
    def delete_game(db, game_id):
        """
        Delete a game record.
        """
        db.execute(
            "DELETE FROM Games WHERE game_id = %s",
            (game_id,)
        )
        db.commit()

        return True, None

    @staticmethod
    def get_games_by_status(db, status):
        """
        Retrieve all game IDs with a given status.
        """
        result = db.execute(
            "SELECT game_id FROM Games WHERE status = %s",
            (status,)
        )

        rows = result.fetchall()
        if not rows:
            return False, "No Games With That Status Found"

        return True, rows

    @staticmethod
    def get_seed(db, game_id):
        """
        Retrieve the seed used for a game.
        """
        result = db.execute(
            "SELECT seed FROM Games WHERE game_id = %s",
            (game_id,)
        )

        row = result.fetchone()
        if row is None:
            return False, "Game Does Not Exist"

        return True, row["seed"]
```

## MoveRepository Class:

The MoveRepository class stores and retrieves game moves in order, supporting paused game recovery and replay based state restoration.

```
import json

class MoveRepository:
    """
    Handles all database operations related to moves.

    This repository is responsible for:
    - storing moves made during a game
    - retrieving moves for a game or user
    - deleting stored moves
    - counting how many moves exist in a game

    All methods return:
    (True, value) on success OR (False, error_message) on failure.
    """

    @staticmethod
    def add_move(db, game_id, user_id, move_number, move_type, card=None, cards=None,
meld_index=None):
        """
        Store a move in the database.

        A move may involve either:
        - a single card
        - multiple cards
        - no card data

        Card data is converted into JSON before being stored.

        Args:
            db: The database connection object.
            game_id (int): The ID of the game.
            user_id (int): The ID of the user making the move.
            move_number (int): The order number of the move in the game.
            move_type (str): The type of move performed.
            card: A single card object involved in the move.
            cards: A list of card objects involved in the move.
            meld_index (int, optional): The index of the meld affected by the move.

        Returns:
            tuple: (True, move_id) after the move is stored.
        """
        if cards is not None:
            card_string = json.dumps([c.to_dict() for c in cards])
        elif card is not None:
            card_string = json.dumps(card.to_dict())
        else:
            card_string = None

        db.execute(
```

```
            "INSERT INTO Moves (game_id, user_id, move_number, move_type, card, meld_index) VALUES (%s, %s, %s, %s, %s, %s)",
        (game_id, user_id, move_number, move_type, card_string, meld_index)
    )
    db.commit()

    move_id = db.cursor.lastrowid
    return True, move_id

@staticmethod
def get_moves_by_game(db, game_id):
    """
    Retrieve all moves made in a game.

    Results are ordered by move number so the game can be replayed
    in the correct sequence.

    Args:
        db: The database connection object.
        game_id (int): The ID of the game.

    Returns:
        tuple: (True, rows) if moves are found, otherwise (False, error_message).
    """
    result = db.execute(
        "SELECT * FROM Moves WHERE game_id = %s ORDER BY move_number ASC",
        (game_id,)
    )

    rows = result.fetchall()
    if not rows:
        return False, "No Moves Found For This Game"
    return True, rows

@staticmethod
def delete_all_moves_in_game(db, game_id):
    """
    Delete all moves stored for a game.

    Args:
        db: The database connection object.
        game_id (int): The ID of the game.

    Returns:
        tuple: (True, None) after deletion.
    """
    db.execute(
        "DELETE FROM Moves WHERE game_id = %s",
        (game_id,)
    )
    db.commit()

    return True, None

@staticmethod
```

```
    def get_moves_by_user(db, user_id):
        """
        Retrieve all moves made by a specific user.

        Args:
            db: The database connection object.
            user_id (int): The ID of the user.

        Returns:
            tuple: (True, rows) if moves are found, otherwise (False, error_message).
        """
        result = db.execute(
            "SELECT move_id FROM Moves WHERE user_id = %s",
            (user_id,)
        )

        rows = result.fetchall()
        if not rows:
            return False, "No Moves Found For This User"
        return True, rows

    @staticmethod
    def delete_all_moves_by_user(db, user_id):
        """
        Delete all moves made by a specific user.

        Args:
            db: The database connection object.
            user_id (int): The ID of the user.

        Returns:
            tuple: (True, None) after deletion.
        """
        db.execute(
            "DELETE FROM Moves WHERE user_id = %s",
            (user_id,)
        )
        db.commit()

        return True, None

    @staticmethod
    def get_move_count(db, game_id):
        """
        Count how many moves are stored for a game.

        Args:
            db: The database connection object.
            game_id (int): The ID of the game.

        Returns:
            tuple: (True, count) on success.
        """
        result = db.execute(
            "SELECT COUNT(*) AS count FROM Moves WHERE game_id = %s",
```

```
        (game_id,)
    )

    row = result.fetchone()
    return True, row["count"]
```

## LinkRepository Class:

The LinkRepository class manages the relationships between users, users and games, through the friends list and game history. This supports basic social features and account-based tracking.

```
class LinkRepository:
    """
    Handles all database operations related to game history and friends.

    This repository is responsible for:
    - storing game history records
    - retrieving game history by player or game
    - updating and deleting game history
    - creating friend requests
    - updating friend request statuses
    - retrieving and deleting friends

    All methods return:
    (True, value) on success OR (False, error_message) on failure.
    """

    @staticmethod
    def create_game_history(db, user_id, game_id, result, role):
        """
        Store a game history record for a user.

        Args:
            db: The database connection object.
            user_id (int): The ID of the user.
            game_id (int): The ID of the game.
            result (str): The result of the game.
            role (str): The role the user had in the game.

        Returns:
            tuple: (True, None) after the record is stored.
        """
        db.execute(
            "INSERT INTO GameHistory(user_id, game_id, result, role) VALUES (%s, %s, %s,
%s)",
            (user_id, game_id, result, role)
        )
        db.commit()

        return True, None

    @staticmethod
    def get_game_history_by_player(db, user_id):
        """
```

```
        Retrieve all game history records for a user.

        Args:
            db: The database connection object.
            user_id (int): The ID of the user.

        Returns:
            tuple: (True, rows) if records are found, otherwise (False, error_message).
        """
        result = db.execute(
            "SELECT * FROM GameHistory WHERE user_id = %s",
            (user_id,)
        )

        rows = result.fetchall()
        if not rows:
            return False, "No Game History Found"
        return True, rows

    @staticmethod
    def get_game_history_by_game(db, game_id):
        """
        Retrieve all game history records for a game.

        Args:
            db: The database connection object.
            game_id (int): The ID of the game.

        Returns:
            tuple: (True, rows) if records are found, otherwise (False, error_message).
        """
        result = db.execute(
            "SELECT * FROM GameHistory WHERE game_id = %s",
            (game_id,)
        )

        rows = result.fetchall()
        if not rows:
            return False, "No Game History Found"
        return True, rows

    @staticmethod
    def delete_all_game_history_by_user(db, user_id):
        """
        Delete all game history records for a user.

        Args:
            db: The database connection object.
            user_id (int): The ID of the user.

        Returns:
            tuple: (True, None) after deletion.
        """
        db.execute(
            "DELETE FROM GameHistory WHERE user_id = %s",
```

```
            (user_id,)
        )
        db.commit()

        return True, None

    @staticmethod
    def update_game_history(db, user_id, game_id, result):
        """
        Update a user's result for a specific game.

        Args:
            db: The database connection object.
            user_id (int): The ID of the user.
            game_id (int): The ID of the game.
            result (str): The updated game result.

        Returns:
            tuple: (True, None) after the record is updated.
        """
        db.execute(
            "UPDATE GameHistory SET result = %s WHERE user_id = %s AND game_id = %s",
            (result, user_id, game_id)
        )
        db.commit()

        return True, None

    @staticmethod
    def create_friends_list(db, user_id, friend_id):
        """
        Create a pending friend request.

        Args:
            db: The database connection object.
            user_id (int): The ID of the user sending the request.
            friend_id (int): The ID of the user receiving the request.

        Returns:
            tuple: (True, None) after the request is stored.
        """
        db.execute(
            "INSERT INTO FriendsList(user_id, friend_id) VALUES (%s, %s)",
            (user_id, friend_id)
        )
        db.commit()

        return True, None

    @staticmethod
    def update_friend_status(db, sender_id, receiver_id, status):
        """
        Update the status of a pending friend request.

        If the request is accepted, a matching friend record is also created
```

```
        for the receiving user.

        Args:
            db: The database connection object.
            sender_id (int): The ID of the user who sent the request.
            receiver_id (int): The ID of the user responding to the request.
            status (str): The updated request status.

        Returns:
            tuple: (True, None) if updated, otherwise (False, error_message).
        """
        result = db.execute(
            """
            UPDATE FriendsList
            SET status = %s
            WHERE user_id = %s AND friend_id = %s AND status = 'Pending'
            """,
            (status, sender_id, receiver_id)
        )

        if result.rowcount == 0:
            return False, "Request Not Found Or Not Authorised"

        db.execute(
            """
            INSERT INTO FriendsList(user_id, friend_id, status)
            VALUES (%s, %s, %s)
            """,
            (receiver_id, sender_id, status)
        )

        db.commit()

        return True, None

    @staticmethod
    def get_friends(db, user_id):
        """
        Retrieve all accepted friends for a user.

        Args:
            db: The database connection object.
            user_id (int): The ID of the user.

        Returns:
            tuple: (True, rows) if friends are found, otherwise (False, error_message).
        """
        result = db.execute(
            "SELECT * FROM FriendsList WHERE user_id = %s AND status = %s",
            (user_id, "Accepted")
        )

        rows = result.fetchall()
        if not rows:
            return False, "No Friends Found"
```

```
        return True, rows

    @staticmethod
    def get_friends_by_status(db, user_id, status):
        """
        Retrieve friends or friend requests for a user by status.

        Args:
            db: The database connection object.
            user_id (int): The ID of the user.
            status (str): The friend request status to filter by.

        Returns:
            tuple: (True, rows) if records are found, otherwise (False, error_message).
        """
        result = db.execute(
            "SELECT * FROM FriendsList WHERE user_id = %s AND status = %s",
            (user_id, status)
        )

        rows = result.fetchall()
        if not rows:
            return False, "No Requests Found"
        return True, rows

    @staticmethod
    def delete_friend(db, user_id, friend_id):
        """
        Delete a friend record between two users.

        Args:
            db: The database connection object.
            user_id (int): The ID of the user.
            friend_id (int): The ID of the friend.

        Returns:
            tuple: (True, None) after deletion.
        """
        db.execute(
            "DELETE FROM FriendsList WHERE user_id = %s AND friend_id = %s",
            (user_id, friend_id)
        )
        db.commit()

        return True, None

    @staticmethod
    def delete_friends_by_user(db, user_id):
        """
        Delete all friend records for a user.

        Args:
            db: The database connection object.
            user_id (int): The ID of the user.
```

```
        Returns:
            tuple: (True, None) after deletion.
        """
        db.execute(
            "DELETE FROM FriendsList WHERE user_id = %s",
            (user_id,)
        )
        db.commit()

        return True, None
```

## DatabaseService class:

The DatabaseService class manages the connection between database repositories, abstracts and adds higher level logic fo rtthe repositories, supporting authentication and persistence objectives.

```
from SQLConnections.DBConnections import DBConnection
from SQLConnections.UserRepository import UserRepository
from SQLConnections.GameRepository import GameRepository
from SQLConnections.MoveRepository import MoveRepository
from SQLConnections.LinkRepository import LinkRepository
from SQLConnections.Hashing import hash_password

class DatabaseService:
    """
    Acts as the service layer between the program and the repositories.

    This class combines repository methods where extra logic is needed,
    such as validating users, checking passwords, and formatting results.
    """

    def __init__(self):
        """
        Create the database service and open a database connection.
        """
        self.db = DBConnection()

    def close(self):
        """
        Close the database connection.
        """
        self.db.close()

    def sign_up(self, username, password, email):
        """
        Create a new user account.
        """
        return UserRepository.create_user(self.db, username, password, email)

    def log_in(self, username, password):
        """
        Log a user in by checking the entered password against the stored hash.
        """
        user, error = UserRepository.get_user_by_username(self.db, username)
```

```
        if user is None:
            return None, error

        stored_hash = user["password"]
        salt = user["salt"]

        _, hashed_password = hash_password(password, salt)

        if hashed_password == stored_hash:
            return user, None

        return None, "Incorrect Password"

    def get_user_by_id(self, user_id):
        """
        Retrieve a user by their ID.
        """
        return UserRepository.get_user_by_id(self.db, user_id)

    def change_password(self, user_id, old_password, new_password):
        """
        Change a user's password.
        """
        return UserRepository.change_password(self.db, user_id, old_password, new_password)

    def change_username(self, user_id, new_username):
        """
        Change a user's username.
        """
        return UserRepository.change_username(self.db, user_id, new_username)

    def delete_user(self, user_id):
        """
        Delete a user and all related records linked to that user.
        """
        MoveRepository.delete_all_moves_by_user(self.db, user_id)
        LinkRepository.delete_friends_by_user(self.db, user_id)
        LinkRepository.delete_all_game_history_by_user(self.db, user_id)

        return UserRepository.delete_user(self.db, user_id)

    def send_friend_request(self, user_id, friend_id):
        """
        Send a friend request from one user to another.
        """
        success, _ = UserRepository.get_user_by_id(self.db, user_id)
        if not success:
            return False, "Requesting User Not Found"

        success, _ = UserRepository.get_user_by_id(self.db, friend_id)
        if not success:
            return False, "Target User Not Found"
```

```
        return LinkRepository.create_friends_list(self.db, user_id, friend_id)

    def accept_friend_request(self, sender_id, receiver_id):
        """
        Accept a pending friend request.
        """
        return LinkRepository.update_friend_status(self.db, sender_id, receiver_id, "Accepted")

    def reject_friend_request(self, user_id, friend_id):
        """
        Reject or remove a pending friend request.
        """
        return LinkRepository.delete_friend(self.db, user_id, friend_id)

    def get_all_users_except(self, user_id):
        """
        Retrieve all users except the current user, excluding users who are
        already friends or already part of a pending request.
        """
        success, rows = UserRepository.get_all_users_except(self.db, user_id)
        if not success:
            return False, rows

        friend_ids = set()
        pending_ids = set()

        success, friends = self.get_friends(user_id)
        if success:
            for friend in friends:
                friend_ids.add(friend["user_id"])

        success, pending = self.get_pending_requests(user_id)
        if success:
            for request in pending:
                pending_ids.add(request["user_id"])

        other_users = []

        for user in rows:
            other_id = user["user_id"]

            if other_id in friend_ids or other_id in pending_ids:
                continue

            other_users.append({
                "user_id": other_id,
                "username": user["username"]
            })

        if other_users == []:
            return False, "No Users Found"

        return True, other_users

    def get_friends(self, user_id):
```

```
        """
        Retrieve all accepted friends for a user.
        """
        success, rows = LinkRepository.get_friends_by_status(self.db, user_id, "Accepted")
        if not success:
            return False, rows

        friends = []

        for row in rows:
            other_id = row["friend_id"]

            success, username = UserRepository.get_username_by_user_id(self.db, other_id)
            if success:
                friends.append({
                    "user_id": other_id,
                    "username": username
                })

        if friends == []:
            return False, "No Friends Found"

        return True, friends

    def get_pending_requests(self, user_id):
        """
        Retrieve all pending friend requests linked to a user.
        """
        success, rows = LinkRepository.get_pending_requests_for_user(self.db, user_id)
        if not success:
            return False, rows

        requests = []

        for row in rows:
            if row["user_id"] == user_id:
                other_id = row["friend_id"]
                direction = "outgoing"
            else:
                other_id = row["user_id"]
                direction = "incoming"

            success, username = UserRepository.get_username_by_user_id(self.db, other_id)
            if success:
                requests.append({
                    "user_id": other_id,
                    "username": username,
                    "direction": direction
                })

        if requests == []:
            return False, "No Pending Requests"

        return True, requests
```

```
    def create_game(self, ruleset, seed):
        """
        Create a new game in the lobby state.
        """
        return GameRepository.create_game(self.db, ruleset, "In Lobby", seed)

    def start_game(self, game_id):
        """
        Mark a game as in progress.
        """
        return GameRepository.change_status(self.db, game_id, "In Progress")

    def get_game(self, game_id):
        """
        Retrieve a game by its ID.
        """
        return GameRepository.get_game(self.db, game_id)

    def get_ruleset(self, game_id):
        """
        Retrieve the stored ruleset for a game.
        """
        return GameRepository.get_ruleset(self.db, game_id)

    def get_seed(self, game_id):
        """
        Retrieve the seed used for a game.
        """
        return GameRepository.get_seed(self.db, game_id)

    def get_games_in_lobby(self):
        """
        Retrieve all games currently in the lobby.
        """
        return GameRepository.get_games_by_status(self.db, "In Lobby")

    def pause_game(self, game_id):
        """
        Mark a game as paused.
        """
        return GameRepository.change_status(self.db, game_id, "Paused")

    def unpause_game(self, game_id):
        """
        Mark a paused game as back in progress.
        """
        return GameRepository.change_status(self.db, game_id, "In Progress")

    def get_paused_games_by_user(self, user_id):
        """
        Retrieve all paused games linked to a user.
        """
        result = self.db.execute(
            """SELECT Games.game_id FROM Games
               JOIN GameHistory ON Games.game_id = GameHistory.game_id
```

```
            WHERE GameHistory.user_id = %s AND Games.status = 'Paused'""",
        (user_id,)
    )

    rows = result.fetchall()
    if rows == []:
        return False, "No Paused Games Found"

    return True, rows

def get_lobbies(self):
    """
    Retrieve all games currently in the lobby.
    """
    return GameRepository.get_games_by_status(self.db, "In Lobby")

def get_game_status(self, game_id):
    """
    Retrieve the current status of a game.
    """
    return GameRepository.get_status(self.db, game_id)

def delete_game(self, game_id):
    """
    Delete a game and all moves linked to it.
    """
    MoveRepository.delete_all_moves_in_game(self.db, game_id)
    return GameRepository.delete_game(self.db, game_id)

def add_player_to_game(self, user_id, game_id, role):
    """
    Add a user to a game's history records.
    """
    return LinkRepository.create_game_history(self.db, user_id, game_id, "In Progress",
role)

def record_game_result(self, user_id, game_id, result):
    """
    Store the final result for a user in a game.
    """
    return LinkRepository.update_game_history(self.db, user_id, game_id, result)

def get_player_history(self, user_id):
    """
    Retrieve the game history for a user.
    """
    success, rows = LinkRepository.get_game_history_by_user(self.db, user_id)
    if not success:
        return False, rows

    history = []

    for row in rows:
        user, error = UserRepository.get_user_by_id(self.db, row["user_id"])
        if user is None:
```

```
            continue

        history.append({
            "game_id": row["game_id"],
            "username": user["username"],
            "result": row["result"],
            "role": row["role"]
        })

    if history == []:
        return False, "No Game History Found"

    return True, history

  def get_game_players(self, game_id):
    """
    Retrieve all users linked to a game.
    """
    return LinkRepository.get_game_history_by_game(self.db, game_id)

    def add_move(self, game_id, user_id, move_type, card=None, cards=None,
meld_index=None):
    """
    Add a move to a game using the next available move number.
    """
    success, move_count = MoveRepository.get_move_count(self.db, game_id)
    if not success:
        return False, move_count

    move_number = move_count + 1

    return MoveRepository.add_move(
        self.db,
        game_id,
        user_id,
        move_number,
        move_type,
        card=card,
        cards=cards,
        meld_index=meld_index
    )

  def get_moves(self, game_id):
    """
    Retrieve all moves stored for a game.
    """
    return MoveRepository.get_moves_by_game(self.db, game_id)

  def get_move_count(self, game_id):
    """
    Retrieve the number of moves stored for a game.
    """
    return MoveRepository.get_move_count(self.db, game_id)

  def get_user_guest_status(self, user_id):
```

```
        """
        Retrieve whether a user is a guest account.
        """
        return UserRepository.get_user_guest_status(self.db, user_id)
```

# Frontend Layer:

## Authentication Decorators:

The authentication decorator functions provide reusable checks for authentication and registration.

```
from flask import jsonify, session
from functools import wraps

def registered_only(f):
    """
    Restrict access to registered users only.

    This decorator checks that:
    - a user is logged in
    - the logged-in account is not a guest account

    If either condition fails, a JSON error response is returned instead
    of allowing the route to continue.
    """
    @wraps(f)
    def wrapper(*args, **kwargs):
        if not session.get("user_id"):
            return jsonify({"error": "Account Required"}), 401

        if session.get("guest", False):
            return jsonify({"error": "Registered Account Required"}), 403

        return f(*args, **kwargs)

    return wrapper

def user_required(f):
    """
    Restrict access to logged-in users only.

    This decorator checks whether a user ID exists in the session.
    If not, the request is rejected with a JSON error response.
    """
    @wraps(f)
    def wrapper(*args, **kwargs):
        if not session.get("user_id"):
            return jsonify({"error": "Account Required"}), 401

        return f(*args, **kwargs)
```

```
    return wrapper
```

## Game Decorators:

The Game Decorator functions control access based on whether the current user is in a game, supporting correct enforcement.

```
from flask import jsonify, session
from functools import wraps

def in_game(f):
    """
    Restrict access to users who are currently in a game.

    This decorator checks whether a game_id exists in the session.
    If not, the request is rejected.
    """
    @wraps(f)
    def wrapper(*args, **kwargs):
        if not session.get("game_id"):
            return jsonify({"error": "Not In A Game"}), 403

        return f(*args, **kwargs)

    return wrapper

def not_in_game(f):
    """
    Restrict access to users who are not currently in a game.

    This decorator ensures the user does not already have a game_id
    stored in their session.
    """
    @wraps(f)
    def wrapper(*args, **kwargs):
        if session.get("game_id"):
            return jsonify({"error": "Already In A Game"}), 403

        return f(*args, **kwargs)

    return wrapper
```

## Socket Decorators:

The Socket Decorators behave in the same way as the previous two, but protect Socket-IO events rather than routes.

```
from functools import wraps
from flask import session
from flask_socketio import emit

def socket_user_required(f):
    """
    Restrict a Socket.IO event to logged-in users only.
```

```
    This decorator checks whether a user ID exists in the session.
    If not, an error event is emitted and the handler is not run.
    """
    @wraps(f)
    def wrapper(*args, **kwargs):
        if not session.get("user_id"):
            emit("error", {"error": "Login Required"})
            return

        return f(*args, **kwargs)

    return wrapper

def socket_registered_only(f):
    """
    Restrict a Socket.IO event to registered users only.

    This decorator checks that:
    - a user is logged in
    - the account is not a guest account

    If either condition fails, an error event is emitted and the
    handler is not run.
    """
    @wraps(f)
    def wrapper(*args, **kwargs):
        if not session.get("user_id"):
            emit("error", {"error": "Login Required"})
            return

        if session.get("guest", False):
            emit("error", {"error": "Registered Account Required"})
            return

        return f(*args, **kwargs)

    return wrapper

def socket_in_game(f):
    """
    Restrict a Socket.IO event to users currently in a game.

    This decorator checks whether a game ID exists in the session.
    If not, an error event is emitted and the handler is not run.
    """
    @wraps(f)
    def wrapper(*args, **kwargs):
        if not session.get("game_id"):
            emit("error", {"error": "Not In A Game"})
            return

        return f(*args, **kwargs)

    return wrapper
```

## Authentication routes:

The authentication routes defines HTTP routes for signup, login, logout, guest access, and account changes.

```
from flask import Blueprint, request, jsonify, session
from FrontEnd.auth_decorators import registered_only, user_required

auth_blueprint = Blueprint("auth", __name__)

def auth_routes(game_service):
    """
    Register all authentication-related routes.

    These routes handle:
    - sign up
    - log in
    - log out
    - guest account creation
    - changing account details
    - deleting an account
    """

    @auth_blueprint.route("/sign_up", methods=["POST"])
    def sign_up():
        """
        Create a new registered user account and log the user in immediately.
        """
        data = request.get_json() or {}

        username = data.get("username", "").strip()
        password = data.get("password", "")
        email = data.get("email", "").strip()

        if not username or not password or not email:
            return jsonify({"error": "Username, password, and email are required"}), 400

        success, message = game_service.sign_up(username, password, email)
        if success:
            message, status_code = _login_user(username, password)
            if status_code >= 400:
                return jsonify({"error": message}), status_code
            return jsonify({"message": message}), status_code

        return jsonify({"error": message}), 400

    @auth_blueprint.route("/login", methods=["POST"])
    def login():
        """
        Log a registered user into an existing account.
        """
        data = request.get_json() or {}
```

```
    username = data.get("username", "").strip()
    password = data.get("password", "")

    if not username or not password:
        return jsonify({"error": "Username and password are required"}), 400

    message, status_code = _login_user(username, password)
    if status_code >= 400:
        return jsonify({"error": message}), status_code

    return jsonify({"message": message}), status_code

@auth_blueprint.route("/log_out", methods=["POST"])
@user_required
def log_out():
    """
    Log the current user out and clear their session.
    """
    user_id = session.get("user_id")
    guest = session.get("guest", False)

    game_service.log_out(user_id, guest)
    session.clear()

    return jsonify({"message": "Logged Out"}), 200

@auth_blueprint.route("/guest", methods=["POST"])
def guest_login():
    """
    Create and log in a guest account.

    If a session already exists, the existing session details are returned.
    """
    if session.get("user_id"):
        return jsonify({
            "message": "Session already exists",
            "user_id": session["user_id"],
            "username": session.get("username"),
            "guest": session.get("guest", False)
        }), 200

    success, user = game_service.create_guest()
    if not success:
        return jsonify({"error": user}), 400

    session["user_id"] = user["user_id"]
    session["username"] = user["username"]
    session["guest"] = True

    return jsonify({
        "message": "Guest Account Created",
        "user_id": user["user_id"],
        "username": user["username"],
        "guest": True
    }), 200
```

```
@auth_blueprint.route("/change_password", methods=["POST"])
@registered_only
def change_password():
    """
    Change the password of the currently logged-in registered user.
    """
    data = request.get_json() or {}

    user_id = session.get("user_id")
    old_password = data.get("old_password")
    new_password = data.get("new_password")

    if not old_password or not new_password:
        return jsonify({"error": "Old password and new password are required"}), 400

        success, error = game_service.change_password(user_id, old_password, new_password)
    if success:
        return jsonify({"message": "Changed Password"}), 200

    return jsonify({"error": error}), 400

@auth_blueprint.route("/change_username", methods=["POST"])
@registered_only
def change_username():
    """
    Change the username of the currently logged-in registered user.
    """
    data = request.get_json() or {}

    user_id = session.get("user_id")
    new_username = data.get("new_username", "").strip()

    if not new_username:
        return jsonify({"error": "New username is required"}), 400

    success, error = game_service.change_username(user_id, new_username)
    if success:
        session["username"] = new_username
        return jsonify({"message": "Username Changed"}), 200

    return jsonify({"error": error}), 400

@auth_blueprint.route("/delete_account", methods=["POST"])
@registered_only
def delete_account():
    """
    Delete the currently logged-in registered user's account.
    """
    success, error = game_service.delete_account(session["user_id"])
    if success:
        session.clear()
        return jsonify({"message": "Account Deleted"}), 200
```

```
        return jsonify({"error": error}), 400

    def _login_user(username, password):
        """
        Authenticate a user and store their details in the session.

        Args:
            username (str): The username entered by the user.
            password (str): The password entered by the user.

        Returns:
            tuple: A message and HTTP status code.
        """
        success, user = game_service.log_in(username, password)

        if success:
            session["user_id"] = user["user_id"]
            session["username"] = user["username"]
            session["guest"] = False
            return "Logged In", 200

        return user, 401
```

## Game Routes:

The Game Routes define HTTP routes for creating, joining, starting, pausing, ending, and rejoining games, supporting the client server game flow,

```
from functools import wraps
from flask import session
from flask_socketio import emit

def socket_user_required(f):
    """
    Restrict a Socket.IO event to logged-in users only.

    This decorator checks whether a user ID exists in the session.
    If not, an error event is emitted and the handler is not run.
    """
    @wraps(f)
    def wrapper(*args, **kwargs):
        if not session.get("user_id"):
            emit("error", {"error": "Login Required"})
            return

        return f(*args, **kwargs)

    return wrapper

def socket_registered_only(f):
    """
    Restrict a Socket.IO event to registered users only.

    This decorator checks that:
    - a user is logged in
```

```
    - the account is not a guest account

    If either condition fails, an error event is emitted and the
    handler is not run.
    """
    @wraps(f)
    def wrapper(*args, **kwargs):
        if not session.get("user_id"):
            emit("error", {"error": "Login Required"})
            return

        if session.get("guest", False):
            emit("error", {"error": "Registered Account Required"})
            return

        return f(*args, **kwargs)

    return wrapper

def socket_in_game(f):
    """
    Restrict a Socket.IO event to users currently in a game.

    This decorator checks whether a game ID exists in the session.
    If not, an error event is emitted and the handler is not run.
    """
    @wraps(f)
    def wrapper(*args, **kwargs):
        if not session.get("game_id"):
            emit("error", {"error": "Not In A Game"})
            return

        return f(*args, **kwargs)

    return wrapper
```

## Game Events:

The Game Events define Socket.IO events, supporting synchronised multiplayer updates, move handling, pausing, and game state broadcasting.

```
from flask_socketio import join_room, emit
from .socket_decorators import socket_in_game, socket_user_required
from flask import session

def game_events(socketio, game_service):
    """
    Register all Socket.IO event handlers related to game activity.

    These events handle:
    - joining a game room
    - retrieving lobby players
    - applying moves
    - pausing a game on disconnect
```

```
    - resuming a paused game
    - sending updated game state to clients
    """

    @socketio.on("join_game")
    @socket_user_required
    @socket_in_game
    def on_join():
        """
        Add the current user to the Socket.IO room for their active game.
        """
        game_id = session.get("game_id")
        if not game_id:
            emit("error", {"error": "No Active Game"})
            return

        join_room(str(game_id))
        emit("message", {"message": "Joined Game Room"})

    @socketio.on("get_lobby_players")
    @socket_user_required
    @socket_in_game
    def on_get_lobby_players():
        """
        Send the current lobby player list to everyone in the game room.
        """
        game_id = session.get("game_id")
        if not game_id:
            emit("error", {"error": "Missing Game ID"})
            return

        emit_lobby_players(game_id)

    def emit_lobby_players(game_id):
        """
        Retrieve the list of players in the lobby and emit it to the game room.

        Args:
            game_id: The ID of the game whose lobby players should be sent.
        """
        success, players = game_service.get_lobby_players(game_id)

        if not success:
            emit("error", {"error": players})
            return

        emit(
            "lobby_players",
            {
                "success": True,
                "game_id": game_id,
                "players": players
            },
            to=str(game_id)
        )
```

```
@socketio.on("apply_move")
@socket_user_required
@socket_in_game
def on_move(data):
    """
    Apply a move made by the current player and then emit the updated game state.
    """
    move = {
        "move_type": data["move_type"],
        "user_id": session["user_id"],
        "card": data.get("card"),
        "cards": data.get("cards"),
        "meld_index": data.get("meld_index")
    }

    success, error = game_service.apply_move(session["game_id"], move)

    if not success:
        emit("error", {"success": False, "error": error})
        return

    emit_game_state()

@socketio.on("disconnect")
def on_disconnect():
    """
    Pause the current game if a user disconnects while in an active game.
    """
    user_id = session.get("user_id")
    game_id = session.get("game_id")

    if not user_id or not game_id:
        return

    success, message = game_service.pause_game(game_id)
    if not success:
        return

    session.pop("game_id", None)

    emit(
        "game_paused",
        {
            "message": "A Player Disconnected. The Game Was Paused."
        },
        to=str(game_id)
    )

@socketio.on("resume_game")
@socket_user_required
@socket_in_game
def on_resume_game():
    """
    Resume a paused game and emit the refreshed game state.
```

```
        """
        success, message = game_service.resume_game(session["game_id"])

        if success:
            emit("message", {"message": message})
            emit_game_state()
            return

        emit("error", {"error": message})

    def emit_game_state():
        """
        Retrieve the latest game state for the current session user
        and emit it to all players in the game room.
        """
        success, state = game_service.get_game_state(session["game_id"], session["user_id"])

        if not success:
            emit("error", {"error": state})
            return

        emit(
            "game_state",
            {"success": True, "state": state},
            to=str(session["game_id"]),
            include_self=True
        )
```

## Social Routes:

The social routes define HTTP routes for displaying the friends list, getting pending request and game history, and supporting social features.

```
from flask import Blueprint, request, jsonify, session
from FrontEnd.auth_decorators import registered_only

social_blueprint = Blueprint("social", __name__)

def social_routes(game_service):

    @social_blueprint.route("/friends", methods = ["GET"])
    @registered_only
    def get_friends():
        success, data = game_service.get_friends(session["user_id"])
        if success:
            return jsonify({"friends": data}), 200
        return jsonify({"error": data}), 400

    @social_blueprint.route("/pending_requests", methods = ["GET"])
    @registered_only
    def get_requests():
        success, data = game_service.get_pending_requests(session["user_id"])
        if success:
            return jsonify({"requests": data}), 200
        return jsonify({"error": data}), 400
```

```
    @social_blueprint.route("/history", methods = ["GET"])
    @registered_only
    def get_player_history():
        success, data = game_service.get_player_history(session["user_id"])
        if success:
            return jsonify({"history": data}), 200
        return jsonify({"error": data}), 400
```

## Social Events:

The Social Events define Socket.IO events, supporting friend requests, lobby invites, and game history updates.

```
from flask_socketio import SocketIO, join_room, emit
from .socket_decorators import socket_user_required, socket_registered_only

from flask import session

def social_events(socketio, game_service):

    @socketio.on("connect")
    def on_connect():
        user_id = session.get("user_id")
        if user_id is not None:
            join_room(f"user_{user_id}")

    @socketio.on("friend_request")
    @socket_registered_only
    def on_friend_request(data):
        success, error = game_service.send_friend_request(session["user_id"],
data["friend_id"])
        if not success:
            emit("error", {"error" : error})
            return
        emit("friend_request_received", {"from_id": session["user_id"], "from_username":
session["username"]}, to=f"user_{data['friend_id']}")

    @socketio.on("accept_request")
    @socket_registered_only
    def on_accept_request(data):
        success, error = game_service.accept_friend_request(session["user_id"],
data["friend_id"])
        if not success:
            emit("error", {"error": error})
            return
        emit("friend_request_accepted", {"from_id": session["user_id"], "from_username":
session["username"]}, to=f"user_{data['friend_id']}")

    @socketio.on("reject_request")
    @socket_registered_only
    def on_rejest_request(data):
        success, error = game_service.reject_friend_request(session["user_id"],
data["friend_id"])
```

```
        if not success:
            emit("error", {"error": error})
            return
        emit("friend_request_rejected", {"from_id": session["user_id"], "from_username":
session["username"]}, to=f"user_{data['friend_id']}")

    @socketio.on("get_socials")
    @socket_registered_only
    def on_get_socials():
        user_id = session.get("user_id")

        if user_id is None:
            emit("error", {"error": "Missing User Id"})

        success, friends = game_service.get_friends(user_id)
        if not success:
            friends = []

        success, pending = game_service.get_pending_requests(user_id)
        if not success:
            pending = []

        success, others = game_service.get_all_users_except(user_id)
        if not success:
            others = []

        emit("social_list",
            {
                "success": True,
                "friends": friends,
                "pending": pending,
                "others": others
            })


    @socketio.on("get_history")
    @socket_registered_only
    def on_get_history():
        user_id = session.get("user_id")

        success, history = game_service.get_player_history(user_id)
        if not success:
            emit("game_history", {"success": False, "history": [], "error": history})
            return

        emit("game_history", {"success": True, "history": history})

    @socketio.on("invite_to_lobby")
    @socket_registered_only
    def on_invite_to_lobby(data):
        user_id = session.get("user_id")
        username = session.get("username")
        friend_id = data.get("friend_id")
        game_id = session.get("game_id")
```

```
    if not user_id or not friend_id or not game_id:
        emit("error", {"error": "Missing data"})
        return

    success, error = game_service.send_lobby_invite(user_id, friend_id, game_id)
    if not success:
        emit("error", {"error": error})
        return

    emit(
        "lobby_invite_received",
        {
            "from_id": user_id,
            "from_username": username,
            "game_id": game_id
        },
        to=f"user_{friend_id}"
    )
@socketio.on("accept_lobby_invite")
@socket_user_required
def on_accept_lobby_invite(data):
    user_id = session.get("user_id")
    game_id = data.get("game_id")

    if not user_id or not game_id:
        emit("error", {"error": "Missing data"})
        return

    game_service.add_player_to_game(user_id, game_id, "Player")

    emit(
        "lobby_joined",
        {"game_id": game_id},
    )

@socketio.on("ready")
@socket_user_required
def on_ready(data):
    user_id = session.get("user_id")
    game_id = data.get("game_id")
    success, error = game_service.ready(user_id, game_id)
    if not success:
        emit("error", {"error": error})
        return
    emit("message", {"message": "Readied Up"})
    return
```

# Appendix:

## Glossary:

Rummy: A popular card game, featuring melds, laying off, and drawing from the Draw Pile and the discard pile.

Draw pile: The draw pile consists of a collection of face down cards which players draw from during their turn. Specificities depend on the selected rule set.

Discard pile: The discard pile consists of a collection of face down cards which is formed from player discards. Players may optionally draw from it. Specificities depend on the selected rule set.

Meld(s): A meld is a collection of cards that are validated by the active rule set, typically to reduce a player's hand, categorised into sets and runs.

Wilds(s): A wild card that represents itself into another card when forming melds. Specificities depend on the selected rule set.

Ace(s): A card that can be treated as high, low, or both depending on the selected rule set.

Set(s): A category of melds where the consisting cards are the same rank but different suits. Specificities depend on the selected rule set.

Run(s): A category of meld where all cards are sequential and the same suit. Specificities depend on the selected rule set.

Laying off: Adding a card from the players to meld on the table. If it reaches the maximum number of cards for the type of meld, no more cards may be added.

## Bibliography:

[1] CardzMania 2026, *CardzMania - Play 50+ card games online with up to 12 players*. Available at: https://www.cardzmania.com/ (Accessed: February 12, 2026.)